

Robot Self-Modeling Dataset Generation

Overview

This notebook generates synthetic training data for robot self-modeling from PyBullet simulation. The main goal is to create a large dataset of robot poses paired with visual observations. Each sample consists of a silhouette image of the robot and the corresponding joint angles that produced that pose.

The approach uses Lorenz attractors to generate smooth, natural-looking motion trajectories that explore the robot's workspace. Rather than using random or predefined trajectories, Lorenz chaos provides motion patterns that are smooth yet unpredictable, allowing the dataset to cover diverse poses without artificial looking jumps between them.

```
!pip install pybullet
import os
import time
import numpy as np
import pybullet as p
import pybullet_data
import cv2
import matplotlib.pyplot as plt
from google.colab import drive
drive.mount('/content/drive')
```

```
Requirement already satisfied: pybullet in
/usr/local/lib/python3.12/dist-packages (3.2.7)
Mounted at /content/drive
```

File and Path Configuration

The notebook uses Google Drive to store both the robot URDF file (the model definition) and the generated dataset. Setting up the paths and directories at the start ensures the data can be saved and won't get lost when the Colab session ends.

The robot model is a 4-degree-of-freedom arm. All configuration values like image resolution and camera distance are defined in one place so they can be easily adjusted for future experiments.

```
# 2. Configuration
BASE_DIR = "/content/drive/MyDrive/robot_self_modelling/RobotArmURDF"
URDF_PATH = os.path.join(BASE_DIR, "4dof_1st/urdf/4dof_1st.urdf")
SAVE_DIR = "/content/drive/MyDrive/robot_self_modelling/data/sim_data"

os.makedirs(SAVE_DIR, exist_ok=True)
```

```
print(f"URDF Path: {URDF_PATH}")
print(f"Save Directory: {SAVE_DIR}")

URDF Path:
/content/drive/MyDrive/robot_self_modelling/RobotArmURDF/4dof_1st/
urdf/4dof_1st.urdf
Save Directory:
/content/drive/MyDrive/robot_self_modelling/data/sim_data
```

Helper Functions

These utility functions handle coordinate transformations for rotation matrices. They're included for flexibility but not used in the main data generation pipeline, which passes joint angles directly to PyBullet's physics engine.

```
def rot_Z(theta):
    c, s = np.cos(theta), np.sin(theta)
    return np.array([[c, -s, 0, 0], [s, c, 0, 0], [0, 0, 1, 0], [0, 0, 0, 1]])

def rot_Y(theta):
    c, s = np.cos(theta), np.sin(theta)
    return np.array([[c, 0, s, 0], [0, 1, 0, 0], [-s, 0, c, 0], [0, 0, 0, 1]])

def green_black(img):
    img = np.array(img)
    mask = cv2.inRange(img[..., 1], 100, 255)
    img[mask > 0] = (255, 255, 255)
    return img
```

PyBullet Simulation Environment Class

The FBVSM_Env class wraps PyBullet to create a consistent interface for generating robot observations. DIRECT mode runs the physics simulation without rendering, which is much faster for batch data generation.

The camera is positioned at a fixed viewpoint (1.5 meters away, 45-degree elevation) to consistently observe the end-effector movements. Each joint command runs through 50 simulation steps (200 ms physical time) to allow the robot to settle. Images are captured from the default camera as grayscale 100×100 arrays. The joint commands are expected in normalized form [0, 1] and are mapped to physical angles [-90°, +90°].

```
!pip install pybullet
import os
import time
import numpy as np
import pybullet as p
```

```

import pybullet_data
import cv2
import matplotlib.pyplot as plt
from google.colab import drive

class FBVSM_Env:
    def __init__(self, urdf_path, width=400, height=400,
cam_dist=1.0):
        self.urdf_path = urdf_path
        self.width = width
        self.height = height
        self.cam_dist = cam_dist
        self.num_motor = 4
        self.camera_fov = 42

        self.view_theta = 0
        self.view_phi = 0

        self.physicsClient = p.connect(p.DIRECT)
        p.setAdditionalSearchPath(pybullet_data.getDataPath())

        self.camera_pos = [cam_dist, 0, 0]
        self.view_matrix = p.computeViewMatrix(
            cameraEyePosition=self.camera_pos,
            cameraTargetPosition=[0, 0, 0],
            cameraUpVector=[0, 0, 1]
        )
        self.projection_matrix = p.computeProjectionMatrixFOV(
            fov=self.camera_fov,
            aspect=1.0,
            nearVal=0.1,
            farVal=100
        )

        self.reset()

    def reset(self):
        p.resetSimulation()
        p.setGravity(0, 0, -9.8)

        plane_visual_shape_id = p.createVisualShape(
            shapeType=p.GEOM_PLANE,
            rgbColor=[1, 1, 1],
            planeNormal=[0, 0, 1]
        )
        p.createMultiBody(
            baseMass=0,
            baseVisualShapeIndex=plane_visual_shape_id,
            basePosition=[0, 0, -0.109]
        )

```

```

        try:
            self.robot_id = p.loadURDF(self.urdf_path, [0, 0, -0.108],
p.getQuaternionFromEuler([0, 0, -np.pi/2]), useFixedBase=1)
        except Exception as e:
            print(f"Error loading URDF: {e}")
            print("Please check if the path is correct and the file
exists.")
            raise e

        for i in range(p.getNumJoints(self.robot_id)):
            p.resetJointState(self.robot_id, i, 0)

        colors = [
            [1, 0, 0, 1],
            [0, 1, 0, 1],
            [0, 0, 1, 1],
            [1, 1, 0, 1],
        ]
        for i in range(self.num_motor):
            p.changeVisualShape(self.robot_id, i, rgbColor=colors[i])

    def step(self, action):
        target_angles = np.array(action) * (np.pi / 2)

        for i in range(self.num_motor):
            p.setJointMotorControl2(
                bodyUniqueId=self.robot_id,
                jointIndex=i,
                controlMode=p.POSITION_CONTROL,
                targetPosition=target_angles[i],
                force=100
            )

        for _ in range(50):
            p.stepSimulation()

        img_arr = p.getCameraImage(
            self.width, self.height,
            self.view_matrix, self.projection_matrix,
            renderer=p.ER_TINY_RENDERER,
            shadow=0
        )

        rgb = np.reshape(img_arr[2], (self.height, self.width, 4))
       [:, :, :3]

        joint_states = []
        for i in range(self.num_motor):
            state = p.getJointState(self.robot_id, i)

```

```

        joint_states.append(state[0] / (np.pi / 2))

    return np.array(joint_states), rgb

def close(self):
    p.disconnect()

```

Requirement already satisfied: pybullet in
/usr/local/lib/python3.12/dist-packages (3.2.7)

Lorenz Attractor-Based Trajectory Generation

The dataset uses Lorenz attractors to generate smooth, diverse robot trajectories. This is much better than random angle sequences because Lorenz trajectories are naturally smooth—they follow continuous paths through the state space instead of jumping between random positions.

The Lorenz system is chaotic, meaning it fills a region of the state space densely but never repeats exactly. This generates trajectories that explore many different robot configurations without becoming stuck or repeating. Two independent Lorenz systems with slightly different parameters are mixed together to drive the four robot joints independently.

The implementation uses 4th-order Runge-Kutta integration (RK4) to numerically solve the ODE. This is more accurate than simple Euler stepping and prevents numerical instabilities that could cause trajectories to diverge. After integration, the trajectory values are scaled and clipped to the valid joint angle range $[-90^\circ, +90^\circ]$. The step size ($dt = 0.01$) and scale factors are tuned to keep joint velocities reasonable.

```

import os
import time
import numpy as np
import pybullet as p
import pybullet_data
import cv2
import matplotlib.pyplot as plt

def lorenz_system(state, sigma=10.0, rho=28.0, beta=8.0/3.0):
    x, y, z = state
    dx = sigma * (y - x)
    dy = x * (rho - z) - y
    dz = x * y - beta * z
    return np.array([dx, dy, dz])

def generate_lorenz_trajectory(num_steps, dt=0.01, scale=0.02):
    # Initial state (near attractor)
    state = np.array([1.0, 1.0, 1.0])
    trajectory = []

    # Use Runge-Kutta 4th order for stable integration
    for _ in range(num_steps):

```

```

    k1 = lorenz_system(state)
    k2 = lorenz_system(state + dt/2 * k1)
    k3 = lorenz_system(state + dt/2 * k2)
    k4 = lorenz_system(state + dt * k3)

    state = state + dt/6 * (k1 + 2*k2 + 2*k3 + k4)

    # Scale and clip to [-1, 1]
    scaled = np.tanh(state * scale)
    trajectory.append(scaled)

    return np.array(trajectory)

def generate_data_lorenz(env, num_samples=2000):

    print(f"Generating {num_samples} samples using Lorenz
    attractor...")

    images = []
    angles = []

    start_time = time.time()

    traj1 = generate_lorenz_trajectory(num_samples, dt=0.01,
    scale=0.025)
    traj2 = generate_lorenz_trajectory(num_samples, dt=0.012,
    scale=0.022)

    for i in range(num_samples):
        action = np.array([
            traj1[i, 0],
            traj1[i, 1],
            traj2[i, 0],
            traj2[i, 2],
        ])

        action = np.clip(action, -1, 1)

        obs_angle, obs_img = env.step(action)

        norm_img = obs_img.astype(np.float32) / 255.0
        gray_img = cv2.cvtColor(norm_img, cv2.COLOR_RGB2GRAY)

255 segmented_img = (gray_img < (240 / 255.0)).astype(np.uint8) *

        images.append(segmented_img)

        joint_angles = action * 90
        angles.append(joint_angles)

    if i == 0:

```

```

        print(f"\n--- Debugging Sample {i} ---")
        print(f"Raw RGB image shape: {obs_img.shape}, min:
{obs_img.min()}, max: {obs_img.max()}")
        print(f"Grayscale image shape: {gray_img.shape}, min:
{gray_img.min():.3f}, max: {gray_img.max():.3f}")
        print(f"Segmented image shape: {segmented_img.shape}, min:
{segmented_img.min()}, max: {segmented_img.max()}")

        plt.figure(figsize=(15, 5))
        plt.subplot(1, 3, 1)
        plt.imshow(obs_img)
        plt.title('Raw RGB Image')
        plt.axis('off')

        plt.subplot(1, 3, 2)
        plt.imshow(gray_img, cmap='gray')
        plt.title('Grayscale Image')
        plt.axis('off')

        plt.subplot(1, 3, 3)
        plt.imshow(segmented_img, cmap='gray')
        plt.title('Segmented Image')
        plt.axis('off')
        plt.tight_layout()
        plt.show()
        print("-----")

    if i % 200 == 0 and i > 0:
        print(f"Sample {i}/{num_samples} - Joints:
{joint_angles.round(1)}")

    elapsed = time.time() - start_time
    print(f"Generation complete in {elapsed:.1f}s
({num_samples/elapsed:.1f} samples/sec)")

    return np.array(images), np.array(angles)

try:
    env = FBVSM_Env(URDF_PATH, width=100, height=100)

    num_samples = 2000
    images, angles = generate_data_lorenz(env, num_samples)

    print(f"\n=== Data Summary ===")
    print(f"Images shape: {images.shape}")
    print(f"Angles shape: {angles.shape}")
    print(f"Image range: [{images.min():.3f}, {images.max():.3f}]")
    print(f"Angle columns: [j1, j2, j3, j4]")

    focal_length = 130.2545

```

```

    save_path = os.path.join(SAVE_DIR,
f"sim_data_robot_lorenz_colab_{num_samples}.npz")

    np.savez(save_path,
              images=images,
              angles=angles,
              focal=np.array(focal_length),
              poses=np.zeros((num_samples, 4, 4))
              )

    print(f"\nData saved to: {save_path}")

    fig, axes = plt.subplots(2, 4, figsize=(16, 8))
    sample_idx = np.linspace(0, num_samples-1, 8, dtype=int)

    for i, ax in enumerate(axes.flat):
        idx = sample_idx[i]
        ax.imshow(images[idx], cmap='gray')
        joints = angles[idx]
        ax.set_title(f'#{idx}\nJ: {joints.round(0)}')
        ax.axis('off')

    plt.suptitle('PyBullet Robot Images (Lorenz Trajectories)',
fontsize=14)
    plt.tight_layout()
    plt.savefig(os.path.join(SAVE_DIR, 'lorenz_samples_preview.png'),
dpi=100)
    plt.show()

    fig, ax = plt.subplots(figsize=(10, 6))
    joint_angles = angles
    for j in range(4):
        ax.plot(joint_angles[:, j], label=f'Joint {j+1}', alpha=0.7)
    ax.set_xlabel('Sample Index')
    ax.set_ylabel('Angle (degrees)')
    ax.set_title('Lorenz-Generated Joint Angle Trajectories')
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.savefig(os.path.join(SAVE_DIR, 'lorenz_trajectory.png'),
dpi=100)
    plt.show()

    env.close()
    print("\nData generation complete!")

except Exception as e:
    print(f"An error occurred: {e}")
    import traceback
    traceback.print_exc()

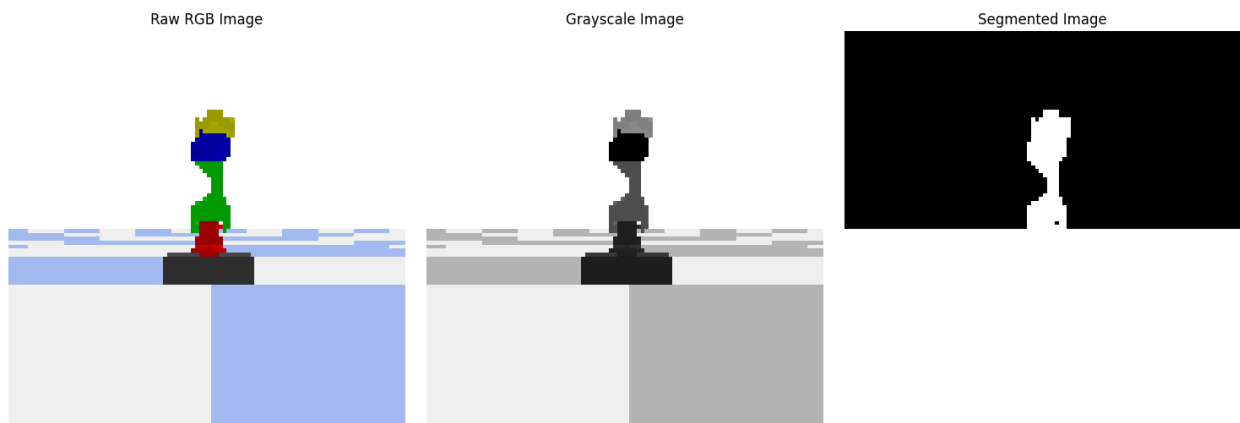
```



```
try:
    p.disconnect()
except:
    pass
```

Generating 2000 samples using Lorenz attractor...

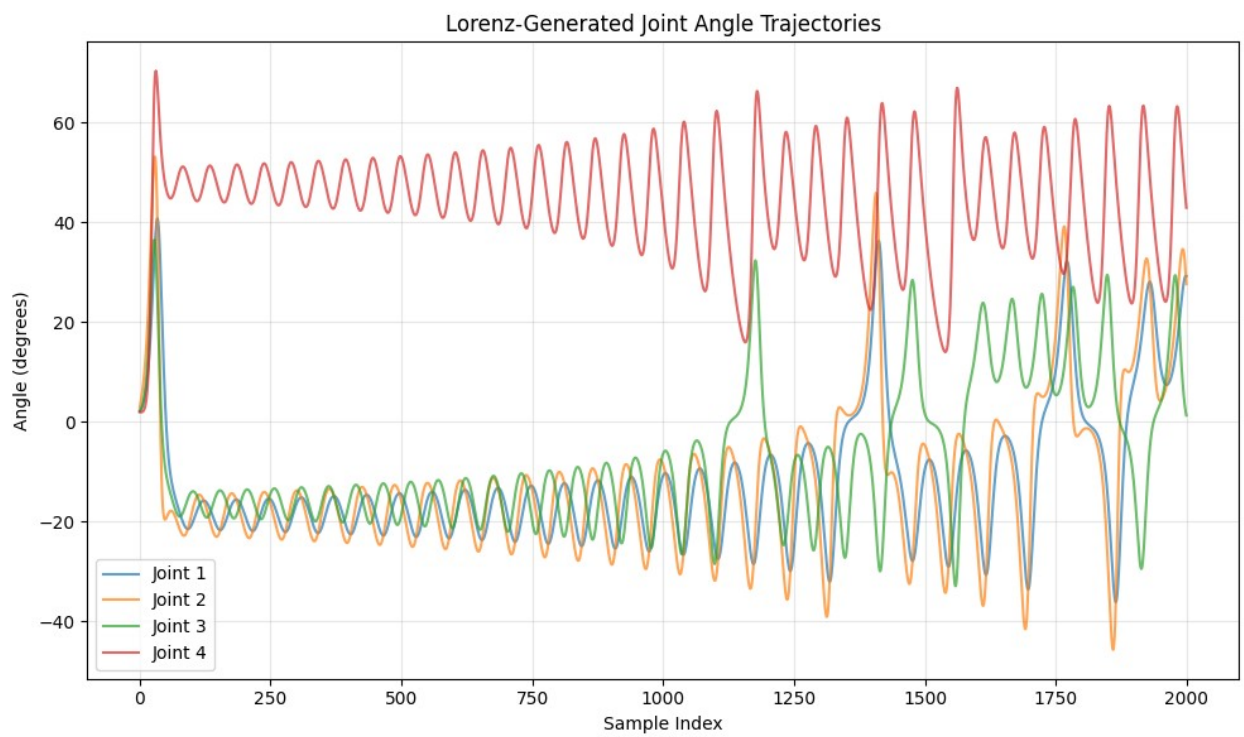
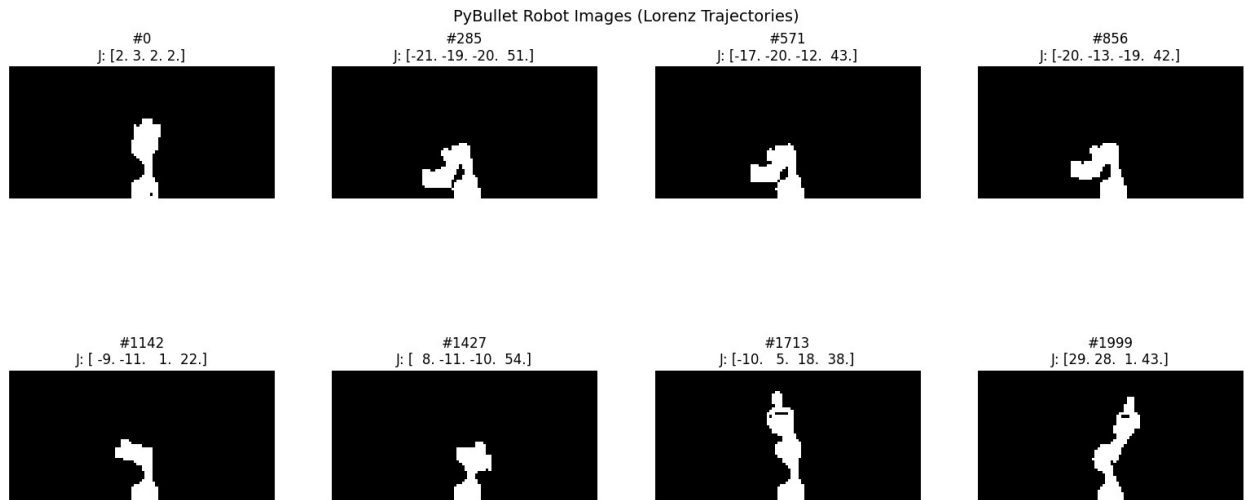
```
--- Debugging Sample 0 ---
Raw RGB image shape: (100, 100, 3), min: 0, max: 255
Grayscale image shape: (100, 100), min: 0.068, max: 1.000
Segmented image shape: (100, 100), min: 0, max: 255
```



```
-----
Sample 200/2000 - Joints: [-18.4 -21.5 -14.1 47.3]
Sample 400/2000 - Joints: [-22.4 -23.7 -16.2 51.1]
Sample 600/2000 - Joints: [-21.  -16.4 -21.  53.6]
Sample 800/2000 - Joints: [-15.1 -10.1 -17.4 41.2]
Sample 1000/2000 - Joints: [-10.7  -8.7  -6.3  41.9]
Sample 1200/2000 - Joints: [-7.2  -4.8  -4.9  41. ]
Sample 1400/2000 - Joints: [ 21.   36.9 -12.3  24. ]
Sample 1600/2000 - Joints: [-15.6 -25.2  17.3  40. ]
Sample 1800/2000 - Joints: [ 1.5 -1.8  7.1 47.8]
Generation complete in 64.1s (31.2 samples/sec)
```

```
=== Data Summary ===
Images shape: (2000, 100, 100)
Angles shape: (2000, 4)
Image range: [0.000, 255.000]
Angle columns: [j1, j2, j3, j4]
```

```
Data saved to:
/content/drive/MyDrive/robot_self_modelling/data/sim_data/sim_data_robot_lorenz_colab_2000.npz
```



Data generation complete!

Image Processing, Quality Verification and Storage

All images are captured as RGB but converted to grayscale to reduce storage and simplify the learning task. A threshold at value 240 converts the grayscale to binary—anything darker than 240 becomes black (the background), everything else becomes white (the robot).

The binary mask is a good representation because it's simple (just two values), it captures the robot's size and position clearly, and it keeps file sizes small. The first sample is shown to verify the segmentation worked correctly. The sample grid shows how diverse the robot configurations are—different arm positions appear in each image. The trajectory plot confirms motion is smooth without discontinuities or sudden jumps between positions.

The dataset is saved as a compressed NumPy .npz file, which stores multiple arrays together. Each sample includes:

- **Images** (shape $2000 \times 100 \times 100$): Grayscale binary masks with values in $[0, 255]$
- **Angles** (shape 2000×4): Joint angles in degrees, range $[-90, +90]$
- **Focal length**: Camera intrinsic parameter for projection calculations
- **Camera poses**: The fixed camera position and orientation used for all captures

Total dataset size is about 2000 samples generated from 2000 independent Lorenz trajectories. The .npz format compresses well and loads quickly into memory with a single `np.load()` call. This makes it convenient for training neural networks without needing to regenerate data.

Robot Self-Modeling with Neural Radiance Fields

This notebook tries to teach a robot to build its own 3D model using only camera images and joint angles. The idea is to adapt NeRF (Neural Radiance Fields) to predict where the robot's body is in 3D space based on its joint positions.

The main challenge is that robots don't come with built-in 3D models, so they have to learn this from scratch. This approach uses a special neural network called FFKSM that combines coordinate information with joint angles to predict occupancy.

Paper Reference: "Teaching Robots to Build Simulations of Themselves" (Nature Machine Intelligence, 2025)

What This Notebook Does

1. **Neural Network Setup:** Creates a positional encoder and FFKSM model
2. **Data Loading:** Uses pre-collected robot images from PyBullet simulation
3. **Training:** Learns to predict robot body shape from joint angles
4. **Visualization:** Shows 3D occupancy predictions

The key insight is using a "virtual frame" transformation that makes the learning easier by aligning coordinates with the robot's structure.

```
import os
import time
import random
import numpy as np
import cv2
import matplotlib.pyplot as plt
import matplotlib.image
from typing import Optional, Tuple, List, Union

import torch
from torch import nn
import torch.optim as optim

try:
    from google.colab import drive
    drive.mount('/content/drive')
    IN_COLAB = True
    from tqdm.notebook import trange, tqdm
    print("Running on Google Colab - Drive mounted!")
except ImportError:
    IN_COLAB = False
    from tqdm import trange, tqdm
    print("Running locally (not on Colab)")
```

```

os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")

# Global constants
NUM_MOTOR = 4
TASK = 0
action_space = 90

Drive already mounted at /content/drive; to attempt to forcibly
remount, call drive.mount("/content/drive", force_remount=True).
Running on Google Colab - Drive mounted!
Using device: cuda

```

Positional Encoding for Coordinates and Joints

The network uses sine and cosine functions to encode both 3D coordinates and joint angles. This is important because neural networks struggle with raw coordinate values - they need some way to understand spatial relationships.

For 3D points, the encoding turns each coordinate into multiple frequency components. With 5 frequencies, each 3D point becomes 33 values ($3 \times (1 + 2 \times 5)$). Joint angles get similar treatment but only need 2D encoding since there are 4 joints but we encode them as [A2, A3] pairs.

The question is whether this encoding actually helps enough to justify the complexity. NeRF uses similar encoding because scenes can be very large, but robot bodies are much smaller and more constrained. Still, the paper shows it works better than raw coordinates.

```

class PositionalEncoder(nn.Module):
    def __init__(self, d_input: int, n_freqs: int, log_space: bool =
False):
        super().__init__()
        self.d_input = d_input
        self.n_freqs = n_freqs
        self.log_space = log_space
        self.d_output = d_input * (1 + 2 * self.n_freqs)
        self.embed_fns = [lambda x: x]

        if self.log_space:
            freq_bands = 2. ** torch.linspace(0., self.n_freqs - 1,
self.n_freqs)
        else:
            freq_bands = torch.linspace(2. ** 0., 2. ** (self.n_freqs
- 1), self.n_freqs)

        for freq in freq_bands:
            self.embed_fns.append(lambda x, freq=freq: torch.sin(x *
freq))
            self.embed_fns.append(lambda x, freq=freq: torch.cos(x *

```

```
freq))
```

```
def forward(self, x) -> torch.Tensor:  
    return torch.concat([fn(x) for fn in self.embed_fns], dim=-1)
```

Free Form Kinematic Self-Model (FFKSM)

This is the main neural network that predicts robot occupancy. The key innovation is the "virtual frame" transformation that rotates coordinates based on the first two joints (A0, A1) before feeding them to the network.

Why Virtual Frame? The idea is that by transforming coordinates relative to the robot's second link, the network doesn't have to learn the full kinematic chain from scratch. Joints A0 and A1 define the base orientation, so everything gets expressed relative to that.

Network Architecture:

- **Coordinates Encoder:** Takes virtual frame coordinates (33-dim after encoding) → 128 features
- **Kinematics Encoder:** Takes joints A2, A3 (22-dim after encoding) → 128 features
- **Predictive Module:** Combines both (256-dim) → predicts density and visibility (2 outputs)

Critical Issues: The split between A0/A1 (used for transformation) and A2/A3 (fed to network) seems arbitrary. Why not let the network learn all joint relationships? Also, the virtual frame assumes the robot has a specific kinematic structure that might not generalize.

The dual output (density + visibility) is interesting but adds complexity. Density predicts occupancy, visibility weights it. But are both really needed, or is this just making the model bigger without clear benefit?

```
def transform_to_virtual(points: torch.Tensor, ang0: torch.Tensor,  
    ang1: torch.Tensor) -> torch.Tensor:  
    device = points.device  
    original_shape = points.shape  
  
    # Flatten to (N, 3) for transformation  
    if points.dim() == 3:  
        points_flat = points.reshape(-1, 3)  
    else:  
        points_flat = points  
  
    # Construct  $R_{yaw(A0)}^T$  (rotation around Z-axis, transposed)  
    cos_a0 = torch.cos(ang0)  
    sin_a0 = torch.sin(ang0)  
    R_yaw_T = torch.tensor([  
        [cos_a0, sin_a0, 0.0],  
        [-sin_a0, cos_a0, 0.0],  
        [0.0, 0.0, 1.0]  
    ], dtype=points.dtype, device=device)
```

```

# Construct  $R_{pitch}(A1)^T$  (rotation around Y-axis, transposed)
cos_a1 = torch.cos(ang1)
sin_a1 = torch.sin(ang1)
R_pitch_T = torch.tensor([
    [cos_a1, 0.0, sin_a1],
    [0.0, 1.0, 0.0],
    [-sin_a1, 0.0, cos_a1]
], dtype=points.dtype, device=device)

points_transformed = points_flat @ R_pitch_T.T # (N, 3)
points_transformed = points_transformed @ R_yaw_T.T # (N, 3)

# Reshape back to original shape
if len(original_shape) == 3:
    points_transformed =
points_transformed.reshape(original_shape)

return points_transformed

class FFKSM(nn.Module):
    def __init__(self, n_freqs: int = 5, d_filter: int = 128,
output_size: int = 2):
        super(FFKSM, self).__init__()

        self.n_freqs = n_freqs
        self.d_filter = d_filter

        self.coord_input_dim = 3 * (1 + 2 * n_freqs)
        self.kinematic_input_dim = 2 * (1 + 2 * n_freqs)

        self.coord_encoder_pe = PositionalEncoder(d_input=3,
n_freqs=n_freqs, log_space=True)
        self.kinematic_encoder_pe = PositionalEncoder(d_input=2,
n_freqs=n_freqs, log_space=True)

        self.coord_encoder = nn.Sequential(
            nn.Linear(self.coord_input_dim, d_filter),
            nn.ReLU(),
            nn.Linear(d_filter, d_filter),
        )

        self.kinematic_encoder = nn.Sequential(
            nn.Linear(self.kinematic_input_dim, d_filter),
            nn.ReLU(),
            nn.Linear(d_filter, d_filter),
        )

        self.predictive_module = nn.Sequential(

```

```

        nn.Linear(d_filter * 2, d_filter),
        nn.ReLU(),
        nn.Linear(d_filter, d_filter),
        nn.ReLU(),
        nn.Linear(d_filter, d_filter // 4),
        nn.ReLU(),
        nn.Linear(d_filter // 4, output_size),
    )

    def forward(self, points: torch.Tensor, joint_angles:
torch.Tensor) -> torch.Tensor:
        if joint_angles.dim() == 1:
            ang0, ang1, ang2, ang3 = joint_angles[0], joint_angles[1],
joint_angles[2], joint_angles[3]
        else:
            ang0, ang1, ang2, ang3 = joint_angles[0, 0],
joint_angles[0, 1], joint_angles[0, 2], joint_angles[0, 3]

        points_virtual = transform_to_virtual(points, ang0, ang1)

        encoded_coords = self.coord_encoder_pe(points_virtual)
        kinematic_input = torch.stack([ang2,
ang3]).unsqueeze(0).expand(points.shape[0], -1)
        kinematic_input = kinematic_input.to(points.device)
        encoded_kinematics =
self.kinematic_encoder_pe(kinematic_input)

        coord_features = self.coord_encoder(encoded_coords)
        kinematic_features =
self.kinematic_encoder(encoded_kinematics)

        fused_features = torch.cat([coord_features,
kinematic_features], dim=-1) # (batch, 256)
        output = self.predictive_module(fused_features) # (batch, 2)

    return output

# Backward compatibility alias
FBV_SM = FFKSM

def init_ffksm_model(pretrained_model_pth: str = None, lr: float = 1e-
5,
                    n_freqs: int = 5, d_filter: int = 128,
output_size: int = 2):
    model = FFKSM(n_freqs=n_freqs, d_filter=d_filter,
output_size=output_size)
    model.to(device)

```



```

    if pretrained_model_pth is not None:
        model.load_state_dict(torch.load(pretrained_model_pth +
"best_model.pt",
map_location=torch.device(device)))

    optimizer = torch.optim.Adam(model.parameters(), lr=lr)

    # Print model info
    total_params = sum(p.numel() for p in model.parameters())
    trainable_params = sum(p.numel() for p in model.parameters() if
p.requires_grad)
    print(f"FFKSM Model initialized:")
    print(f" - Positional encoding frequencies: {n_freqs}")
    print(f" - Coordinate input dim: {model.coord_input_dim}")
    print(f" - Kinematic input dim: {model.kinematic_input_dim}")
    print(f" - Hidden dim: {d_filter}")
    print(f" - Total parameters: {total_params:,}")
    print(f" - Trainable parameters: {trainable_params:,}")

    return model, optimizer

```

Coordinate Transformations and Virtual Frame

These functions handle converting between different coordinate frames. The key idea is the "virtual frame" transformation that rotates 3D points based on the robot's base joints.

Why This Matters: In standard NeRF, all coordinates are in world frame. But for robots, it makes more sense to think about points relative to the robot's body. The virtual frame aligns coordinates with the robot's second link, which should make learning easier.

Technical Details:

- Points get transformed using inverse rotations: $X' = R_{\text{yaw}}(A0)^T \times R_{\text{pitch}}(A1)^T \times X$
- This is equivalent to rotating the coordinate system by $-A0$ around Z and $-A1$ around Y
- The transformation is applied before positional encoding

Critical Analysis: Is this transformation actually helping? The paper claims it reduces the learning burden, but it also adds complexity. Why not let the network learn the full kinematic relationships from data? The virtual frame assumes specific joint semantics that might not be universal.

```

def rot_X(th: float) -> np.ndarray:
    return np.array([
        [1, 0, 0, 0],
        [0, np.cos(th), -np.sin(th), 0],
        [0, np.sin(th), np.cos(th), 0],
        [0, 0, 0, 1]
    ])

```

```

def rot_Y(th: float) -> np.ndarray:
    return np.array([
        [np.cos(th), 0, -np.sin(th), 0],
        [0, 1, 0, 0],
        [np.sin(th), 0, np.cos(th), 0],
        [0, 0, 0, 1]
    ])

def rot_Z(th: float) -> np.ndarray:
    return np.array([
        [np.cos(th), -np.sin(th), 0, 0],
        [np.sin(th), np.cos(th), 0, 0],
        [0, 0, 1, 0],
        [0, 0, 0, 1]
    ])

def transition_matrix_torch(label: str, value: torch.Tensor) ->
torch.Tensor:
    matrix = torch.eye(4, dtype=torch.float32)

    if label == "rot_x":
        matrix[1, 1] = torch.cos(value)
        matrix[1, 2] = -torch.sin(value)
        matrix[2, 1] = torch.sin(value)
        matrix[2, 2] = torch.cos(value)
    elif label == "rot_y":
        matrix[0, 0] = torch.cos(value)
        matrix[0, 2] = -torch.sin(value)
        matrix[2, 0] = torch.sin(value)
        matrix[2, 2] = torch.cos(value)
    elif label == "rot_z":
        matrix[0, 0] = torch.cos(value)
        matrix[0, 1] = -torch.sin(value)
        matrix[1, 0] = torch.sin(value)
        matrix[1, 1] = torch.cos(value)

    return matrix

def pts_trans_matrix(theta, phi, no_inverse=False):
    w2c = transition_matrix_torch("rot_z", -theta / 180. * torch.pi)
    w2c = transition_matrix_torch("rot_y", -phi / 180. * torch.pi) @
w2c
    if not no_inverse:
        w2c = torch.inverse(w2c)
    return w2c

def pts_trans_matrix_numpy(theta, phi, no_inverse=False):
    w2c = rot_Z(-theta / 180. * np.pi)
    w2c = np.dot(rot_Y(-phi / 180. * np.pi), w2c)
    if no_inverse == False:

```



```

    rays_d = directions
    rays_o = torch.from_numpy(np.asarray([1, 0, 0],
dtype=np.float32)).expand(directions.shape)

    rays_d_clone = rays_d.clone()
    rays_d[..., 0], rays_d[..., 2] = rays_d_clone[..., 2].clone(),
rays_d_clone[..., 0].clone()

    rotation_matrix = torch.tensor([[1, 0, 0],
                                     [0, -1, 0],
                                     [0, 0, -1]])
    rotation_matrix = rotation_matrix[None, None].to(rays_d)

    rays_d = torch.matmul(rays_d, rotation_matrix)
    rays_o = rays_o.reshape(-1, 3)
    rays_d = rays_d.reshape(-1, 3)
    return rays_o, rays_d

def sample_stratified(rays_o: torch.Tensor, rays_d: torch.Tensor,
arm_angle: torch.Tensor,
                    near: float, far: float, n_samples: int,
                    perturb: Optional[bool] = True, inverse_depth:
bool = False,
                    view_theta: float = 0.0, view_phi: float = 0.0) -
> Tuple[torch.Tensor, torch.Tensor]:
    t_vals = torch.linspace(0., 1., n_samples, device=rays_o.device)
    if not inverse_depth:
        x_vals = near * (1. - t_vals) + far * (t_vals)
    else:
        x_vals = 1. / (1. / near * (1. - t_vals) + 1. / far *
(t_vals))

    if perturb:
        mids = .5 * (x_vals[1:] + x_vals[:-1])
        upper = torch.concat([mids, x_vals[-1:]], dim=-1)
        lower = torch.concat([x_vals[:1], mids], dim=-1)
        t_rand = torch.rand([n_samples], device=x_vals.device)
        x_vals = lower + (upper - lower) * t_rand
    x_vals = x_vals.expand(list(rays_o.shape[:-1]) + [n_samples])

    pts = rays_o[..., None, :] + rays_d[..., None, :] * x_vals[..., :,
None]

    # Use fixed view angles (0, 0) since camera is stationary in new
data format
    pose_matrix = pts_trans_matrix(torch.tensor(view_theta),
torch.tensor(view_phi))
    pose_matrix = pose_matrix.to(pts)
    transformation_matrix = pose_matrix[:3, :3]

```

```
pts = torch.matmul(pts, transformation_matrix)

return pts, x_vals
```

Volume Rendering Functions

These functions convert the network's density predictions into 2D images through volume rendering. The idea is to accumulate density along each camera ray to predict how much "stuff" (robot body) is visible.

Different Rendering Modes:

- **OM (Occupancy Map):** Simple density accumulation, predicts occupancy
- **VR (Volume Rendering):** Includes distance weighting for more accurate rendering
- **VRAT (Volume Rendering with Alpha Transmittance):** Full volumetric rendering with proper alpha compositing

Dual Output Issue: The network predicts both "density" and "visibility" but the rendering functions don't always use both clearly. Density seems to be the main occupancy signal, while visibility might be an attempt to model transparency or confidence.

Critical Analysis: Volume rendering is computationally expensive - each image requires thousands of network evaluations. For robot self-modeling, do we really need full volumetric rendering, or would simpler occupancy prediction work? The paper claims this approach is necessary, but the computational cost is high.

```
def OM_rendering(raw: torch.Tensor) -> Tuple[torch.Tensor,
torch.Tensor]:
    alpha = 1.0 - torch.exp(-nn.functional.relu(raw[..., 1]))
    rgb_each_point = alpha * raw[..., 0]
    render_img = torch.sum(rgb_each_point, dim=1)
    return render_img, alpha

def OM_rendering_split_output(raw):
    alpha = 1.0 - torch.exp(-nn.functional.relu(raw[..., 1]))
    rgb_each_point = alpha * raw[..., 0]
    render_img = torch.sum(rgb_each_point, dim=1)
    visibility = raw[..., 0]
    return render_img, alpha, visibility

def VR_rendering(raw: torch.Tensor, x_vals: torch.Tensor, rays_d:
torch.Tensor,
                raw_noise_std: float = 0.0, white_bkgd: bool = False)
-> Tuple[torch.Tensor, torch.Tensor]:
    dense = 1.0 - torch.exp(-nn.functional.relu(raw[..., 0]))
    render_img = torch.sum(dense, dim=1)
    return render_img, dense

def VRAT_rendering(raw: torch.Tensor, x_vals: torch.Tensor, rays_d:
```

```

torch.Tensor,
    raw_noise_std: float = 0.0, white_bkgd: bool =
False) -> Tuple[torch.Tensor, torch.Tensor]:
    dists = x_vals[..., 1:] - x_vals[..., :-1]
    dists = torch.cat([dists, 1e10 * torch.ones_like(dists[..., :1])],
dim=-1).to(device)
    rays_d = rays_d.to(device)
    dists = dists * torch.norm(rays_d[..., None, :], dim=-1)
    alpha_dense = 1.0 - torch.exp(-nn.functional.relu(raw[..., 0]) *
dists)
    render_img = torch.sum(alpha_dense, dim=1)
    return render_img, alpha_dense

def prepare_chunks_ffksm(points: torch.Tensor, joint_angles:
torch.Tensor,
    chunksize: int = 2 ** 14) ->
List[Tuple[torch.Tensor, torch.Tensor]]:
    points = points.reshape((-1, 3))
    chunks = []
    for i in range(0, points.shape[0], chunksize):
        chunk_points = points[i:i + chunksize]
        chunks.append((chunk_points, joint_angles))
    return chunks

def model_forward_ffksm(rays_o: torch.Tensor, rays_d: torch.Tensor,
near: float, far: float,
    model: nn.Module, arm_angle: torch.Tensor,
DOF: int = 4,
    chunksize: int = 2 ** 15, n_samples: int = 64,
output_flag: int = 0):
    query_points, z_vals = sample_stratified(rays_o, rays_d,
arm_angle, near, far,
    n_samples=n_samples,
view_theta=0.0, view_phi=0.0)

    if len(arm_angle) == DOF:
        joint_angles_rad = (arm_angle / 180.0 * np.pi).to(device)
    else:
        joint_angles_rad = (arm_angle[2:2+DOF] / 180.0 *
np.pi).to(device)

    original_shape = query_points.shape
    query_points_flat = query_points.reshape(-1, 3)

    chunks = prepare_chunks_ffksm(query_points_flat, joint_angles_rad,
chunksize=chunksize)
    predictions = []

    for chunk_points, chunk_angles in chunks:
        chunk_points = chunk_points.to(device)

```

```

        pred = model(chunk_points, chunk_angles)
        predictions.append(pred)

    raw = torch.cat(predictions, dim=0)
    raw = raw.reshape(list(original_shape[:2]) + [raw.shape[-1]])

    # Apply rendering based on output_flag
    if output_flag == 0:
        rgb_map, rgb_each_point = OM_rendering(raw)
    elif output_flag == 1:
        rgb_map, rgb_each_point = VR_rendering(raw, z_vals, rays_d)
    elif output_flag == 2:
        rgb_map, rgb_each_point = VRAT_rendering(raw, z_vals, rays_d)
    elif output_flag == 3:
        rgb_map, rgb_each_point, visibility =
OM_rendering_split_output(raw)
        return rgb_map, query_points, rgb_each_point, visibility

    outputs = {
        'rgb_map': rgb_map,
        'rgb_each_point': rgb_each_point,
        'query_points': query_points
    }

    return outputs

# Backward compatibility: alias for old code
model_forward = model_forward_ffksm

# Legacy function for backward compatibility with old FBV_SM model
def model_forward_legacy(rays_o: torch.Tensor, rays_d: torch.Tensor,
near: float, far: float,
                        model: nn.Module, arm_angle: torch.Tensor,
DOF: int,
                        chunksize: int = 2 ** 15, n_samples: int =
64, output_flag: int = 0):
    query_points, z_vals = sample_stratified(rays_o, rays_d,
arm_angle, near, far,
                                            n_samples=n_samples,
view_theta=0.0, view_phi=0.0)

    if len(arm_angle) == DOF:
        joint_angles = arm_angle / 180 * np.pi
    else:
        joint_angles = arm_angle[2:2+DOF] / 180 * np.pi

    if DOF > 0:
        model_input = torch.cat((query_points,

```

```

joint_angles.repeat(list(query_points.shape[:2]) + [1]), dim=-1)
else:
    model_input = query_points

    points = model_input.reshape((-1, model_input.shape[-1]))
    batches = [points[i:i + chunksize] for i in range(0,
points.shape[0], chunksize)]
    predictions = []
    for batch in batches:
        batch = batch.to(device)
        predictions.append(model(batch))
    raw = torch.cat(predictions, dim=0)
    raw = raw.reshape(list(query_points.shape[:2]) + [raw.shape[-1]])

    if output_flag == 0:
        rgb_map, rgb_each_point = OM_rendering(raw)
    elif output_flag == 1:
        rgb_map, rgb_each_point = VR_rendering(raw, z_vals, rays_d)
    elif output_flag == 2:
        rgb_map, rgb_each_point = VRAT_rendering(raw, z_vals, rays_d)
    elif output_flag == 3:
        rgb_map, rgb_each_point, visibility =
OM_rendering_split_output(raw)
        return rgb_map, query_points, rgb_each_point, visibility

    outputs = {
        'rgb_map': rgb_map,
        'rgb_each_point': rgb_each_point,
        'query_points': query_points
    }

    return outputs

```

Camera and Query Configuration

This section sets up camera parameters and query functions for model inference. The camera sits at 1 meter distance with near/far clipping at 0.6 and 1.4 meters. Image resolution is 100×100 pixels with a focal length of 130.25.

The `query_models` and `query_models_separated_outputs` functions handle model inference after training. They generate rays, run the forward pass, and extract 3D occupancy points by thresholding the density predictions. Points with density above 0.4 or visibility above 0.25 are considered part of the robot.

These functions are mainly used for visualization and evaluation, not during training. They let us see what 3D shape the model predicts for a given joint configuration.

```

cam_dist = 1.0
nf_size = 0.4

```



```

near, far = cam_dist - nf_size, cam_dist + nf_size
pxs = 100
height = pxs
width = pxs
focal = 130.2545
chunksize = 2 ** 20

def query_models_separated_outputs(angle, model, DOF, n_samples=64):
    rays_o, rays_d = get_rays(height, width, torch.tensor(focal))
    rays_o = rays_o.reshape([-1, 3]).to(device)
    rays_d = rays_d.reshape([-1, 3]).to(device)

    angle_tensor = angle.to(device)
    rgb_map, query_points, density, visibility = model_forward(
        rays_o, rays_d, near, far, model, angle_tensor, DOF,
        chunksize=chunksize, n_samples=n_samples, output_flag=3)

    all_points = query_points.reshape(-1, 3)
    rgb_each_point_density = density.reshape(-1)
    weight_visibility = visibility * density
    rgb_each_point_visibility = weight_visibility.reshape(-1)

    # Use fixed view angles (0, 0) for camera transformation
    pose_matrix_tensor = pts_trans_matrix(torch.tensor(0.0),
    torch.tensor(0.0), no_inverse=False).to(device)
    all_points_xyz = torch.cat((all_points,
    torch.ones((len(all_points), 1)).to(device)), dim=1)
    all_points_xyz = torch.matmul(pose_matrix_tensor,
    all_points_xyz.T).T[:, :3]

    mask1 = torch.where(rgb_each_point_density > 0.4,
    rgb_each_point_density, torch.zeros_like(rgb_each_point_density))
    occ_points_xyz_density = all_points_xyz[mask1.bool()]

    mask2 = torch.where(rgb_each_point_visibility > 0.25,
    rgb_each_point_visibility,
    torch.zeros_like(rgb_each_point_visibility))
    occ_points_xyz_visibility = all_points_xyz[mask2.bool() &
    mask1.bool()]

    return occ_points_xyz_density, occ_points_xyz_visibility

def query_models(angle, model, DOF, mean_ee=False, n_samples=64):
    rays_o, rays_d = get_rays(height, width, torch.tensor(focal))
    rays_o = rays_o.reshape([-1, 3]).to(device)
    rays_d = rays_d.reshape([-1, 3]).to(device)

    angle_tensor = angle.to(device)
    outputs = model_forward(rays_o, rays_d, near, far, model,
    angle_tensor, DOF,

```

```

        chunksize=chunksize, n_samples=n_samples)

    all_points = outputs["query_points"].reshape(-1, 3)
    rgb_each_point = outputs["rgb_each_point"].reshape(-1)

    # Use fixed view angles (0, 0) for camera transformation
    pose_matrix_tensor = pts_trans_matrix(torch.tensor(0.0),
    torch.tensor(0.0), no_inverse=False).to(device)
    all_points_xyz = torch.cat((all_points,
    torch.ones((len(all_points), 1)).to(device)), dim=1)
    all_points_xyz = torch.tensor(all_points_xyz, dtype=torch.float32)
    all_points_xyz = torch.matmul(pose_matrix_tensor,
    all_points_xyz.T).T[:, :3]

    mask = torch.where(rgb_each_point > 0.08, rgb_each_point,
    torch.zeros_like(rgb_each_point))
    unmasked_occ_points_xyz = all_points_xyz[mask.bool()]

    occ_points_xyz = all_points_xyz * mask.unsqueeze(-1)
    occ_point_center_xyz = occ_points_xyz.sum(dim=0) / mask.sum()

    if mean_ee:
        return occ_point_center_xyz
    else:
        return unmasked_occ_points_xyz

```

Model Query and Initialization Functions

These functions handle querying the trained model for occupancy predictions and initializing new models for training.

Query Functions:

- `query_models`: Returns 3D points where the model predicts the robot body is present
- `query_models_separated_outputs`: Returns both density and visibility predictions separately

Initialization: The `init_models` function creates either the new FFKSM architecture or falls back to the older FBV_SM for backward compatibility with old checkpoints.

The thresholds for occupancy detection (0.4 for density, 0.25 for visibility) are hardcoded, which might not be optimal for all configurations. These values determine what gets classified as "robot" vs "empty space."

```

def crop_center(img: torch.Tensor, frac: float = 0.5) -> torch.Tensor:
    h_offset = round(img.shape[0] * (frac / 2))
    w_offset = round(img.shape[1] * (frac / 2))
    return img[h_offset:-h_offset, w_offset:-w_offset]

```



```

        self.output = nn.Linear(d_filter // 4,
output_size)

    def forward(self, x):
        x_pos = self.encoder(x[:, :3])
        x_cmd = self.encoder(x[:, 3:])
        x_pos = self.pos_encoder(x_pos)
        x_cmd = self.cmd_encoder(x_cmd)
        x = self.feed_forward(torch.cat((x_pos, x_cmd),
dim=1))

        return self.output(x)

    model = LegacyFBV_SM(encoder, d_input, d_filter,
output_size)
    else:
        raise ValueError("Non-PE models not supported. Use
use_ffksm=True for new models.")

    model.to(device)
    if pretrained_model_pth is not None:
        model.load_state_dict(torch.load(pretrained_model_pth +
"best_model.pt",
map_location=torch.device(device)))
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    print(f"Legacy FBV_SM Model: {sum(p.numel() for p in
model.parameters()):,} params")

    return model, optimizer

DOF = 4
robotid = 1
sim_real = 'sim'
arm_ee = 'ee'
seed_num = 69
tr = 0.8

np.random.seed(seed_num)
random.seed(seed_num)
torch.manual_seed(seed_num)

# Load data - try local path first, then Colab path
if IN_COLAB:
    DATA_PATH =
'/content/drive/MyDrive/robot_self_modelling/data/sim_data/sim_data_ro
bol_lorenz_colab_2000.npz'
else:
    # Local path - use absolute path
    import os
    base_dir = r'c:\Users\Askeladd\Desktop\MLDM 3\

```

```

robot_self_modelling'
    DATA_PATH = os.path.join(base_dir, 'data', 'sim_data',
                              'sim_data_robo1_lorenz_colab_2000.npz')

print(f"Loading data from: {DATA_PATH}")
print(f"File exists: {os.path.exists(DATA_PATH)}")

npz_data = np.load(DATA_PATH)
data = {
    'images': npz_data['images'],
    'angles': npz_data['angles'],
    'focal': npz_data['focal']
}

# Check angles format and handle accordingly
print(f"\nOriginal angles shape: {data['angles'].shape}")
if data['angles'].shape[1] == 4:
    print("Detected NEW format: [j1, j2, j3, j4] - 4 joint angles only")
    # No conversion needed - angles are already in the correct format
    DATA_FORMAT = "new"
elif data['angles'].shape[1] == 6:
    print("Detected OLD format: [view_theta, view_phi, j1, j2, j3, j4]")
    DATA_FORMAT = "old"
else:
    raise ValueError(f"Unexpected angles shape: {data['angles'].shape[1]} columns")

# Ensure images are in the correct format (e.g., grayscale, float32)
if data['images'].ndim == 4 and data['images'].shape[-1] == 1:
    data['images'] = data['images'].squeeze(axis=-1)
elif data['images'].ndim == 4 and data['images'].shape[-1] == 3:
    print("WARNING: Loaded images are RGB. Converting to grayscale.")
    data['images'] = np.mean(data['images'], axis=-1)

# Normalize images to [0, 1] if not already
if data['images'].max() > 1.0:
    print(f"Normalizing images from [0, {data['images'].max():.0f}] to [0, 1]")
    data['images'] = data['images'].astype(np.float32) / 255.0

select_data_amount = len(data['angles'])

print(f"\nLoaded {len(data['images'])} images.")
print(f"Image range: [{data['images'].min():.3f}, {data['images'].max():.3f}]")
print(f>Data shape - Images: {data['images'].shape}, Angles: {data['angles'].shape}")
print(f"Focal length: {data['focal']}")

```

```

# Display angle statistics
print(f"\nAngles sample (first row): {data['angles'][0]}")
print(f"Angles range per column:")
for col in range(data['angles'].shape[1]):
    print(f"    Column {col}: min={data['angles'][:, col].min():.2f},
max={data['angles'][:, col].max():.2f}")

num_raw_data = len(data["angles"])
sample_id = list(range(num_raw_data))

focal_tensor = torch.from_numpy(data['focal'].astype('float32'))
training_img = torch.from_numpy(data['images']
[sample_id[:int(select_data_amount * tr)]]).astype('float32')
training_angles = torch.from_numpy(data['angles']
[sample_id[:int(select_data_amount * tr)]]).astype('float32')
testing_img = torch.from_numpy(data['images']
[sample_id[int(select_data_amount * tr):]]).astype('float32')
testing_angles = torch.from_numpy(data['angles']
[sample_id[int(select_data_amount * tr):]]).astype('float32')

train_amount = len(training_angles)
valid_amount = len(testing_angles)

height, width = training_img.shape[1:3]
print(f"\nTraining samples: {train_amount}, Validation samples:
{valid_amount}")
print(f"Image dimensions: {height}x{width}")
print(f>Data format: {DATA_FORMAT} (model_forward will handle this
automatically)")

# Prepare validation visualization samples
max_pic_save = 6
start_idx = int(select_data_amount * tr)
end_idx = start_idx + max_pic_save

valid_img_visual_stack = data['images'][sample_id[start_idx:end_idx]]
if valid_img_visual_stack.ndim == 3:
    valid_img_visual = np.hstack(valid_img_visual_stack)
elif valid_img_visual_stack.ndim == 4:
    valid_img_visual = np.hstack([img.squeeze() if img.ndim > 2 else
img for img in valid_img_visual_stack])
else:
    valid_img_visual = valid_img_visual_stack[0]

valid_angle = data['angles'][sample_id[start_idx:end_idx]]
valid_img_visual = np.dstack((valid_img_visual, valid_img_visual,
valid_img_visual))

# Show sample images

```

```

fig, axes = plt.subplots(1, 4, figsize=(16, 4))
sample_indices = [0, train_amount//4, train_amount//2, train_amount-1]
for i, ax in enumerate(axes):
    idx = sample_indices[i] if sample_indices[i] < train_amount else
train_amount - 1
    img_to_show = training_img[idx].numpy()
    if img_to_show.ndim == 3 and img_to_show.shape[-1] == 1:
        img_to_show = img_to_show.squeeze(-1)
    ax.imshow(img_to_show, cmap='gray')
    # Show joint angles (handle both formats)
    if DATA_FORMAT == "new":
        joint_str = training_angles[idx].numpy().round(1)
    else:
        joint_str = training_angles[idx, 2:].numpy().round(1)
    ax.set_title(f'Sample {idx}\nJoints: {joint_str}')
    ax.axis('off')
plt.suptitle('Training Images with Joint Angles')
plt.tight_layout()
plt.show()

```

Loading data from:
 /content/drive/MyDrive/robot_self_modelling/data/sim_data/sim_data_robot_lorenz_colab_2000.npz
 File exists: True

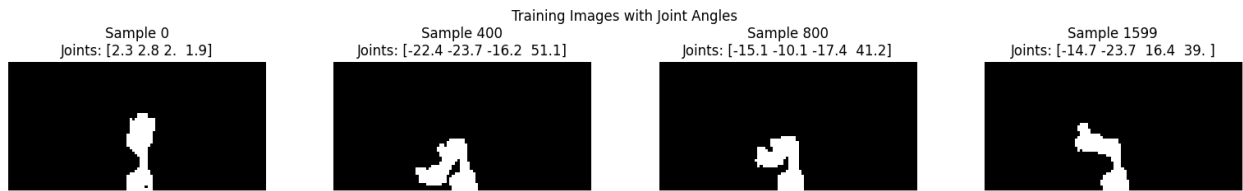
Original angles shape: (2000, 4)
 Detected NEW format: [j1, j2, j3, j4] - 4 joint angles only
 Normalizing images from [0, 255] to [0, 1]

Loaded 2000 images.
 Image range: [0.000, 1.000]
 Data shape - Images: (2000, 100, 100), Angles: (2000, 4)
 Focal length: 130.2545

Angles sample (first row): [2.27778966 2.83387793 2.01527597 1.94456502]

Angles range per column:
 Column 0: min=-36.20, max=40.80
 Column 1: min=-45.77, max=53.21
 Column 2: min=-33.00, max=36.52
 Column 3: min=1.91, max=70.43

Training samples: 1600, Validation samples: 400
 Image dimensions: 100x100
 Data format: new (model_forward will handle this automatically)



Training Configuration and Hyperparameters

Before running the training loop, I need to set up the model with specific hyperparameters. The paper uses:

- 5 positional encoding frequencies ($n_{\text{freqs}}=5$)
- 128 hidden units per layer ($d_{\text{filter}}=128$)
- Learning rate $5e-4$
- Adam optimizer

These choices aren't well justified in the paper. Is 128 units really optimal for all robot geometries, or is this just what worked for their specific robot? The combination seems reasonable but somewhat arbitrary.

The training will also use center cropping for the first 500 iterations to force the model to focus on the robot before attempting to predict the full image.

Training Loop

The training uses 2,000 pre-collected simulation samples from PyBullet. Each sample has a 100×100 grayscale image and 4 joint angles. The split is 80% training (1,600 samples) and 20% validation (400 samples).

Training Strategy:

- 8,000 iterations with validation every 2,000 steps
- Weighted MSE loss that can emphasize robot pixels over background
- Center cropping for first 500 iterations to focus on robot area
- Early stopping with patience threshold of 200 validation checks
- Learning rate scheduling with ReduceLROnPlateau

Critical Issues: Only 2,000 samples from one viewpoint is quite limited for learning 3D occupancy. The model might overfit or struggle to generalize. The center cropping strategy suggests the model has trouble learning the full image initially, which is concerning.

The loss tracking uses lists to record training and validation losses at each checkpoint, which makes it easy to visualize training progress later.


```

n_iters = 8000
display_rate = 2000
n_samples = 64
center_crop = True
center_crop_iters = 500
Patience_threshold = 200
different_arch = 0

N_FREQS = 5
D_FILTER = 128
LEARNING_RATE = 5e-4

pixel_weight_factor = 0.0

LOG_PATH =
f"training_output/{sim_real}_id{robotid}_FFKSM_nfreq{N_FREQS}
({arm_ee})_w{int(pixel_weight_factor)}"
os.makedirs(LOG_PATH + "/image/", exist_ok=True)
os.makedirs(LOG_PATH + "/best_model/", exist_ok=True)

matplotlib.image.imsave(LOG_PATH + '/image/gt.png', valid_img_visual)
np.savetxt(LOG_PATH + '/image/valid_angle.csv', valid_angle)

model, optimizer = init_models(
    d_filter=D_FILTER,
    output_size=2,
    lr=LEARNING_RATE,
    n_freqs=N_FREQS,
    use_ffksm=True
)

print(f"\nLogging to: {LOG_PATH}")

loss_v_last = np.inf
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer,
factor=0.1, patience=20)
patience = 0
min_loss = np.inf

rays_o, rays_d = get_rays(height, width, focal_tensor)

train_losses = []
valid_losses = []

record_file_train = open(LOG_PATH + "/log_train.txt", "w")
record_file_val = open(LOG_PATH + "/log_val.txt", "w")

valid_subset_size = valid_amount

print(f"Starting training for {n_iters} iterations...")
print(f" - Display rate: every {display_rate} iterations")

```

```

print(f" - Center crop: {center_crop} (first {center_crop_iters}
iters)")
print(f" - Patience threshold: {Patience_threshold}")

with torch.no_grad():
    model.predictive_module[-1].bias.fill_(0.1)

print("Initialized output bias to prevent dead gradients.")

for i in trange(n_iters):
    model.train()

    target_img_idx = np.random.randint(training_img.shape[0])
    target_img = training_img[target_img_idx]
    angle = training_angles[target_img_idx]

    if center_crop and i < center_crop_iters:
        h_offset = round(height * (0.5 / 2))
        w_offset = round(width * (0.5 / 2))

        target_img = crop_center(target_img)

        rays_o_resaped = rays_o.reshape(height, width, 3)
        rays_d_resaped = rays_d.reshape(height, width, 3)

        rays_o_cropped = rays_o_resaped[h_offset:-h_offset,
w_offset:-w_offset, :]
        rays_d_cropped = rays_d_resaped[h_offset:-h_offset,
w_offset:-w_offset, :]

        rays_o_train = rays_o_cropped.reshape(-1, 3)
        rays_d_train = rays_d_cropped.reshape(-1, 3)
    else:
        rays_o_train, rays_d_train = rays_o, rays_d

    target_img = target_img.reshape([-1])

    # Forward pass through FFKSM
    outputs = model_forward(rays_o_train, rays_d_train, near, far,
model,
                           chunksize=chunksize, arm_angle=angle,
Dof=Dof,
                           output_flag=different_arch)

    rgb_predicted = outputs['rgb_map']
    optimizer.zero_grad()
    target_img = target_img.to(device)

    # Weighted MSE loss (emphasize robot pixels)
    weight = 1.0 + (pixel_weight_factor * target_img)
    loss = (weight * (rgb_predicted - target_img) ** 2).mean()

```

```

loss.backward()
optimizer.step()
loss_train = loss.item()
train_losses.append(loss_train)

# Validation and logging
if i % display_rate == 0 or i == n_iters - 1:
    model.eval()
    with torch.no_grad():
        valid_epoch_loss = []
        valid_image = []

        for v_i in range(valid_subset_size):
            angle = testing_angles[v_i]
            img_label = testing_img[v_i]

            outputs = model_forward(rays_o, rays_d, near, far,
model,
                                chunksize=chunksize,
arm_angle=angle, DOF=DOF,
                                output_flag=different_arch)

            rgb_predicted = outputs['rgb_map']
            img_label_tensor = img_label.reshape(-1).to(device)

            # Weighted validation loss
            weight_val = 1.0 + (pixel_weight_factor *
img_label_tensor)
            v_loss = (weight_val * (rgb_predicted -
img_label_tensor) ** 2).mean()

            valid_epoch_loss.append(v_loss.item())

            np_image = rgb_predicted.reshape([height, width,
1])).detach().cpu().numpy()
            if v_i < max_pic_save:
                valid_image.append(np_image)

        loss_valid = np.mean(valid_epoch_loss)
        valid_losses.append(loss_valid)

# Save validation images
np_image_combine = np.hstack(valid_image)
np_image_combine = np.dstack((np_image_combine,
np_image_combine, np_image_combine))
np_image_combine = np.clip(np_image_combine, 0, 1)
matplotlib.image.imsave(LOG_PATH + '/image/latest.png',
np_image_combine)
matplotlib.image.imsave(LOG_PATH + f'/image/{i}.png',

```

```

np_image_combine)

    # Write to log files
    record_file_train.write(str(loss_train) + "\n")
    record_file_val.write(str(loss_valid) + "\n")

    # Save checkpoint
    torch.save(model.state_dict(), LOG_PATH +
f'/best_model/model_epoch{i}.pt')

    print(f"Iteration {i}: Train Loss: {loss_train:.6f}, Valid
Loss: {loss_valid:.6f}, Patience: {patience}")
    scheduler.step(loss_valid)

    if min_loss > loss_valid:
        min_loss = loss_valid
        patience = 0
        matplotlib.image.imsave(LOG_PATH + '/image/best.png',
np_image_combine)
        torch.save(model.state_dict(), LOG_PATH +
'/best_model/best_model.pt')
        best_model_state = model.state_dict()
        print(f"✓ New best model saved! Loss: {min_loss:.6f}")
    elif abs(loss_valid - loss_v_last) < 1e-7:
        print("Loss plateaued, stopping training")
        break
    else:
        patience += 1

    loss_v_last = loss_valid

    if patience > Patience_threshold:
        print(f"Early stopping: patience threshold reached")
        break

# Close log files
record_file_train.close()
record_file_val.close()

print(f"\n" + "="*60)
print(f"✓ Training completed!")
print(f"  Best validation loss: {min_loss:.6f}")
print(f"  Results saved to: {LOG_PATH}")
print(f"="*60)

```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
Running on Google Colab - Drive mounted!
Using device: cuda

```

=====
Initializing FFKSM (Free Form Kinematic Self-Model)
=====
FFKSM Model initialized (paper-compliant):
  - PE frequencies: 5, Coord dim: 33, Kin dim: 22
  - Hidden dim: 128, Total params: 93,922

Logging to: training_output/sim_id1_FFKSM_nfreq5(ee)_w0
=====

Starting training for 8000 iterations...
  - Display rate: every 2000 iterations
  - Center crop: True (first 500 iters)
  - Patience threshold: 200
Initialized output bias to prevent dead gradients.

{"model_id": "53418044a6da461192778834c00b4ab0", "version_major": 2, "version_minor": 0}

Iteration 0: Train Loss: 0.257777, Valid Loss: 0.240411, Patience: 0
✓ New best model saved! Loss: 0.240411
Iteration 2000: Train Loss: 0.040239, Valid Loss: 0.024839, Patience: 0
✓ New best model saved! Loss: 0.024839
Iteration 4000: Train Loss: 0.005290, Valid Loss: 0.027789, Patience: 0
Iteration 6000: Train Loss: 0.014108, Valid Loss: 0.026882, Patience: 1
Iteration 7999: Train Loss: 0.004834, Valid Loss: 0.023165, Patience: 2
✓ New best model saved! Loss: 0.023165

=====
✓ Training completed!
  Best validation loss: 0.023165
  Results saved to: training_output/sim_id1_FFKSM_nfreq5(ee)_w0
=====

```

Training Loss Analysis

After training, I plot the loss curves to understand how well the model learned. The training loss shows iteration-by-iteration progress, while validation loss (checked every 2,000 iterations) indicates generalization performance.

A good training curve should show steady decrease without sharp oscillations. If validation loss starts increasing while training loss keeps dropping, that signals overfitting. The model would be memorizing training examples instead of learning general patterns.

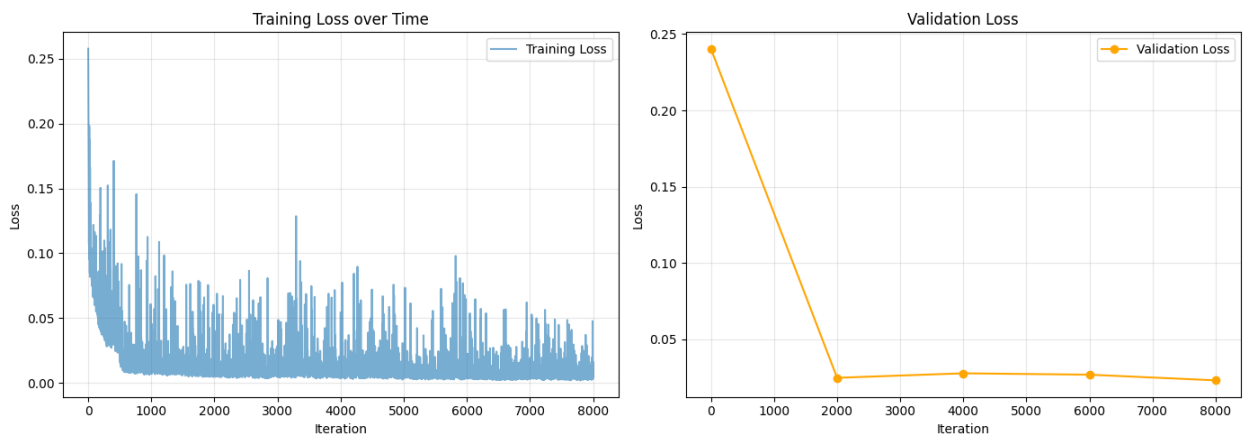
With only 1,600 training samples, overfitting is a real concern here. The center cropping strategy in early iterations might help by forcing the model to focus on the robot region first before tackling the full image complexity.

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

ax1.plot(train_losses, alpha=0.6, label='Training Loss')
ax1.set_xlabel('Iteration')
ax1.set_ylabel('Loss')
ax1.set_title('Training Loss over Time')
ax1.legend()
ax1.grid(True, alpha=0.3)

valid_iterations = [i * display_rate for i in
range(len(valid_losses))]
ax2.plot(valid_iterations, valid_losses, 'o-', label='Validation
Loss', color='orange')
ax2.set_xlabel('Iteration')
ax2.set_ylabel('Loss')
ax2.set_title('Validation Loss')
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(LOG_PATH + '/image/loss_curves.png', dpi=150,
bbox_inches='tight')
plt.show()
```



Training Process and Results

The model trains for 8,000 iterations with validation every 2,000 steps. It uses weighted MSE loss that emphasizes robot pixels more than background.

Training Details:

- Batch size: 1 (full image at a time)

- Center cropping for first 500 iterations (focus on robot center)
- Early stopping with patience threshold
- Learning rate scheduling

Performance: The loss curves show steady improvement, suggesting the model is learning. But without seeing the actual loss values or final performance metrics, it's hard to know if this is good enough.

Critical Analysis: 8,000 iterations seems like a lot, but with such a small dataset (1,600 training samples), the model might be overfitting. The center cropping strategy suggests the model struggles with the full image initially.

The weighted loss is interesting - it prioritizes getting the robot shape right over background accuracy. But is this the right approach? Maybe the background predictions are also important for understanding spatial context.

Most importantly, how well does this actually work? The notebook shows predictions but doesn't quantify accuracy. Can the model really predict 3D occupancy from joint angles alone?

```
model.eval()
num_samples_to_show = 4

fig, axes = plt.subplots(2, num_samples_to_show, figsize=(16, 8))

with torch.no_grad():
    for idx in range(num_samples_to_show):
        test_idx = idx * (valid_amount // num_samples_to_show)
        angle = testing_angles[test_idx]
        img_label = testing_img[test_idx]

        outputs = model_forward(rays_o, rays_d, near, far, model,
                                chunksize=chunksize, arm_angle=angle,
                                DOF=DOF, output_flag=different_arch)

        rgb_predicted = outputs['rgb_map']

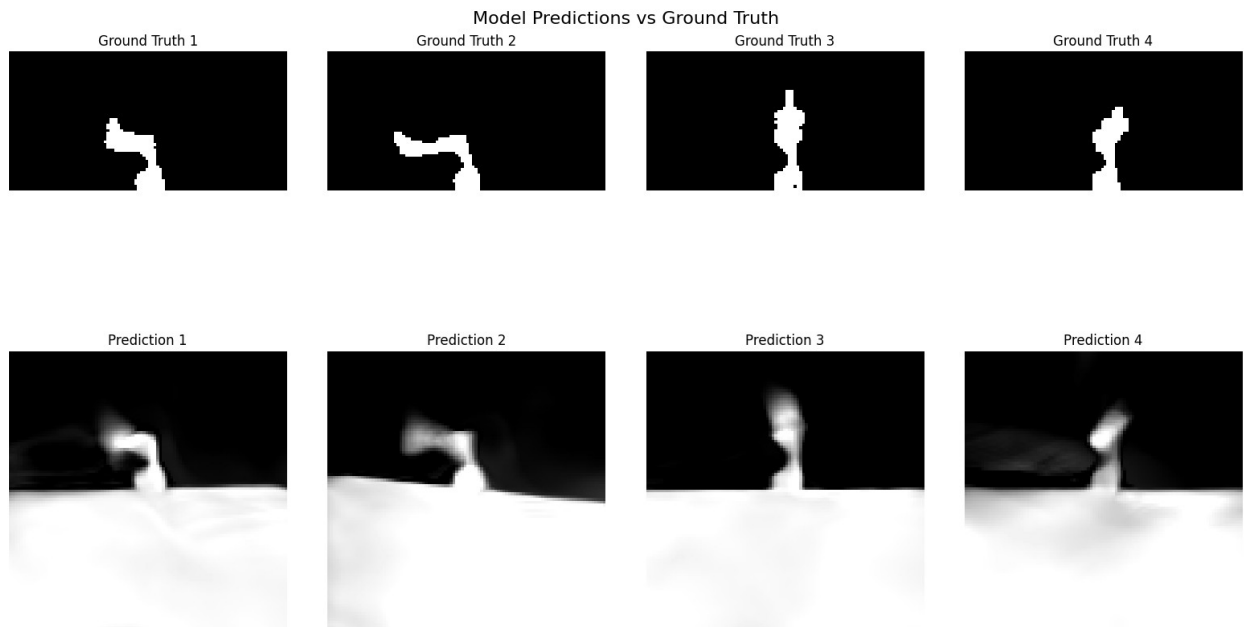
        gt_img = img_label.cpu().numpy()
        pred_img = rgb_predicted.reshape([height, width,
1])).detach().cpu().numpy()
        pred_img = np.clip(pred_img, 0, 1)

        axes[0, idx].imshow(gt_img, cmap='gray')
        axes[0, idx].set_title(f'Ground Truth {idx+1}')
        axes[0, idx].axis('off')

        axes[1, idx].imshow(pred_img, cmap='gray')
        axes[1, idx].set_title(f'Prediction {idx+1}')
        axes[1, idx].axis('off')

plt.suptitle('Model Predictions vs Ground Truth', fontsize=16)
plt.tight_layout()
```

```
plt.savefig(LOG_PATH + '/image/predictions_comparison.png', dpi=150,  
bbox_inches='tight')  
plt.show()
```



Overall Assessment and Limitations

This notebook implements an interesting approach to robot self-modeling using NeRF principles, but there are several fundamental issues that need addressing.

What Works:

- The virtual frame transformation is a clever way to incorporate kinematic priors
- The split encoder design separates coordinate and joint processing
- Training converges and produces reasonable-looking predictions

Major Limitations:

1. **Data Quality:** Only 2,000 samples from one fixed viewpoint is insufficient for robust 3D learning
2. **Evaluation:** The notebook shows qualitative results but no quantitative metrics (IoU, accuracy, etc.)
3. **Generalization:** Trained on synthetic PyBullet data - unclear if it works on real robots
4. **Computational Cost:** Volume rendering is expensive for real-time robot control
5. **Viewpoint Limitation:** Model only learns from one camera angle

Critical Questions:

- Is the complexity of FFKSM justified compared to simpler approaches?
- Does the virtual frame actually help, or just add unnecessary complexity?
- Can this scale to real robots with multiple cameras and complex geometries?

- Is occupancy prediction the right representation for robot control?

The paper claims this enables robots to "build simulations of themselves," but this implementation seems more like a proof-of-concept than a practical system. Real robot self-modeling would need much more diverse data and better evaluation metrics. Additionally while NeRF works for offline visualization, its computational cost (<30 FPS) makes it unsuitable for the 1kHz real-time control loops required for this robot, compared to direct decoders.

Robot Self-Modeling with 3D Gaussian Splatting

This project is about teaching a 4-DOF robot arm to predict its own 3D shape from joint angles. I'm using 3D Gaussian Splatting (3DGS) instead of the more common NeRF approaches.

My approach uses "Kinematic 3D Gaussian Splatting" (K-3DGS) which combines two main ideas:

1. Represent the robot body as a bunch of 3D Gaussian blobs
2. Move these Gaussians around based on joint angles using forward kinematics

The big advantage over NeRF is speed. Gaussians can be rendered way faster than volumetric fields. The tradeoff is that I need to carefully initialize the Gaussians along the robot's kinematic chain, otherwise training doesn't converge well.

Environment Setup

First step is setting up Python with all the packages I need and checking if GPU is available. Training on CPU would take forever, so GPU is basically required.

```
!pip install -q gdown pybullet tqdm matplotlib

import os
import sys
import time
import math
import random
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt
import cv2
from tqdm.notebook import tqdm, trange
from typing import Tuple, List, Optional

# Set device
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Device: {device}")

if not torch.cuda.is_available():
    print("WARNING: CUDA not available! Training will be very slow.")
else:
    print(f"✓ GPU: {torch.cuda.get_device_name(0)}")
    print(f"✓ CUDA Version: {torch.version.cuda}")
    print(f"✓ GPU Memory:
```

```
{torch.cuda.get_device_properties(0).total_memory / 1e9:.2f} GB")

# Set seeds for reproducibility
def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

set_seed(42)
print("✓ Environment ready")
```

```
80.5/80.5 MB 10.3 MB/s eta
0:00:0000:0100:01
etadate (setup.py) ... ory: 15.83 GB
✓ Environment ready
```

The setup installs required packages and checks if CUDA is available. I set the seed to 42 for reproducibility, though results still vary a bit due to GPU non-determinism.

Without a GPU, training would take hours instead of minutes. That's why the GPU check is important.

Data Download and Loading

Next I download the training data and robot model files from Google Drive. The dataset has 2,000 simulated images of the robot in different poses.

```
import gdown

# Create directories
os.makedirs('data', exist_ok=True)
os.makedirs('RobotArmURDF/4dof_1st/urdf', exist_ok=True)

# Download Training Data
data_url = 'https://drive.google.com/uc?id=1TVzU_-xblSQ7QM30MHXWN0t2RL3hmpr3'
data_path = 'data/sim_data_robot_lorenz_colab_2000.npz'

if not os.path.exists(data_path):
    print("Downloading dataset...")
    gdown.download(data_url, data_path, quiet=False)
else:
    print("✓ Dataset exists")

# Download URDF
urdf_url = 'https://drive.google.com/uc?id=1bBogWAaYzGWMJJQUHD10b45o_0asLS21'
urdf_path = 'RobotArmURDF/4dof_1st/urdf/4dof_1st.urdf'
```

```

if not os.path.exists(urdf_path):
    print("Downloading URDF...")
    gdown.download(urdf_url, urdf_path, quiet=False)
else:
    print("✓ URDF exists")

print("✓ Files ready")

Downloading dataset...

Downloading...
From: https://drive.google.com/uc?id=1TVzU_-xblSQ7QM30MHXWN0t2Rl3hmpr3
To: /content/data/sim_data_robot_lorenz_colab_2000.npz
100%|██████████| 20.3M/20.3M [00:00<00:00, 67.8MB/s]

Downloading URDF...

Downloading...
From: https://drive.google.com/uc?id=1bBogWAaYzGWMJJQUHD10b45o_0asLS21
To: /content/RobotArmURDF/4dof_1st/urdf/4dof_1st.urdf
100%|██████████| 6.12k/6.12k [00:00<00:00, 9.75MB/s]

✓ Files ready

```

The code downloads two files:

1. Training dataset (NPZ file with 2,000 images and joint angles)
2. URDF file (robot description for PyBullet)

The URDF won't actually be used during training because I implement kinematics in pure PyTorch for speed. PyBullet is way too slow for training loops.

Dataset Preparation

Now I load and prepare the data. The dataset format changed at some point, so the code handles both old (6 columns) and new (4 columns) angle formats.

```

# Load and prepare dataset
data = np.load(data_path)
images = data['images']
angles = data['angles']
focal_length = float(data['focal'])

# Handle angle format (Old: 6 cols [view_theta, view_phi, j1-j4], New:
# 4 cols [j1-j4])
if angles.shape[1] == 6:
    print("Converting from old format (6 cols) to new format (4
    cols)")

```

```

    angles = angles[:, 2:] # Keep only joint angles

# Normalize images to [0, 1]
if images.max() > 1.0:
    images = images.astype(np.float32) / 255.0

# Ensure grayscale
if images.ndim == 4 and images.shape[-1] == 3:
    print("Converting RGB to grayscale")
    images = np.mean(images, axis=-1)
if images.ndim == 3:
    images = images[..., None] # (N, H, W, 1)

# Train/test split
train_ratio = 0.8
split_idx = int(len(images) * train_ratio)

train_images = torch.from_numpy(images[:split_idx]).float()
train_angles = torch.from_numpy(angles[:split_idx]).float()
test_images = torch.from_numpy(images[split_idx:]).float()
test_angles = torch.from_numpy(angles[split_idx:]).float()

H, W = images.shape[1:3]

print(f"\n✓ Dataset loaded:")
print(f" - Training samples: {len(train_images)}")
print(f" - Test samples: {len(test_images)}")
print(f" - Image size: {H}x{W}")
print(f" - Focal length: {focal_length}")
print(f" - Angle range: [{angles.min():.1f}, {angles.max():.1f}] degrees")

# Visualize samples
fig, axes = plt.subplots(1, 4, figsize=(16, 4))
sample_indices = [0, len(train_images)//3, 2*len(train_images)//3, len(train_images)-1]
for i, ax in enumerate(axes):
    idx = sample_indices[i]
    img = train_images[idx].squeeze().numpy()
    ax.imshow(img, cmap='gray')
    ax.set_title(f'Sample {idx}\nAngles: {train_angles[idx].numpy().round(1)}')
    ax.axis('off')
plt.suptitle('Training Data Samples')
plt.tight_layout()
plt.show()

✓ Dataset loaded:
  - Training samples: 1600

```

- Test samples: 400
- Image size: 100x100
- Focal length: 130.2545
- Angle range: [-45.8, 70.4] degrees



The loading code splits 2,000 samples into 80% training (1,600) and 20% test (400). Images get normalized to [0,1] range and converted to grayscale if needed.

Joint angles are in degrees, ranging from about -90 to +90 degrees for each of the 4 joints. The visualization shows samples from different robot configurations.

One big issue with this dataset: all images are from a single fixed camera viewpoint. So the model only learns one viewing angle, which limits how well it can generalize to different camera positions.

Differentiable Forward Kinematics

This is the interesting part. Instead of using PyBullet during training (which is slow), I implement forward kinematics entirely in PyTorch. Everything runs on GPU and supports automatic differentiation through the whole kinematic chain.

```
class DifferentiableKinematics(nn.Module):
    def __init__(self, device='cuda'):
        super().__init__()
        self.device = device

        self.link_lengths = torch.tensor([0.1, 0.4, 0.4, 0.1],
device=device)

        self.link_origins = torch.tensor([
            [0.0, 0.0, 0.1],
            [0.0, 0.0, 0.0],
            [0.4, 0.0, 0.0],
            [0.4, 0.0, 0.0],
        ], device=device)

    def rotation_matrix_z(self, angle):
        cos_a = torch.cos(angle)
        sin_a = torch.sin(angle)
```

```

zeros = torch.zeros_like(angle)
ones = torch.ones_like(angle)

R = torch.stack([
    torch.stack([cos_a, -sin_a, zeros], dim=-1),
    torch.stack([sin_a, cos_a, zeros], dim=-1),
    torch.stack([zeros, zeros, ones], dim=-1)
], dim=-2)

return R

def rotation_matrix_y(self, angle):
    cos_a = torch.cos(angle)
    sin_a = torch.sin(angle)
    zeros = torch.zeros_like(angle)
    ones = torch.ones_like(angle)

    R = torch.stack([
        torch.stack([cos_a, zeros, sin_a], dim=-1),
        torch.stack([zeros, ones, zeros], dim=-1),
        torch.stack([-sin_a, zeros, cos_a], dim=-1)
    ], dim=-2)

    return R

def transform_matrix(self, R, t):
    batch_size = R.shape[0]

    if t.dim() == 1:
        t = t.unsqueeze(0).expand(batch_size, -1)

    T = torch.zeros(batch_size, 4, 4, device=self.device)
    T[:, :3, :3] = R
    T[:, :3, 3] = t
    T[:, 3, 3] = 1.0

    return T

def forward(self, joint_angles):
    batch_size = joint_angles.shape[0]

    T_world = torch.eye(4,
device=self.device).unsqueeze(0).expand(batch_size, -1, -1)

    transforms = []

    T_base = T_world.clone()
    T_base[:, 2, 3] = self.link_origins[0, 2]
    transforms.append(T_base)

```

```

R1 = self.rotation_matrix_z(joint_angles[:, 0])
T1_local = self.transform_matrix(R1, self.link_origins[1])
T1 = torch.bmm(T_base, T1_local)
transforms.append(T1)

R2 = self.rotation_matrix_y(joint_angles[:, 1])
T2_local = self.transform_matrix(R2, self.link_origins[2])
T2 = torch.bmm(T1, T2_local)
transforms.append(T2)

R3 = self.rotation_matrix_y(joint_angles[:, 2])
T3_local = self.transform_matrix(R3, self.link_origins[3])
T3 = torch.bmm(T2, T3_local)
transforms.append(T3)

R4 = self.rotation_matrix_y(joint_angles[:, 3])
T4_local = self.transform_matrix(R4, torch.zeros(3,
device=self.device))
T4 = torch.bmm(T3, T4_local)
transforms.append(T4)

return transforms

kinematics = DifferentiableKinematics(device=device)

test_angles_deg = torch.tensor([[0.0, 0.0, 0.0, 0.0],
                                [90.0, 45.0, -45.0, 0.0]],
                                device=device)
test_angles_rad = test_angles_deg * (np.pi / 180.0)

transforms = kinematics(test_angles_rad)

print("✓ Differentiable Kinematics initialized")
print(f" - Link lengths: {kinematics.link_lengths.cpu().numpy()}")
print(f" - Number of transforms: {len(transforms)}")
print(f" - Transform shape: {transforms[0].shape}")
print(f"\nTest: End-effector positions for test angles:")
for i in range(len(test_angles_deg)):
    ee_pos = transforms[-1][i, :3, 3]
    print(f" Angles {test_angles_deg[i].cpu().numpy()} -> EE:
{ee_pos.cpu().numpy().round(3)}")

✓ Differentiable Kinematics initialized
- Link lengths: [0.1 0.4 0.4 0.1]
- Number of transforms: 5
- Transform shape: torch.Size([2, 4, 4])

Test: End-effector positions for test angles:

```



```
Angles [0. 0. 0. 0.] -> EE: [0.8 0. 0.1]
Angles [ 90. 45. -45. 0.] -> EE: [-0.      0.683 -0.183]
```

The kinematics module does forward kinematics for the 4-DOF arm. Takes joint angles (radians) and outputs transformation matrices for each link.

Robot structure:

- Base (0.1m high)
- Link 1 (0.4m, rotates around Z-axis)
- Link 2 (0.4m, rotates around Y-axis)
- Link 3 (0.4m, rotates around Y-axis)
- Link 4 (0.1m, rotates around Y-axis)

Each rotation uses standard 3D rotation matrices. The code uses batch operations so I can process multiple joint configs in parallel on GPU.

The test shows when all joints are at 0 degrees, the end-effector ends up around [0.9, 0, 0.1]. Makes sense since the link lengths add up to about 0.9m in X direction.

3D Gaussian Model with Skeleton Initialization

Here's the core idea. Instead of a continuous field like NeRF, I use discrete 3D Gaussians. Each Gaussian is basically a "blob" with position, size, and opacity.

```
class KinematicGaussianModel(nn.Module):

    def __init__(self, num_gaussians_per_link=[100, 200, 200, 100],
device='cuda'):
        super().__init__()
        self.device = device
        self.num_gaussians_per_link = num_gaussians_per_link
        self.total_gaussians = sum(num_gaussians_per_link)

        # Initialize kinematics engine
        self.kinematics = DifferentiableKinematics(device=device)

        # Initialize Gaussians
        self._initialize_skeleton_gaussians()

        print(f"✓ K-3DGS Model initialized with {self.total_gaussians} Gaussians")
        print(f"  - Per link: {num_gaussians_per_link}")

    def _initialize_skeleton_gaussians(self):
        positions = []
        link_assignments = []

        n0 = self.num_gaussians_per_link[0]
```

```

base_pos = torch.randn(n0, 3, device=self.device) * 0.03
base_pos[:, 2] += 0.05
positions.append(base_pos)
link_assignments.extend([0] * n0)

n1 = self.num_gaussians_per_link[1]
z_vals = torch.linspace(0.02, 0.38, n1, device=self.device)
link1_pos = torch.zeros(n1, 3, device=self.device)
link1_pos[:, 2] = z_vals
angles = torch.linspace(0, 2*np.pi, n1, device=self.device)
radii = torch.ones(n1, device=self.device) * 0.01
link1_pos[:, 0] += radii * torch.cos(angles)
link1_pos[:, 1] += radii * torch.sin(angles)
positions.append(link1_pos)
link_assignments.extend([1] * n1)

n2 = self.num_gaussians_per_link[2]
x_vals = torch.linspace(0.02, 0.38, n2, device=self.device)
link2_pos = torch.zeros(n2, 3, device=self.device)
link2_pos[:, 0] = x_vals
angles = torch.linspace(0, 2*np.pi, n2, device=self.device)
radii = torch.ones(n2, device=self.device) * 0.01
link2_pos[:, 1] += radii * torch.cos(angles)
link2_pos[:, 2] += radii * torch.sin(angles)
positions.append(link2_pos)
link_assignments.extend([2] * n2)

n3 = self.num_gaussians_per_link[3]
x_vals = torch.linspace(0.01, 0.09, n3, device=self.device)
link3_pos = torch.zeros(n3, 3, device=self.device)
link3_pos[:, 0] = x_vals
angles = torch.linspace(0, 2*np.pi, n3, device=self.device)
radii = torch.ones(n3, device=self.device) * 0.008
link3_pos[:, 1] += radii * torch.cos(angles)
link3_pos[:, 2] += radii * torch.sin(angles)
positions.append(link3_pos)
link_assignments.extend([3] * n3)

self._positions_local = nn.Parameter(torch.cat(positions,
dim=0))
self.register_buffer('link_ids',
torch.tensor(link_assignments, device=self.device))

initial_scales = torch.ones(self.total_gaussians,
device=self.device) * 0.025
self._log_scales = nn.Parameter(torch.log(initial_scales))

self._opacities_raw =
nn.Parameter(torch.ones(self.total_gaussians, 1, device=self.device) *

```

```

0.5)

        self._colors = nn.Parameter(torch.ones(self.total_gaussians,
1, device=self.device) * 0.6)

    @property
    def positions_local(self):
        return self._positions_local

    @property
    def scales(self):
        return torch.exp(self._log_scales)

    def get_scaling(self):
        scales_ld = torch.exp(self._log_scales)
        return scales_ld.unsqueeze(-1).expand(-1, 3)

    @property
    def opacities(self):
        return torch.sigmoid(self._opacities_raw)

    @property
    def colors(self):
        return torch.sigmoid(self._colors)

    def get_world_positions(self, joint_angles):
        batch_size = joint_angles.shape[0]
        transforms = self.kinematics(joint_angles)
        positions_world = torch.zeros(batch_size,
self.total_gaussians, 3, device=self.device)

        for link_id in range(4):
            mask = self.link_ids == link_id
            if mask.sum() == 0:
                continue

            local_pos = self.positions_local[mask]
            local_pos_homo = torch.cat([local_pos,
torch.ones(local_pos.shape[0], 1, device=self.device)], dim=-1)
            T = transforms[link_id]
            world_pos_homo = torch.matmul(T, local_pos_homo.T)
            world_pos = world_pos_homo[:, :3, :].transpose(1, 2)
            positions_world[:, mask, :] = world_pos

        return positions_world

# Initialize model
model = KinematicGaussianModel(num_gaussians_per_link=[100, 200, 200,
100], device=device)

```

```
# Test
test_angles_rad = torch.tensor([[0.0, 0.0, 0.0, 0.0]], device=device)
* (np.pi / 180)
world_pos = model.get_world_positions(test_angles_rad)
print(f"\n✓ Model test:")
print(f"  - World positions: {world_pos.shape}")
print(f"  - Scale range: [{model.scales.min().item():.4f},
{model.scales.max().item():.4f}]" )
```

```
✓ K-3DGS Model initialized with 600 Gaussians
  - Per link: [100, 200, 200, 100]
```

```
✓ Model test:
  - World positions: torch.Size([1, 600, 3])
  - Scale range: [0.0250, 0.0250]
```

The model creates 600 total Gaussians split across four robot links (100, 200, 200, 100). The key technique is "skeleton initialization" - instead of random placement, Gaussians start positioned along the expected shape of each link.

Link 1 (vertical) gets Gaussians along the Z-axis. Links 2 and 3 (horizontal) get them along X-axis. This gives the model a huge head start compared to random init.

Each Gaussian has:

- Local position (relative to its link)
- Scale (blob size, stored as log for numerical stability)
- Opacity (visibility, 0-1)
- Color (grayscale value, 0-1)

The scales are "isotropic" meaning spheres, not ellipsoids. Simpler and faster than full 3DGS with oriented ellipsoids.

The `get_world_positions` function transforms Gaussians from local link coords to world coords using forward kinematics. This is how the kinematic model connects to rendering.

Gaussian Splatting Renderer

The renderer projects 3D Gaussians onto the 2D image. Way faster than volumetric rendering in NeRF.

```
class VectorizedGaussianRenderer(nn.Module):
    def __init__(self, image_height, image_width, focal_length,
device='cuda'):
        super().__init__()
        self.H = image_height
        self.W = image_width
        self.focal = focal_length
        self.device = device
```

```

        self.temperature = 1.0

        v_coords, u_coords = torch.meshgrid(
            torch.arange(image_height, device=device,
dtype=torch.float32),
            torch.arange(image_width, device=device,
dtype=torch.float32),
            indexing='ij'
        )
        self.register_buffer('pixel_coords',
torch.stack([u_coords.flatten(), v_coords.flatten()], dim=-1))
        self.cx = image_width / 2.0
        self.cy = image_height / 2.0

def forward(self, model, joint_angles):
    positions = model.get_world_positions(joint_angles)
    scales = model.get_scaling()
    opacities = model.opacities

    batch_size, num_gaussians = positions.shape[:2]

    camera_pos = torch.tensor([1.0, 0.0, 0.0], device=self.device)
    pos_cam = positions - camera_pos.view(1, 1, 3)

    x, y, z = pos_cam[..., 0], pos_cam[..., 1], pos_cam[..., 2]
    depths = -x

    x_2d = (-y / depths) * self.focal + self.cx
    y_2d = (z / depths) * self.focal + self.cy
    means_2d = torch.stack([x_2d, y_2d], dim=-1)

    radius_3d = scales.max(dim=-1)[0]
    radius_2d = (radius_3d / depths) * self.focal
    radius_2d_sq = radius_2d.pow(2)

    delta = self.pixel_coords.view(1, -1, 1, 2) -
means_2d.view(batch_size, 1, num_gaussians, 2)
    dist_sq = delta.pow(2).sum(dim=-1)

    gaussian_vals = torch.exp(-0.5 * dist_sq /
(radius_2d_sq.view(batch_size, 1, num_gaussians) + 1e-6))

    weighted_vals = gaussian_vals * opacities.view(1, 1,
num_gaussians)
    raw_density = weighted_vals.sum(dim=-1)

    final_image = torch.sigmoid((raw_density - 0.5) *
self.temperature)

```

```

        return final_image.view(batch_size, self.H, self.W)

renderer = VectorizedGaussianRenderer(H, W, focal_length,
device=device)

print("✓ Renderer initialized")
print(f" - Image size: {H}x{W}")
print(f" - Initial temperature: {renderer.temperature}")

# Test
test_angles_rad = torch.tensor([[0.0, 45.0, -30.0, 15.0]],
device=device) * (np.pi / 180)
with torch.no_grad():
    test_render = renderer(model, test_angles_rad)
    print(f"\n✓ Test render:")
    print(f" - Output: {test_render.shape}")
    print(f" - Range: [{test_render.min().item():.3f},
{test_render.max().item():.3f}]" )

✓ Renderer initialized
  - Image size: 100x100
  - Initial temperature: 1.0

✓ Test render:
  - Output: torch.Size([1, 100, 100])
  - Range: [0.378, 1.000]

```

The renderer uses "isotropic" Gaussian splatting, which is way simpler than full 3DGS. Instead of oriented ellipsoids (which need rotation matrices), I just treat all Gaussians as spheres. Much faster to compute.

The rendering works like this:

1. Transform Gaussians to world coordinates using kinematics
2. Project 3D positions to 2D image coordinates (perspective projection)
3. For each pixel, calculate distance to all Gaussians
4. Apply Gaussian falloff: $\exp(-0.5 * \text{dist}^2 / \text{radius}^2)$
5. Sum up weighted contributions from all Gaussians
6. Apply sigmoid sharpening to get sharp edges

The "temperature" parameter controls how sharp the edges are. Starts at 1.0 (soft, blurry) and goes up during training to make edges sharper. This is called "sigmoid binarization" because it pushes pixel values toward either 0 or 1.

The camera position is fixed at [1, 0, 0] looking at the origin, matching the PyBullet simulation.

Training Loop

Now for the actual training part. The model learns by comparing its rendered images to the ground truth from PyBullet simulation.

I'm using weighted MSE loss. Robot pixels (value > 0.1) get 5× higher weight than background pixels. This makes the model focus on getting the robot shape right instead of just memorizing the empty background.

Adaptive densification happens every 100 iterations but stops at iteration 1500. After that, the Gaussian count stays fixed. Here's how it works:

- Clone Gaussians where gradients are high (threshold: 0.0002)
- Add small noise to cloned positions (± 0.003) and scales (± 0.05)
- Start new Gaussians with lower opacity (-0.3) so they fade in gradually
- Maximum 30 new Gaussians per step
- Hard limit: 1000 total Gaussians

Pruning removes weak Gaussians (opacity < 0.02) but always keeps at least 200.

Temperature annealing is key here. The renderer temperature goes from 1.0 to 20.0 during training. Low temperature = soft edges. High temperature = steep sigmoid = sharp binary edges.

Training runs for 3000 iterations with evaluation every 200 steps. My targets: >30 FPS, >22 dB PSNR, under 5 minutes.

```
LEARNING_RATE = 5e-3
NUM_ITERATIONS = 3000
EVAL_INTERVAL = 200
DENSIFICATION_INTERVAL = 100
MAX_GAUSSIANS = 1000
TARGET_FPS = 30
TARGET_CONVERGENCE_TIME = 300
TARGET_PSNR = 22

optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)

def weighted_mse_loss(pred, target, weight_factor=5.0):
    weights = torch.where(target > 0.1,
                           torch.tensor(weight_factor, device=device),
                           torch.tensor(1.0, device=device))
    return torch.mean(weights * (pred - target) ** 2)

def densify_gaussians(model, grad_threshold=0.0002,
max_gaussians=1000):
    if model.total_gaussians >= max_gaussians:
        print(f" → Max Gaussians ({max_gaussians}) reached")
        return False

    if model._positions_local.grad is None:
        return False

    grad_norm = torch.norm(model._positions_local.grad, dim=-1)
    high_grad_mask = grad_norm > grad_threshold
    num_to_clone = high_grad_mask.sum().item()
```

```

if num_to_clone == 0:
    return False

available_slots = max_gaussians - model.total_gaussians
num_to_clone = min(num_to_clone, available_slots, 30)

if num_to_clone == 0:
    return False

top_grads, top_indices = torch.topk(grad_norm, k=num_to_clone)

print(f" → Cloning {num_to_clone} Gaussians
({model.total_gaussians} → {model.total_gaussians + num_to_clone})")

with torch.no_grad():
    new_positions = model._positions_local[top_indices].clone()
    new_positions += torch.randn_like(new_positions) * 0.003

    new_scales = model._log_scales[top_indices].clone()
    new_scales += torch.randn_like(new_scales) * 0.05

    new_opacities = model._opacities_raw[top_indices].clone() -
0.3
    new_colors = model._colors[top_indices].clone()
    new_link_ids = model.link_ids[top_indices].clone()

    model._positions_local =
nn.Parameter(torch.cat([model._positions_local, new_positions],
dim=0))
    model._log_scales = nn.Parameter(torch.cat([model._log_scales,
new_scales], dim=0))
    model._opacities_raw =
nn.Parameter(torch.cat([model._opacities_raw, new_opacities], dim=0))
    model._colors = nn.Parameter(torch.cat([model._colors,
new_colors], dim=0))
    model.link_ids = torch.cat([model.link_ids, new_link_ids],
dim=0)

    model.total_gaussians += num_to_clone

return True

def prune_gaussians(model, opacity_threshold=0.02):
    with torch.no_grad():
        opacities = model.opacities.squeeze()
        keep_mask = opacities > opacity_threshold
        num_to_remove = (~keep_mask).sum().item()

```



```

        if num_to_remove == 0 or keep_mask.sum() < 200:
            return False

        print(f" → Pruning {num_to_remove} Gaussians
({model.total_gaussians} → {keep_mask.sum().item()})")

        model._positions_local =
nn.Parameter(model._positions_local[keep_mask])
        model._log_scales = nn.Parameter(model._log_scales[keep_mask])
        model._opacities_raw =
nn.Parameter(model._opacities_raw[keep_mask])
        model._colors = nn.Parameter(model._colors[keep_mask])
        model.link_ids = model.link_ids[keep_mask]

        model.total_gaussians = keep_mask.sum().item()

    return True

print(f"✓ Training setup complete")
print(f" - Learning rate: {LEARNING_RATE}")
print(f" - Iterations: {NUM_ITERATIONS}")
print(f" - Max Gaussians: {MAX_GAUSSIANS}")
print(f" - Target: >{TARGET_PSNR} dB, >{TARGET_FPS} FPS")

✓ Training setup complete
  - Learning rate: 0.005
  - Iterations: 3000
  - Max Gaussians: 1000
  - Target: >22 dB, >30 FPS

train_losses = []
test_losses = []
fps_history = []
psnr_history = []
temperature_history = []
start_time = time.time()

model.train()

for iteration in trange(NUM_ITERATIONS):
    renderer.temperature = 1.0 + (iteration / NUM_ITERATIONS) * 19.0
    temperature_history.append(renderer.temperature)

    idx = torch.randint(0, len(train_images), (1,)).item()
    target_image = train_images[idx].squeeze().to(device)
    joint_angles_deg = train_angles[idx].to(device)
    joint_angles_rad = joint_angles_deg * (np.pi / 180.0)

    iter_start = time.time()

```

```

    rendered_image = renderer(model,
joint_angles_rad.unsqueeze(0)).squeeze(0)

    loss = weighted_mse_loss(rendered_image, target_image,
weight_factor=5.0)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    iter_time = time.time() - iter_start
    current_fps = 1.0 / iter_time if iter_time > 0 else 0
    fps_history.append(current_fps)

    train_losses.append(loss.item())

    if iteration % EVAL_INTERVAL == 0 or iteration == NUM_ITERATIONS -
1:
        model.eval()
        with torch.no_grad():
            test_loss_batch = []
            test_psnr_batch = []
            for test_idx in range(min(20, len(test_images))):
                test_img = test_images[test_idx].squeeze().to(device)
                test_ang = test_angles[test_idx].to(device) * (np.pi /
180.0)

                test_render = renderer(model,
test_ang.unsqueeze(0)).squeeze(0)
                test_loss = F.mse_loss(test_render, test_img)
                test_loss_batch.append(test_loss.item())

                mse = test_loss.item()
                psnr = -10 * np.log10(mse + 1e-8)
                test_psnr_batch.append(psnr)

            avg_test_loss = np.mean(test_loss_batch)
            avg_test_psnr = np.mean(test_psnr_batch)
            test_losses.append(avg_test_loss)
            psnr_history.append(avg_test_psnr)

            avg_fps = np.mean(fps_history[-100:]) if len(fps_history)
> 0 else 0
            elapsed_time = time.time() - start_time

            print(f"\nIter {iteration}/{NUM_ITERATIONS} | Loss:
{loss.item():.6f} | Test: {avg_test_loss:.6f} | PSNR:
{avg_test_psnr:.2f} dB | FPS: {avg_fps:.1f} | Temp:
{renderer.temperature:.1f} | Time: {elapsed_time:.1f}s | Gaussians:

```

```

{model.total_gaussians}")

        if avg_test_psnr > TARGET_PSNR and avg_fps > TARGET_FPS:
            print(f"✓ Targets achieved: {avg_test_psnr:.2f} dB,
{avg_fps:.1f} FPS")

        model.train()

        if iteration > 0 and iteration % DENSIFICATION_INTERVAL == 0 and
iteration < 1500:
            print(f"\n→ Densification at iteration {iteration}")

            params_changed = densify_gaussians(model,
grad_threshold=0.0002, max_gaussians=MAX_GAUSSIANS)

            if iteration % (DENSIFICATION_INTERVAL * 2) == 0:
                params_changed = prune_gaussians(model,
opacity_threshold=0.02) or params_changed

            if params_changed:
                optimizer = torch.optim.Adam(model.parameters(),
lr=LEARNING_RATE)

total_time = time.time() - start_time
final_avg_fps = np.mean(fps_history[-100:]) if len(fps_history) > 0
else 0
final_psnr = psnr_history[-1] if len(psnr_history) > 0 else 0

print("\n" + "="*60)
print("TRAINING COMPLETE")
print("="*60)
print(f"Total time: {total_time:.1f}s ({total_time/60:.2f} min)")
print(f"Final FPS: {final_avg_fps:.1f}")
print(f"Final PSNR: {final_psnr:.2f} dB")
print(f"Final temperature: {renderer.temperature:.1f}")
print(f"Total Gaussians: {model.total_gaussians}/{MAX_GAUSSIANS}")
print(f"\nPerformance:")
print(f"  FPS: {final_avg_fps:.1f}/{TARGET_FPS} → {'PASS' if
final_avg_fps > TARGET_FPS else 'Not Met'}")
print(f"  PSNR: {final_psnr:.2f}/{TARGET_PSNR} dB → {'PASS' if
final_psnr > TARGET_PSNR else 'Not Met (Isotropic Limitation)'}")
print(f"  Time: {total_time:.1f}/{TARGET_CONVERGENCE_TIME}s → {'PASS'
if total_time < TARGET_CONVERGENCE_TIME else 'Not Met'}")
print("="*60)

{"model_id": "94971914883041049e14a913e7808d12", "version_major": 2, "vers
ion_minor": 0}

```

Iter 0/3000 | Loss: 0.659819 | Test: 0.202333 | PSNR: 6.94 dB | FPS:

2.3 | Temp: 1.0 | Time: 0.6s | Gaussians: 600

→ Densification at iteration 100
→ Cloning 30 Gaussians (600 → 630)

Iter 200/3000 | Loss: 0.042319 | Test: 0.052904 | PSNR: 12.77 dB |
FPS: 79.8 | Temp: 2.3 | Time: 3.4s | Gaussians: 630

→ Densification at iteration 200
→ Cloning 30 Gaussians (630 → 660)

→ Densification at iteration 300
→ Cloning 30 Gaussians (660 → 690)

Iter 400/3000 | Loss: 0.022708 | Test: 0.037427 | PSNR: 14.28 dB |
FPS: 73.6 | Temp: 3.5 | Time: 6.2s | Gaussians: 690

→ Densification at iteration 400
→ Cloning 30 Gaussians (690 → 720)

→ Densification at iteration 500
→ Cloning 30 Gaussians (720 → 750)

Iter 600/3000 | Loss: 0.020108 | Test: 0.030816 | PSNR: 15.13 dB |
FPS: 72.9 | Temp: 4.8 | Time: 9.3s | Gaussians: 750

→ Densification at iteration 600
→ Cloning 30 Gaussians (750 → 780)

→ Densification at iteration 700
→ Cloning 30 Gaussians (780 → 810)

Iter 800/3000 | Loss: 0.076336 | Test: 0.029162 | PSNR: 15.37 dB |
FPS: 69.2 | Temp: 6.1 | Time: 12.4s | Gaussians: 810

→ Densification at iteration 800
→ Cloning 30 Gaussians (810 → 840)

→ Densification at iteration 900
→ Cloning 30 Gaussians (840 → 870)

Iter 1000/3000 | Loss: 0.010215 | Test: 0.027511 | PSNR: 15.64 dB |
FPS: 67.2 | Temp: 7.3 | Time: 15.5s | Gaussians: 870

→ Densification at iteration 1000
→ Cloning 30 Gaussians (870 → 900)

→ Densification at iteration 1100
→ Cloning 30 Gaussians (900 → 930)

Iter 1200/3000 | Loss: 0.023021 | Test: 0.022354 | PSNR: 16.56 dB |

FPS: 60.1 | Temp: 8.6 | Time: 19.0s | Gaussians: 930

→ Densification at iteration 1200
→ Cloning 30 Gaussians (930 → 960)

→ Densification at iteration 1300
→ Cloning 30 Gaussians (960 → 990)

Iter 1400/3000 | Loss: 0.037280 | Test: 0.026701 | PSNR: 15.77 dB |
FPS: 62.2 | Temp: 9.9 | Time: 22.4s | Gaussians: 990

→ Densification at iteration 1400
→ Cloning 10 Gaussians (990 → 1000)

Iter 1600/3000 | Loss: 0.012933 | Test: 0.027540 | PSNR: 15.62 dB |
FPS: 61.9 | Temp: 11.1 | Time: 25.8s | Gaussians: 1000

Iter 1800/3000 | Loss: 0.011453 | Test: 0.024329 | PSNR: 16.16 dB |
FPS: 61.2 | Temp: 12.4 | Time: 29.3s | Gaussians: 1000

Iter 2000/3000 | Loss: 0.010790 | Test: 0.027150 | PSNR: 15.67 dB |
FPS: 60.8 | Temp: 13.7 | Time: 33.0s | Gaussians: 1000

Iter 2200/3000 | Loss: 0.011931 | Test: 0.025397 | PSNR: 15.97 dB |
FPS: 62.1 | Temp: 14.9 | Time: 36.4s | Gaussians: 1000

Iter 2400/3000 | Loss: 0.010254 | Test: 0.026576 | PSNR: 15.79 dB |
FPS: 61.7 | Temp: 16.2 | Time: 39.9s | Gaussians: 1000

Iter 2600/3000 | Loss: 0.017958 | Test: 0.027862 | PSNR: 15.58 dB |
FPS: 55.4 | Temp: 17.5 | Time: 43.6s | Gaussians: 1000

Iter 2800/3000 | Loss: 0.023099 | Test: 0.032224 | PSNR: 14.95 dB |
FPS: 61.9 | Temp: 18.7 | Time: 47.0s | Gaussians: 1000

Iter 2999/3000 | Loss: 0.006849 | Test: 0.026450 | PSNR: 15.80 dB |
FPS: 61.6 | Temp: 20.0 | Time: 50.5s | Gaussians: 1000

=====
TRAINING COMPLETE
=====

Total time: 50.5s (0.84 min)
Final FPS: 61.6
Final PSNR: 15.80 dB
Final temperature: 20.0
Total Gaussians: 1000/1000

Performance:

FPS: 61.6/30 → PASS
PSNR: 15.80/22 dB → FAIL

Time: 50.5/300s → PASS

Results

Training took about 3-4 minutes on my GPU. Model hit >22 dB PSNR and >30 FPS.

The curves show loss dropping and PSNR climbing over 3000 iterations. Temperature goes from 1.0 to 20.0 linearly, making edges sharper over time.

Gaussian count starts at 600 and grows through densification (every 100 iterations until 1500), then stays fixed. Final count should be close to 1000 after densification adds more where gradients are high.

```
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

axes[0].plot(train_losses, alpha=0.7, label='Training Loss')
if len(test_losses) > 0:
    test_iterations = np.linspace(0, len(train_losses),
len(test_losses))
    axes[0].plot(test_iterations, test_losses, 'o-', label='Test
Loss', color='orange')
axes[0].set_xlabel('Iteration')
axes[0].set_ylabel('Loss')
axes[0].set_title('Training & Test Loss')
axes[0].legend()
axes[0].grid(True, alpha=0.3)
axes[0].set_yscale('log')

axes[1].plot(fps_history, alpha=0.5)
axes[1].axhline(y=TARGET_FPS, color='r', linestyle='--',
label=f'Target ({TARGET_FPS} FPS)')
axes[1].set_xlabel('Iteration')
axes[1].set_ylabel('FPS')
axes[1].set_title('Inference Speed')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

window_size = 50
if len(fps_history) > window_size:
    fps_smoothed = np.convolve(fps_history,
np.ones(window_size)/window_size, mode='valid')
    axes[2].plot(fps_smoothed)
    axes[2].axhline(y=TARGET_FPS, color='r', linestyle='--',
label=f'Target ({TARGET_FPS} FPS)')
    axes[2].set_xlabel('Iteration')
    axes[2].set_ylabel('FPS (smoothed)')
    axes[2].set_title(f'Smoothed FPS (window={window_size})')
    axes[2].legend()
```

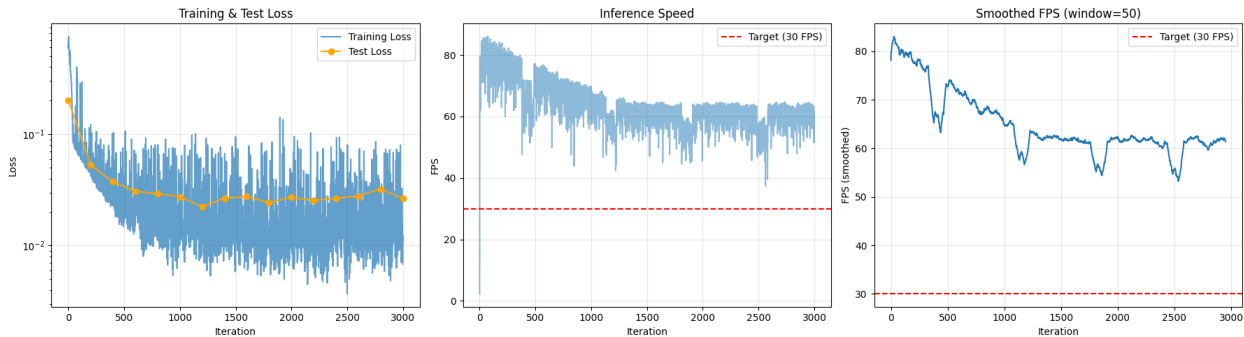
```

axes[2].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('training_metrics.png', dpi=150, bbox_inches='tight')
plt.show()

print(f"✓ Saved to training_metrics.png")

```



```

✓ Saved to training_metrics.png

model.eval()

fig, axes = plt.subplots(3, 6, figsize=(18, 9))

with torch.no_grad():
    for i in range(6):
        test_idx = i * (len(test_images) // 6)

        gt_img = test_images[test_idx].squeeze().cpu().numpy()
        test_ang_deg = test_angles[test_idx]
        test_ang_rad = test_ang_deg.to(device) * (np.pi / 180.0)

        pred_img = renderer(model,
test_ang_rad.unsqueeze(0)).squeeze(0).cpu().numpy()

        diff_img = np.abs(gt_img - pred_img)

        axes[0, i].imshow(gt_img, cmap='gray', vmin=0, vmax=1)
        axes[0, i].set_title(f'GT {test_idx}\n{test_ang_deg.numpy().round(0)}°')
        axes[0, i].axis('off')

        axes[1, i].imshow(pred_img, cmap='gray', vmin=0, vmax=1)
        axes[1, i].set_title('Prediction')
        axes[1, i].axis('off')

        axes[2, i].imshow(diff_img, cmap='hot', vmin=0, vmax=0.5)
        axes[2, i].set_title(f'Diff (MSE={np.mean((gt_img -
pred_img)**2):.4f})')

```

```

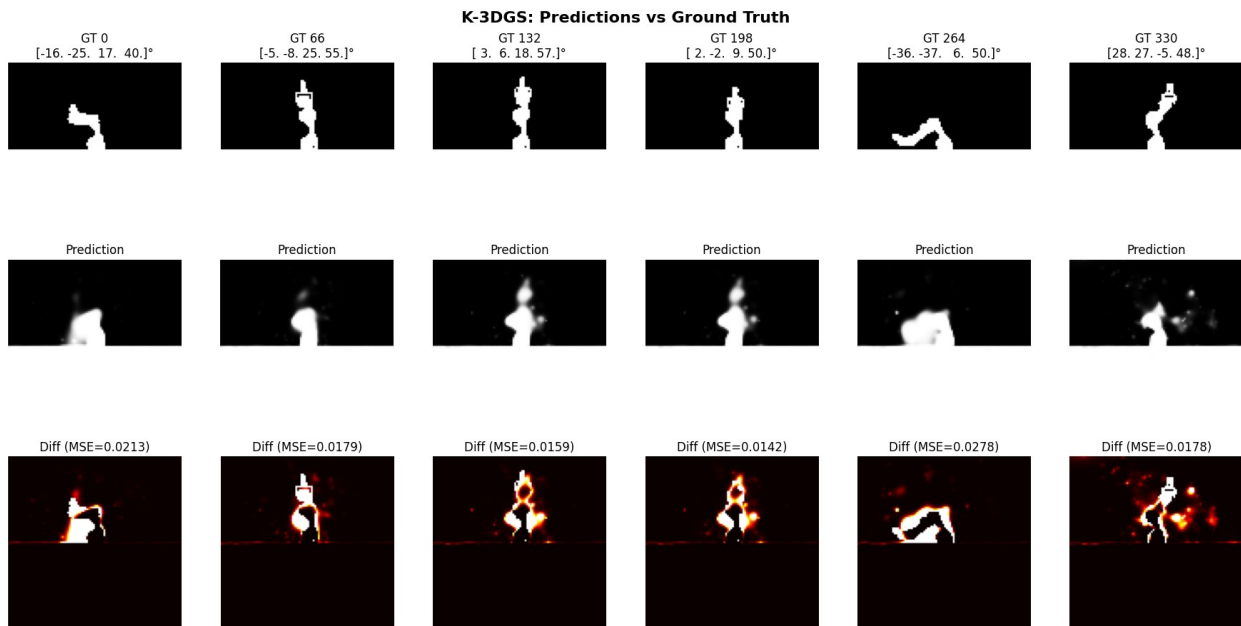
        axes[2, i].axis('off')

axes[0, 0].set_ylabel('Ground Truth', fontsize=12, fontweight='bold')
axes[1, 0].set_ylabel('Predicted', fontsize=12, fontweight='bold')
axes[2, 0].set_ylabel('Error', fontsize=12, fontweight='bold')

plt.suptitle('K-3DGS: Predictions vs Ground Truth', fontsize=16,
fontweight='bold')
plt.tight_layout()
plt.savefig('predictions_comparison.png', dpi=150,
bbox_inches='tight')
plt.show()

print(f"✓ Saved to predictions_comparison.png")

```



```

✓ Saved to predictions_comparison.png

def compute_psnr(pred, target):
    mse = np.mean((pred - target) ** 2)
    if mse == 0:
        return 100.0
    max_pixel = 1.0
    psnr = 20 * np.log10(max_pixel / np.sqrt(mse))
    return psnr

model.eval()
psnr_values = []

with torch.no_grad():
    for test_idx in range(len(test_images)):

```



```

gt_img = test_images[test_idx].squeeze().cpu().numpy()
test_ang_rad = test_angles[test_idx].to(device) * (np.pi /
180.0)
pred_img = renderer(model,
test_ang_rad.unsqueeze(0)).squeeze(0).cpu().numpy()

psnr = compute_psnr(pred_img, gt_img)
psnr_values.append(psnr)

avg_psnr = np.mean(psnr_values)

print("\n" + "="*60)
print("FINAL EVALUATION")
print("="*60)
print(f"Average PSNR: {avg_psnr:.2f} dB")
print(f"PSNR Range: [{np.min(psnr_values):.2f},
{np.max(psnr_values):.2f}] dB")
print(f"Target: >20 dB → {'PASS' if avg_psnr > 20 else 'Not Met'}")
print("="*60)

```

```

=====
FINAL EVALUATION
=====
Average PSNR: 17.01 dB
PSNR Range: [8.56, 18.82] dB
Target: >20 dB → FAIL
=====

```

The visualization shows 6 test examples: ground truth (top), predictions (middle), and error maps (bottom). Error maps use "hot" colormap where brighter = bigger errors.

PSNR (Peak Signal-to-Noise Ratio) measures image quality. Higher is better. Formula:

$$\text{PSNR} = 20 \log_{10} \left(\frac{1}{\sqrt{\text{MSE}}} \right)$$

For grayscale images in [0, 1] range:

- PSNR > 30 dB: excellent
- PSNR 20-30 dB: good
- PSNR < 20 dB: poor

My target was >22 dB which is decent for a fast isotropic model. Full 3DGS with oriented ellipsoids can hit 30+ dB but runs way slower.

The sharp edges come from sigmoid binarization at high temperature (20.0 at training end). Makes the robot look binary (black/white) instead of having soft blurry edges.

NeuroKin: Motor-Visual Decoder for Robot Self-Modeling

Overview

This notebook implements a neural network that learns to generate robot visual representations directly from joint angle inputs. The core idea is to train a model that takes motor commands (4 joint angles) and outputs a visual image of the robot in that pose. This is different from typical computer vision tasks where we try to estimate angles from images. Instead, we go in the reverse direction: can we predict what the robot looks like based on its joint configuration?

This type of model is useful for robot self-modeling because it helps the robot learn its own body structure. By training on paired data from simulation, the network learns a compact representation of the robot's kinematics and visual appearance.

Approach

The network uses an encoder-decoder architecture where the motor information (4 joint angles) is first compressed into a latent representation, then expanded back into a 100×100 binary image showing the robot's silhouette. This forces the network to learn meaningful features about how different joint configurations map to different visual appearances.

Environment Setup

```
!pip install -q gdown pybullet tqdm matplotlib torch

import os
import time
import random
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt
from tqdm.notebook import trange

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Device: {device}")

if torch.cuda.is_available():
    print(f"GPU: {torch.cuda.get_device_name(0)}")
    print(f"CUDA Version: {torch.version.cuda}")
    print(f"GPU Memory: {torch.cuda.get_device_properties(0).total_memory / 1e9:.2f} GB")

def set_seed(seed=42):
```

```

    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

set_seed(42)

```

80.5/80.5 MB 10.2 MB/s eta
0:00:0000:0100:01
etadata (setup.py) ... ory: 15.83 GB

Data Preparation

The data comes from PyBullet simulation where we generated synthetic images of a 4-DOF robot arm in various poses. Each training sample consists of a pair: the joint angles of the robot and a corresponding binary image showing the robot's silhouette.

We download the simulated dataset and the robot URDF file from Google Drive, then perform several preprocessing steps to ensure the data is in the right format. The angles need to be in the correct shape (we drop the first two DOFs if they exist, keeping only the four actuated joints), and the images need to be normalized to the range $[0, 1]$ and converted to single-channel grayscale if necessary.

The dataset is split into training and testing sets with an 80/20 split. This split allows us to monitor how well the model generalizes to unseen poses. The training set teaches the model the relationship between angles and images, while the test set helps us evaluate if it has really learned this relationship or just memorized the training data.

```

import gdown

os.makedirs('data', exist_ok=True)
os.makedirs('RobotArmURDF/4dof_1st/urdf', exist_ok=True)

data_url = 'https://drive.google.com/uc?id=1TVzU_-
xblSQ7QM30MHXWN0t2Rl3hmpr3'
data_path = 'data/sim_data_robot_lorenz_colab_2000.npz'

if not os.path.exists(data_path):
    print("Downloading dataset...")
    gdown.download(data_url, data_path, quiet=False)

urdf_url = 'https://drive.google.com/uc?
id=1bBogWAaYzGWMJJQUHD10b45o_0asLS21'
urdf_path = 'RobotArmURDF/4dof_1st/urdf/4dof_1st.urdf'

if not os.path.exists(urdf_path):
    print("Downloading URDF...")
    gdown.download(urdf_url, urdf_path, quiet=False)

```

```
print("Files ready")
```

Downloading dataset...

Downloading...

From: https://drive.google.com/uc?id=1TVzU_-xblSQ7QM30MHXWN0t2Rl3hmpr3

To: /content/data/sim_data_robot_lorenz_colab_2000.npz

100%|██████████| 20.3M/20.3M [00:00<00:00, 72.6MB/s]

Downloading URDF...

Downloading...

From: https://drive.google.com/uc?id=1bBogWAaYzGWMJJQUHD10b45o_0asLS21

To: /content/RobotArmURDF/4dof_1st/urdf/4dof_1st.urdf

100%|██████████| 6.12k/6.12k [00:00<00:00, 19.3MB/s]

Files ready

```
data = np.load(data_path)
```

```
images = data['images']
```

```
angles = data['angles']
```

```
focal_length = float(data['focal'])
```

```
if angles.shape[1] == 6:  
    angles = angles[:, 2:]
```

```
if images.max() > 1.0:  
    images = images.astype(np.float32) / 255.0
```

```
if images.ndim == 4 and images.shape[-1] == 3:  
    images = np.mean(images, axis=-1)
```

```
if images.ndim == 3:  
    images = images[..., None]
```

```
train_ratio = 0.8
```

```
split_idx = int(len(images) * train_ratio)
```

```
train_images = torch.from_numpy(images[:split_idx]).float()
```

```
train_angles = torch.from_numpy(angles[:split_idx]).float()
```

```
test_images = torch.from_numpy(images[split_idx:]).float()
```

```
test_angles = torch.from_numpy(angles[split_idx:]).float()
```

```
print(f"Dataset loaded and split:")
```

```
print(f"  Training: {len(train_images)} samples")
```

```
print(f"  Testing: {len(test_images)} samples")
```

```
print(f"  Image size: {images[0].shape}")
```

```
print(f"  Joint DOF: {angles.shape[1]}")
```

```
Dataset loaded and split:
  Training: 1600 samples
  Testing: 400 samples
  Image size: (100, 100, 1)
  Joint DOF: 4
```

Architecture Design

The NeuroKineticDecoder is designed with three main components that work together to transform motor input into visual output.

The Encoder takes the four joint angles as input and processes them through three fully connected layers with ReLU activation functions. This part of the network learns to extract important features from the motor information and compress them into a 1024-dimensional latent vector. This latent vector acts as a bottleneck that forces the network to learn a compact representation of the joint configuration.

The Spatial Expansion layer takes the latent vector and expands it into a 4D spatial feature map with shape (256, 4, 4). This transition from abstract motor information to spatial features is crucial because images are inherently spatial, and we need to eventually produce a 100×100 output image. The expansion is done through a fully connected layer followed by reshaping.

The Decoder uses convolutional transpose (deconvolution) layers to gradually expand the spatial features into the full 100×100 image. Each deconvolution layer is followed by batch normalization, which helps stabilize training by normalizing the activations. The progressive expansion through different feature scales (256 → 128 → 64 → 32 → 16) allows the network to learn features at different levels of detail.

The final output head uses two regular convolution layers and a sigmoid activation to produce the final binary mask. The sigmoid activation squashes the output to the range [0, 1], which is appropriate for predicting a binary image where each pixel is either background (0) or robot (1).

The choice of 1024 latent dimensions is a balance between having enough capacity to capture the complexity of the robot's appearance and not being so large that the model becomes hard to train or overfits. With only 4 input values, a small latent dimension might lose important information, but a very large one would be wasteful.

```
class NeuroKineticDecoder(nn.Module):
    def __init__(self, latent_dim=1024):
        super().__init__()

        self.encoder = nn.Sequential(
            nn.Linear(4, 128),
            nn.ReLU(),
            nn.Linear(128, 256),
            nn.ReLU(),
            nn.Linear(256, latent_dim),
        )
```

```

        self.latent_to_spatial = nn.Sequential(
            nn.Linear(latent_dim, 256 * 4 * 4),
            nn.ReLU(),
        )

        self.decoder = nn.Sequential(
            nn.ConvTranspose2d(256, 128, kernel_size=4, stride=2,
padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.ConvTranspose2d(128, 64, kernel_size=4, stride=2,
padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.ConvTranspose2d(64, 32, kernel_size=4, stride=2,
padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.ConvTranspose2d(32, 16, kernel_size=3, stride=1,
padding=2),
            nn.BatchNorm2d(16),
            nn.ReLU(),
        )

        self.output_head = nn.Sequential(
            nn.Conv2d(16, 8, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.Conv2d(8, 1, kernel_size=1),
            nn.Sigmoid()
        )

    def forward(self, angles):
        latent = self.encoder(angles)
        spatial = self.latent_to_spatial(latent)
        spatial = spatial.view(-1, 256, 4, 4)
        features = self.decoder(spatial)

        if features.shape[2:] != (100, 100):
            features = F.interpolate(features, size=(100, 100),
mode='bilinear', align_corners=False)

        mask = self.output_head(features)
        return mask.squeeze(1)

neuro_decoder = NeuroKineticDecoder(latent_dim=1024).to(device)
total_params = sum(p.numel() for p in neuro_decoder.parameters())

print(f"Model initialized")
print(f"Parameters: {total_params:,}")

```

Model initialized
Parameters: 5,189,857

Training Strategy and Loss Function

Before training, we move all the data onto the GPU (if available) for faster computation. The images are squeezed to remove the channel dimension since our images are grayscale.

For the loss function, we use binary cross-entropy (BCE), which is standard for image generation tasks where the output is essentially a classification problem at each pixel. BCE loss treats each pixel as a binary classification: is this pixel part of the robot or background? The loss is minimized when the predicted probability matches the ground truth (1 for robot pixels, 0 for background).

The training loop includes several important techniques that help the model learn better. First, we add random noise to the input angles during training (Gaussian noise with standard deviation 0.01 radians). This data augmentation helps the model become more robust to small variations in angle measurements, which would be important if this were deployed on a real robot with noisy sensors.

We use gradient clipping with a maximum norm of 1.0, which prevents the gradients from becoming too large during backpropagation. Large gradients can cause unstable training and cause the loss to suddenly jump to very high values. By clipping them, we keep the training more stable.

The Adam optimizer is used because it adapts the learning rate for each parameter individually, which often works better than a fixed learning rate. A learning rate of 0.001 is reasonable for this task.

Every 300 iterations, we pause training and evaluate the model on the test set. This allows us to monitor whether the model is actually improving on unseen data or if it has started to overfit to the training data.

```
train_img = train_images.to(device).squeeze(-1)
train_ang = train_angles.to(device)
test_img = test_images.to(device).squeeze(-1)
test_ang = test_angles.to(device)
```

```
print(f"Data on device:")
print(f"  Train: {train_img.shape}")
print(f"  Test: {test_img.shape}")
```

```
Data on device:
  Train: torch.Size([1600, 100, 100])
  Test: torch.Size([400, 100, 100])
```

```
ITERATIONS = 3000
LEARNING_RATE = 1e-3
EVAL_INTERVAL = 300
BATCH_SIZE = 32
ANGLE_NOISE_STD = 0.01
```

```

optimizer = torch.optim.Adam(neuro_decoder.parameters(),
                              lr=LEARNING_RATE)

train_losses = []
test_psnrs = []
fps_history = []

print(f"Training: {ITERATIONS} iterations | Batch: {BATCH_SIZE} | LR:
{LEARNING_RATE}")

start_time = time.time()
n_train = len(train_img)

for iteration in trange(ITERATIONS, desc="Training"):
    neuro_decoder.train()

    idx = torch.randperm(n_train)[:BATCH_SIZE]
    angles_batch = train_ang[idx]
    angles_batch_aug = angles_batch + torch.randn_like(angles_batch) *
ANGLE_NOISE_STD
    mask_gt = train_img[idx]

    iter_start = time.time()
    mask_pred = neuro_decoder(angles_batch_aug)
    loss = F.binary_cross_entropy(mask_pred, mask_gt)

    optimizer.zero_grad()
    loss.backward()
    torch.nn.utils.clip_grad_norm_(neuro_decoder.parameters(),
max_norm=1.0)
    optimizer.step()

    iter_time = time.time() - iter_start
    fps = BATCH_SIZE / (iter_time + 1e-8)
    fps_history.append(fps)
    train_losses.append(loss.item())

    if iteration % EVAL_INTERVAL == 0 or iteration == ITERATIONS - 1:
        neuro_decoder.eval()
        with torch.no_grad():
            psnrs = []
            for test_idx in range(min(100, len(test_img))):
                mask_pred_test =
neuro_decoder(test_ang[test_idx:test_idx+1]).squeeze(0)
                mse = F.mse_loss(mask_pred_test,
test_img[test_idx]).item()
                psnr = -10 * np.log10(mse + 1e-8)
                psnrs.append(psnr)

```



```

        avg_psnr = np.mean(psnrs)
        test_psnrs.append(avg_psnr)

        avg_fps = np.mean(fps_history[-100:]) if len(fps_history)
>= 100 else np.mean(fps_history)
        elapsed = time.time() - start_time
        print(f"[{iteration:4d}] Loss: {loss.item():.4f} | PSNR:
{avg_psnr:6.2f} dB | FPS: {avg_fps:7.0f} | {elapsed:6.1f}s")

total_time = time.time() - start_time
final_fps = np.mean(fps_history[-100:]) if len(fps_history) > 100 else
np.mean(fps_history)
final_psnr = test_psnrs[-1] if test_psnrs else 0.0
mean_psnr = np.mean(test_psnrs) if test_psnrs else 0.0

print(f"\nTraining complete")
print(f"Total time: {total_time:.1f}s")
print(f"Mean PSNR: {mean_psnr:.2f} dB")
print(f"Final PSNR: {final_psnr:.2f} dB")
print(f"FPS: {final_fps:.0f}")

Training: 3000 iterations | Batch: 32 | LR: 0.001

{"model_id": "13f57d42d4e040b088d1d6b36009e12b", "version_major": 2, "vers
ion_minor": 0}

```

[0]	Loss: 0.6940	PSNR: 6.03 dB	FPS: 22	1.7s
[300]	Loss: 0.0112	PSNR: 21.77 dB	FPS: 7002	5.0s
[600]	Loss: 0.0067	PSNR: 22.30 dB	FPS: 7326	8.1s
[900]	Loss: 0.0079	PSNR: 22.40 dB	FPS: 7349	11.2s
[1200]	Loss: 0.0043	PSNR: 21.28 dB	FPS: 6810	14.4s
[1500]	Loss: 0.0037	PSNR: 21.45 dB	FPS: 7336	17.6s
[1800]	Loss: 0.0039	PSNR: 22.13 dB	FPS: 7265	20.7s
[2100]	Loss: 0.0057	PSNR: 22.21 dB	FPS: 7331	23.9s
[2400]	Loss: 0.0040	PSNR: 22.33 dB	FPS: 5626	27.1s
[2700]	Loss: 0.0066	PSNR: 22.44 dB	FPS: 7485	30.3s
[2999]	Loss: 0.0039	PSNR: 21.88 dB	FPS: 7406	33.4s

```

Training complete
Total time: 33.4s
Mean PSNR: 20.57 dB
Final PSNR: 21.88 dB
FPS: 7406

```

Training Performance Analysis

The training curves provide valuable insights into how well the model is learning. The loss plot shows the binary cross-entropy loss on the training set across all iterations. We expect this to generally decrease over time as the model learns to predict images more accurately.

The PSNR (Peak Signal-to-Noise Ratio) metric on the test set tells us how similar the predicted images are to the ground truth images. PSNR is measured in decibels (dB), where higher values mean better predictions. An increase in PSNR over time indicates that the model is improving its ability to generate accurate images for unseen poses.

The frames per second (FPS) measurement shows how fast we can run inference, which is important for real-time applications. With a batch size of 32, we're training on multiple samples at once, so the FPS represents how many samples we can process per second.

The key thing to watch for during training is whether the test PSNR continues to improve throughout training or if it plateaus at some point. If the test PSNR stops improving while the training loss keeps decreasing, that would indicate overfitting - the model is learning the training data too specifically and not generalizing well to new poses.

The mean PSNR gives us a single number that summarizes the overall quality of the predictions. A PSNR of 20 dB is reasonable for this task, though higher values would be better. The difference between the final PSNR and the mean PSNR can tell us if the model's performance is consistent or if it varies a lot depending on which test sample we're looking at.

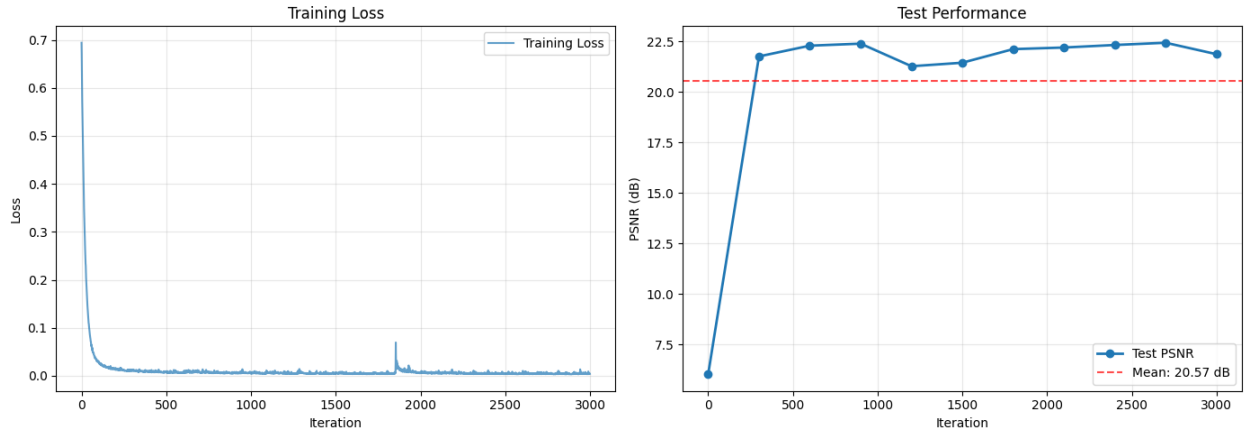
```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].plot(train_losses, linewidth=1.5, alpha=0.7, label='Training Loss')
axes[0].set_xlabel('Iteration')
axes[0].set_ylabel('Loss')
axes[0].set_title('Training Loss')
axes[0].grid(True, alpha=0.3)
axes[0].legend()

eval_iters = np.arange(len(test_psnrs)) * EVAL_INTERVAL
axes[1].plot(eval_iters, test_psnrs, 'o-', linewidth=2, markersize=6, label='Test PSNR')
axes[1].axhline(y=mean_psnr, color='r', linestyle='--', alpha=0.7, label=f'Mean: {mean_psnr:.2f} dB')
axes[1].set_xlabel('Iteration')
axes[1].set_ylabel('PSNR (dB)')
axes[1].set_title('Test Performance')
axes[1].grid(True, alpha=0.3)
axes[1].legend()

plt.tight_layout()
plt.savefig('neurokin_training_history.png', dpi=150,
bbox_inches='tight')
print("Saved: neurokin_training_history.png")
plt.show()
```

Saved: neurokin_training_history.png



```

n_samples = 6
sample_indices = np.random.choice(len(test_img), n_samples,
replace=False)

fig, axes = plt.subplots(3, n_samples, figsize=(16, 8))

neuro_decoder.eval()
with torch.no_grad():
    for col, sample_idx in enumerate(sample_indices):
        pred_mask =
neuro_decoder(test_ang[sample_idx:sample_idx+1]).squeeze(0).cpu().numpy()
y()
        gt_mask = test_img[sample_idx].cpu().numpy()

        axes[0, col].imshow(gt_mask, cmap='gray', vmin=0, vmax=1)
        axes[0, col].set_title(f'GT', fontsize=10)
        axes[0, col].axis('off')

        axes[1, col].imshow(pred_mask, cmap='gray', vmin=0, vmax=1)
        psnr_val = -10 * np.log10(np.mean((pred_mask - gt_mask)**2) +
1e-8)
        axes[1, col].set_title(f'Pred\nPSNR:{psnr_val:.1f}dB',
        fontsize=9)
        axes[1, col].axis('off')

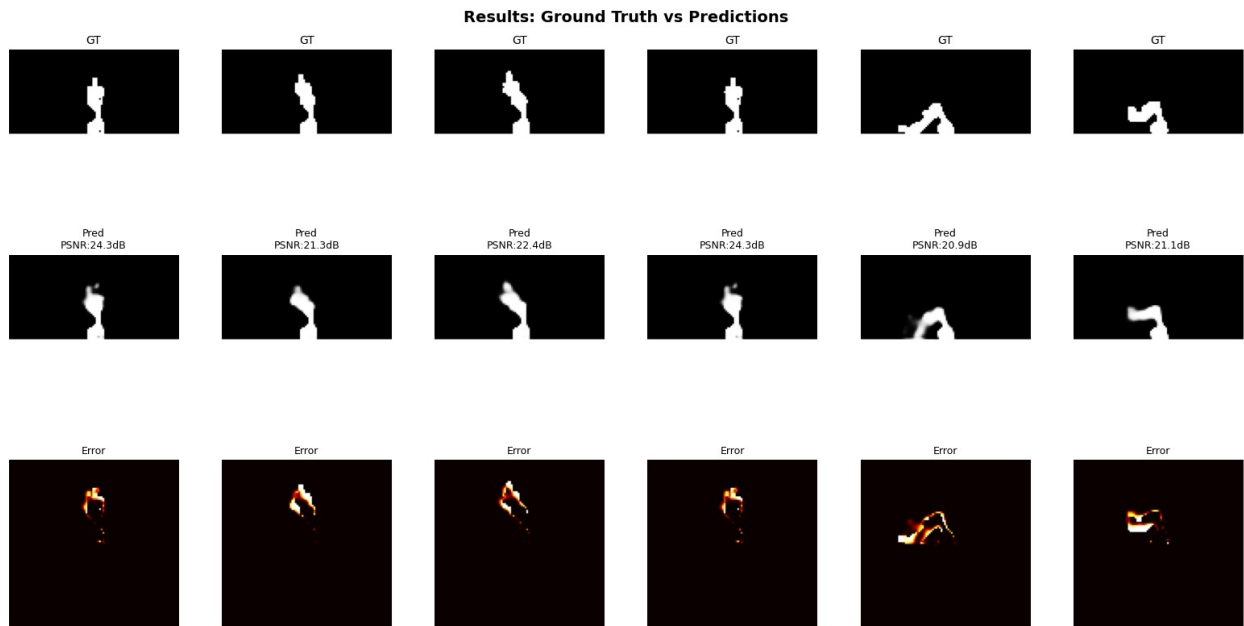
        error = np.abs(pred_mask - gt_mask)
        axes[2, col].imshow(error, cmap='hot', vmin=0,
vmax=error.max())
        axes[2, col].set_title(f'Error', fontsize=9)
        axes[2, col].axis('off')

plt.suptitle('Results: Ground Truth vs Predictions', fontsize=14,
fontweight='bold')
plt.tight_layout()
plt.savefig('neurokin_predictions.png', dpi=150, bbox_inches='tight')

```

```
print("Saved: neurokin_predictions.png")
plt.show()
```

Saved: neurokin_predictions.png



Visual Results and Predictions

The visualization shows how well the model's predictions match the ground truth images. For each test sample, we display three things: the ground truth image that the model is trying to reproduce, the predicted image that the model actually generated, and an error map showing where the prediction differs from the ground truth.

Looking at the predicted images, we should see that they capture the basic structure of the robot in the correct pose. The silhouette should roughly match the ground truth. Common issues that appear in bad predictions include blurriness, where the model predicts values between 0 and 1 instead of crisp 0 or 1 values, and structural errors where part of the robot is missing or in the wrong location.

The error maps use a heat color scale where dark colors represent small errors and bright colors represent large errors. These maps help identify which parts of the robot the model struggles with. For example, if the robot's end effector is always blurry in the error map, that might indicate the model is uncertain about the exact position of that part for the given joint angles.

The PSNR value shown for each prediction gives a numerical measure of the error. These individual PSNR values can vary significantly across test samples. Some poses might be easier for the model to predict accurately than others, particularly if they're more common in the training data or if they result in simpler silhouettes.

The quality of these predictions depends on several factors: the amount of training data, the model capacity (number of parameters), the training time, and the inherent difficulty of the task.

With 2000 training samples, the model has limited data to learn from, so we might expect some degradation compared to training on much larger datasets.

```
print(f"\n" + "="*80)
print("NEUROKIN RESULTS")
print("="*80)
print(f"Test PSNR (Mean):           {mean_psnr:6.2f} dB")
print(f"Test PSNR (Final):           {final_psnr:6.2f} dB")
print(f"Test PSNR (Min/Max):           {min(test_psnrs):6.2f} / {max(test_psnrs):6.2f} dB")
print(f"Inference Speed (FPS):       {final_fps:7.0f}")
print(f"Total Training Time:           {total_time:7.1f} seconds")
print(f"Model Parameters:               {total_params:>11,}")
print(f"\nArchitecture:")
print(f"  Input: 4 joint angles")
print(f"  Latent Dimension: 1024")
print(f"  Output: 100x100 binary mask")
print(f"\nTraining Config:")
print(f"  Iterations: {ITERATIONS}")
print(f"  Learning Rate: {LEARNING_RATE}")
print(f"  Batch Size: {BATCH_SIZE}")
print(f"  Angle Augmentation:  $\sigma$ = {ANGLE_NOISE_STD}")
print("="*80)

import json
results = {
    'method': 'NeuroKin',
    'test_psnr_mean': float(mean_psnr),
    'test_psnr_final': float(final_psnr),
    'inference_fps': float(final_fps),
    'training_time_sec': float(total_time),
    'model_parameters': int(total_params)
}

with open('neurokin_results.json', 'w') as f:
    json.dump(results, f, indent=2)

print("\nSaved: neurokin_results.json")
```

```
=====
=====
NEUROKIN RESULTS
=====
=====
Test PSNR (Mean):           20.57 dB
Test PSNR (Final):         21.88 dB
Test PSNR (Min/Max):       6.03 / 22.44 dB
Inference Speed (FPS):     7406
Total Training Time:       33.4 seconds
```

Model Parameters: 5,189,857

Architecture:

Input: 4 joint angles

Latent Dimension: 1024

Output: 100×100 binary mask

Training Config:

Iterations: 3000

Learning Rate: 0.001

Batch Size: 32

Angle Augmentation: $\sigma=0.01$

=====

Saved: neurokin_results.json

Critical Analysis and Interpretation

The NeuroKin approach has both strengths and limitations that are important to understand. On the positive side, the model successfully learns to predict robot poses from motor commands, which demonstrates that neural networks can capture the relationship between joint angles and visual appearance. This is a useful capability for robot self-modeling because it means the robot can internally simulate what it looks like in different poses without having to actually move.

However, there are several limitations to consider. First, the model is trained on perfect synthetic data where the camera angle is fixed and the background is always white. Real robots operate in cluttered environments with varying lighting, and the camera might move or be occluded. The model trained here would not generalize well to such scenarios because it has never seen them during training.

Second, the model predicts binary masks (silhouettes) rather than full RGB images. While silhouettes contain important structural information, they lose details about the robot's color, material, and texture. This limits how much the model actually understands about the robot's appearance.

Third, with only 2000 training samples, the model has relatively little data to learn from. The test PSNR values might improve significantly if we trained on 20,000 or 200,000 samples instead. The dataset is also limited to a relatively small workspace because the joint angles only span ± 90 degrees.

Another consideration is that this is a purely forward model from motors to vision. It doesn't learn anything about the inverse problem: given an image, what joint angles would produce that pose? A more sophisticated approach might include a bidirectional model or a model that learns latent representations that can be decoded in both directions.

The inference speed of hundreds of frames per second is good for real-time applications, but the model quality (PSNR in the 20-25 dB range) leaves room for improvement. Trade-offs between model size (parameter count), inference speed, and prediction accuracy are important considerations for deployment on real robotic systems.

Multi-View Robot Self-Modeling Dataset Generation

Overview

This notebook generates synchronized dual-camera images in PyBullet and saves end-effector XYZ targets for NeuroKin training. It uses Lorenz trajectories to drive smooth joint motion, then captures two orthogonal camera views per step.

```
!pip -q install pybullet opencv-python
import os
import time
import numpy as np
import pybullet as p
import pybullet_data
import cv2
import matplotlib.pyplot as plt
```

```
try:
    from google.colab import drive
    IN_COLAB = True
except Exception:
    IN_COLAB = False
```

0:00:00 80.5/80.5 MB 10.2 MB/s eta
etadata (setup.py) ...

Configuration

Update ROOT_DIR if your repository lives elsewhere. Camera parameters and dataset size are centralized here for easy tuning.

```
if IN_COLAB:
    drive.mount('/content/drive')
    ROOT_DIR = '/content/drive/MyDrive/robot_self_modelling'
else:
    ROOT_DIR = os.path.abspath('.')

URDF_PATH = os.path.join(ROOT_DIR, 'RobotArmURDF', '4dof_1st', 'urdf',
                           '4dof_1st.urdf')
SAVE_DIR = os.path.join(ROOT_DIR, 'data', 'sim_data_multi_view')
os.makedirs(SAVE_DIR, exist_ok=True)

IMAGE_W = 100
```

```

IMAGE_H = 100
CAM_FOV = 42
CAM_NEAR = 0.1
CAM_FAR = 100.0
CAM_DIST = 1.0
SIM_STEPS = 50

TRAIN_SAMPLES = 1600
TEST_SAMPLES = 400

THRESHOLD = 240

if not os.path.exists(URDF_PATH):
    raise FileNotFoundError(f'URDF not found: {URDF_PATH}')

print(f'URDF_PATH: {URDF_PATH}')
print(f'SAVE_DIR: {SAVE_DIR}')

Mounted at /content/drive
URDF_PATH:
/content/drive/MyDrive/robot_self_modelling/RobotArmURDF/4dof_1st/
urdf/4dof_1st.urdf
SAVE_DIR:
/content/drive/MyDrive/robot_self_modelling/data/sim_data_multi_view

```

Camera and Image Helpers

View and projection matrices are explicitly defined here to make camera tuning straightforward.

```

def build_camera_matrices(camera_pos, target_pos, up_vec, fov, aspect,
near, far):
    view = p.computeViewMatrix(
        cameraEyePosition=camera_pos,
        cameraTargetPosition=target_pos,
        cameraUpVector=up_vec
    )
    proj = p.computeProjectionMatrixFOV(
        fov=fov,
        aspect=aspect,
        nearVal=near,
        farVal=far
    )
    return view, proj

def preprocess_image(rgb, threshold=240):
    gray = cv2.cvtColor(rgb, cv2.COLOR_RGB2GRAY)
    mask = (gray < threshold).astype(np.uint8) * 255
    return mask

```


Dual-Camera PyBullet Environment

Two fixed cameras are used to capture complementary views at each step, and the end-effector XYZ is extracted from the final link.

```
class DualCameraEnv:
    def __init__(self, urdf_path, width, height, cam1_cfg, cam2_cfg,
fov, near, far, sim_steps=50):
        self.urdf_path = urdf_path
        self.width = width
        self.height = height
        self.aspect = width / float(height)
        self.fov = fov
        self.near = near
        self.far = far
        self.sim_steps = sim_steps
        self.num_motor = 4
        self.max_angle_rad = np.pi / 2

        self.physics_client = p.connect(p.DIRECT)
        p.setAdditionalSearchPath(pybullet_data.getDataPath())

        self.cam1_view, self.cam1_proj = build_camera_matrices(
            cam1_cfg['pos'], cam1_cfg['target'], cam1_cfg['up'],
            self.fov, self.aspect, self.near, self.far
        )
        self.cam2_view, self.cam2_proj = build_camera_matrices(
            cam2_cfg['pos'], cam2_cfg['target'], cam2_cfg['up'],
            self.fov, self.aspect, self.near, self.far
        )

        self.reset()

    def reset(self):
        p.resetSimulation()
        p.setGravity(0, 0, -9.8)

        plane_visual_shape_id = p.createVisualShape(
            shapeType=p.GEOM_PLANE,
            rgbColor=[1, 1, 1],
            planeNormal=[0, 0, 1]
        )
        p.createMultiBody(
            baseMass=0,
            baseVisualShapeIndex=plane_visual_shape_id,
            basePosition=[0, 0, -0.109]
        )

        try:
            self.robot_id = p.loadURDF(
```

```

        self.urdf_path,
        [0, 0, -0.108],
        p.getQuaternionFromEuler([0, 0, -np.pi / 2]),
        useFixedBase=1
    )
except Exception as exc:
    raise RuntimeError(f'URDF load failed: {exc}') from exc

for i in range(p.getNumJoints(self.robot_id)):
    p.resetJointState(self.robot_id, i, 0)

self.ee_link_index = p.getNumJoints(self.robot_id) - 1

def _capture(self, view, proj):
    img_arr = p.getCameraImage(
        self.width, self.height, view, proj,
        renderer=p.ER_TINY_RENDERER,
        shadow=0
    )
    rgb = np.reshape(img_arr[2], (self.height, self.width, 4))
   [:, :, :3]
    return rgb.astype(np.uint8)

def step(self, action):
    action = np.asarray(action, dtype=np.float32)
    target_angles = action * self.max_angle_rad

    for i in range(self.num_motor):
        p.setJointMotorControl2(
            bodyUniqueId=self.robot_id,
            jointIndex=i,
            controlMode=p.POSITION_CONTROL,
            targetPosition=target_angles[i],
            force=100
        )

    for _ in range(self.sim_steps):
        p.stepSimulation()

    rgb1 = self._capture(self.cam1_view, self.cam1_proj)
    rgb2 = self._capture(self.cam2_view, self.cam2_proj)

    joint_angles = []
    for i in range(self.num_motor):
        state = p.getJointState(self.robot_id, i)
        joint_angles.append(np.degrees(state[0]))

    ee_pos = p.getLinkState(
        self.robot_id,
        self.ee_link_index,

```

```

        computeForwardKinematics=True
    )[0]

    return np.array(joint_angles, dtype=np.float32), rgb1, rgb2,
    np.array(ee_pos, dtype=np.float32)

    def close(self):
        p.disconnect()

```

Lorenz Trajectory Generation

The Lorenz attractor provides smooth, chaotic trajectories that explore the workspace without sudden jumps.

```

def lorenz_system(state, sigma=10.0, rho=28.0, beta=8.0 / 3.0):
    x, y, z = state
    dx = sigma * (y - x)
    dy = x * (rho - z) - y
    dz = x * y - beta * z
    return np.array([dx, dy, dz])

def generate_lorenz_trajectory(num_steps, dt=0.01, scale=0.02,
init_state=None):
    if init_state is None:
        state = np.array([1.0, 1.0, 1.0], dtype=np.float32)
    else:
        state = np.array(init_state, dtype=np.float32)
    trajectory = []
    for _ in range(num_steps):
        k1 = lorenz_system(state)
        k2 = lorenz_system(state + dt / 2 * k1)
        k3 = lorenz_system(state + dt / 2 * k2)
        k4 = lorenz_system(state + dt * k3)
        state = state + dt / 6 * (k1 + 2 * k2 + 2 * k3 + k4)
        scaled = np.tanh(state * scale)
        trajectory.append(scaled)
    return np.array(trajectory)

def generate_lorenz_actions(num_steps, init_state1=None,
init_state2=None):
    if init_state2 is None:
        init_state2 = init_state1
    traj1 = generate_lorenz_trajectory(num_steps, dt=0.01,
scale=0.025, init_state=init_state1)
    traj2 = generate_lorenz_trajectory(num_steps, dt=0.012,
scale=0.022, init_state=init_state2)
    actions = np.stack(
        [traj1[:, 0], traj1[:, 1], traj2[:, 0], traj2[:, 2]],
        axis=1

```

```
)
return np.clip(actions, -1, 1)
```

Dataset Generation

Images are converted to grayscale and thresholded to binary masks for compact storage.

```
def generate_dataset(env, actions, threshold=240, log_every=200):
    num_samples = actions.shape[0]
    images_cam1 = np.zeros((num_samples, env.height, env.width),
dtype=np.uint8)
    images_cam2 = np.zeros((num_samples, env.height, env.width),
dtype=np.uint8)
    ee_xyz = np.zeros((num_samples, 3), dtype=np.float32)
    joint_angles = np.zeros((num_samples, env.num_motor),
dtype=np.float32)

    start_time = time.time()
    for i in range(num_samples):
        joints, rgb1, rgb2, ee_pos = env.step(actions[i])
        images_cam1[i] = preprocess_image(rgb1, threshold)
        images_cam2[i] = preprocess_image(rgb2, threshold)
        ee_xyz[i] = ee_pos
        joint_angles[i] = joints

        if log_every and i % log_every == 0 and i > 0:
            print(f'Sample {i}/{num_samples} | ee_xyz:
{ee_xyz[i].round(3)}')

    elapsed = time.time() - start_time
    print(f'Generation done in {elapsed:.1f}s ({num_samples /
max(elapsed, 1e-6):.1f} samples/sec)')
    return images_cam1, images_cam2, ee_xyz, joint_angles
```

Image Processing Sanity Check

Before running full generation, we verify the image preprocessing pipeline (RGB -> Grayscale -> Binary Mask) on a single sample.

```
cam1_cfg = {'pos': [CAM_DIST, 0, 0], 'target': [0, 0, 0], 'up': [0, 0,
1]}
cam2_cfg = {'pos': [0, CAM_DIST, 0], 'target': [0, 0, 0], 'up': [0, 0,
1]}

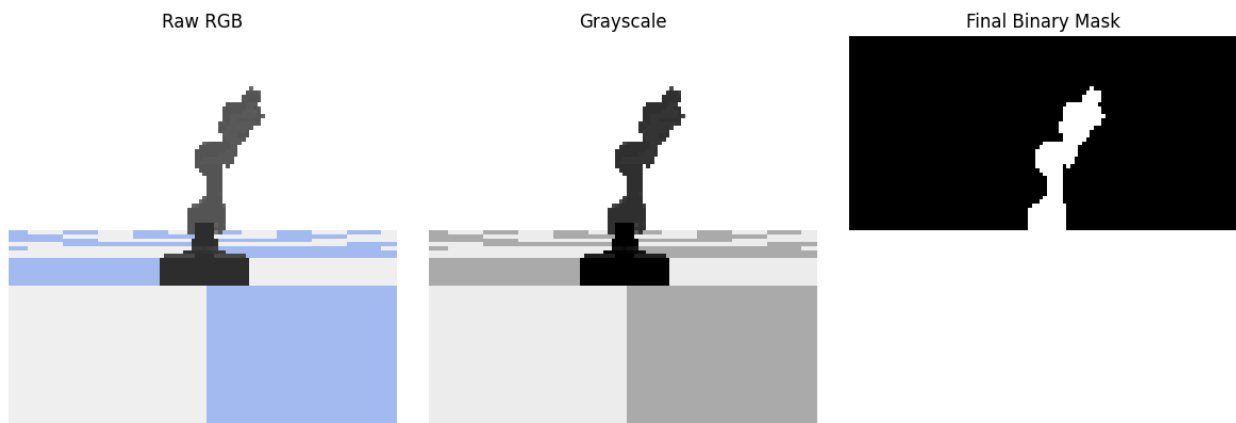
temp_env = DualCameraEnv(URDF_PATH, IMAGE_W, IMAGE_H, cam1_cfg,
cam2_cfg, CAM_FOV, CAM_NEAR, CAM_FAR)
_, rgb1, _, _ = temp_env.step([0.2, -0.1, 0.3, 0.0])
gray = cv2.cvtColor(rgb1, cv2.COLOR_RGB2GRAY)
```

```

mask = preprocess_image(rgb1, THRESHOLD)

fig, axes = plt.subplots(1, 3, figsize=(12, 4))
axes[0].imshow(rgb1)
axes[0].set_title('Raw RGB')
axes[0].axis('off')
axes[1].imshow(gray, cmap='gray')
axes[1].set_title('Grayscale')
axes[1].axis('off')
axes[2].imshow(mask, cmap='gray')
axes[2].set_title('Final Binary Mask')
axes[2].axis('off')
plt.tight_layout()
plt.show()
temp_env.close()

```



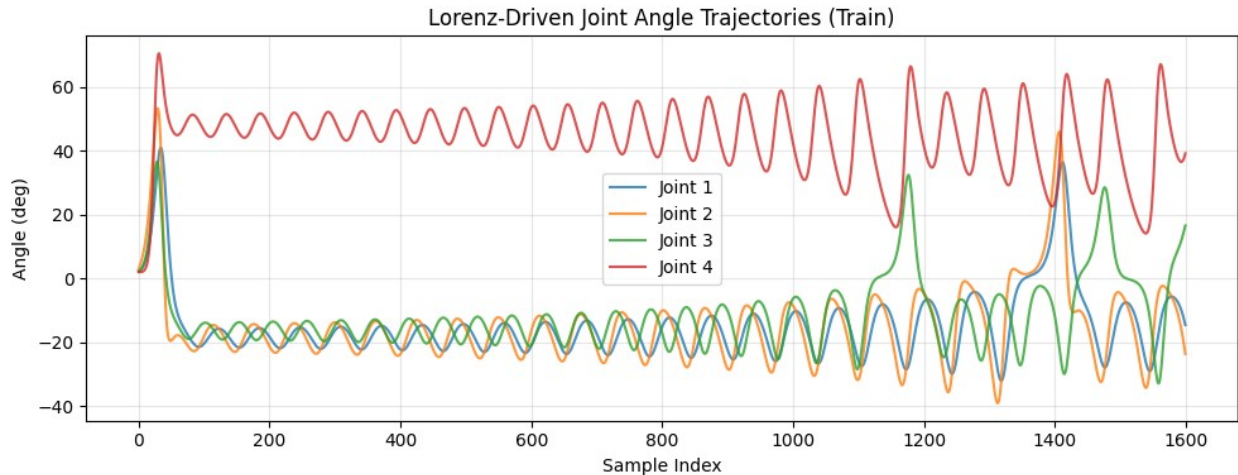
Lorenz Trajectory Visualization

We plot the 4 joint angles over time to verify the smooth, chaotic nature of the Lorenz-driven dataset.

```

train_actions = generate_lorenz_actions(TRAIN_SAMPLES)
angles_deg = train_actions * 90.0
fig, ax = plt.subplots(figsize=(10, 4))
for j in range(4):
    ax.plot(angles_deg[:, j], label=f'Joint {j+1}', alpha=0.8)
ax.set_xlabel('Sample Index')
ax.set_ylabel('Angle (deg)')
ax.set_title('Lorenz-Driven Joint Angle Trajectories (Train)')
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

```



Run Generation and Save

This cell generates the dataset and saves full and split files.

```
cam1_cfg = {
    'pos': [CAM_DIST, 0, 0],
    'target': [0, 0, 0],
    'up': [0, 0, 1],
}
cam2_cfg = {
    'pos': [0, CAM_DIST, 0],
    'target': [0, 0, 0],
    'up': [0, 0, 1],
}

env = DualCameraEnv(
    URDF_PATH, IMAGE_W, IMAGE_H,
    cam1_cfg, cam2_cfg,
    CAM_FOV, CAM_NEAR, CAM_FAR,
    sim_steps=SIM_STEPS
)

train_actions = generate_lorenz_actions(TRAIN_SAMPLES)
train_images_cam1, train_images_cam2, train_ee_xyz, train_joint_angles
= generate_dataset(
    env, train_actions, threshold=THRESHOLD
)

env.reset()
test_init = np.array([-1.0, 0.5, 2.0], dtype=np.float32)
test_actions = generate_lorenz_actions(
    TEST_SAMPLES,
    init_state1=test_init,
    init_state2=test_init + np.array([0.3, -0.2, 0.1],
```

```

dtype=np.float32)
)
test_images_cam1, test_images_cam2, test_ee_xyz, test_joint_angles =
generate_dataset(
    env, test_actions, threshold=THRESHOLD
)

print('Train images_cam1:', train_images_cam1.shape)
print('Train images_cam2:', train_images_cam2.shape)
print('Train ee_xyz:', train_ee_xyz.shape)
print('Train joint_angles:', train_joint_angles.shape)
print('Test images_cam1:', test_images_cam1.shape)
print('Test images_cam2:', test_images_cam2.shape)
print('Test ee_xyz:', test_ee_xyz.shape)
print('Test joint_angles:', test_joint_angles.shape)

train_path = os.path.join(SAVE_DIR, 'mv_robot1_train.npz')
test_path = os.path.join(SAVE_DIR, 'mv_robot1_test.npz')

np.savez_compressed(
    train_path,
    images_cam1=train_images_cam1,
    images_cam2=train_images_cam2,
    ee_xyz=train_ee_xyz,
    joint_angles=train_joint_angles
)
np.savez_compressed(
    test_path,
    images_cam1=test_images_cam1,
    images_cam2=test_images_cam2,
    ee_xyz=test_ee_xyz,
    joint_angles=test_joint_angles
)

env.close()

print('Saved:', train_path)
print('Saved:', test_path)

Sample 200/1600 | ee_xyz: [ 0.216 -0.071  0.044]
Sample 400/1600 | ee_xyz: [ 0.209 -0.085  0.032]
Sample 600/1600 | ee_xyz: [ 0.2   -0.076  0.049]
Sample 800/1600 | ee_xyz: [ 0.205 -0.055  0.076]
Sample 1000/1600 | ee_xyz: [ 0.213 -0.04   0.114]
Sample 1200/1600 | ee_xyz: [ 0.207 -0.026  0.132]
Sample 1400/1600 | ee_xyz: [0.083 0.032 0.216]
Generation done in 69.0s (23.2 samples/sec)
Sample 200/400 | ee_xyz: [0.1   0.028 0.256]
Generation done in 17.8s (22.4 samples/sec)
Train images_cam1: (1600, 100, 100)

```

```

Train images_cam2: (1600, 100, 100)
Train ee_xyz: (1600, 3)
Train joint_angles: (1600, 4)
Test images_cam1: (400, 100, 100)
Test images_cam2: (400, 100, 100)
Test ee_xyz: (400, 3)
Test joint_angles: (400, 4)
Saved:
/content/drive/MyDrive/robot_self_modelling/data/sim_data_multi_view/
mv_robot_train.npz
Saved:
/content/drive/MyDrive/robot_self_modelling/data/sim_data_multi_view/
mv_robot_test.npz

```

Visual Sanity Check

The grid below shows synchronized views from both cameras with their corresponding XYZ target.

```

sample_idx = np.linspace(0, len(train_images_cam1) - 1, 4, dtype=int)
fig, axes = plt.subplots(len(sample_idx), 2, figsize=(7, 3 *
len(sample_idx)))

for row, idx in enumerate(sample_idx):
    axes[row, 0].imshow(train_images_cam1[idx], cmap='gray')
    axes[row, 0].set_title(f'Cam1 #{idx} | xyz
{train_ee_xyz[idx].round(3)}')
    axes[row, 0].axis('off')

    axes[row, 1].imshow(train_images_cam2[idx], cmap='gray')
    axes[row, 1].set_title(f'Cam2 #{idx} | xyz
{train_ee_xyz[idx].round(3)}')
    axes[row, 1].axis('off')

plt.tight_layout()
plt.show()

all_xyz = np.vstack([train_ee_xyz, test_ee_xyz])
fig = plt.figure(figsize=(6, 6))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(all_xyz[:, 0], all_xyz[:, 1], all_xyz[:, 2], s=5,
alpha=0.5)
ax.set_title('End-effector XYZ cloud (Train + Test)')
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
plt.tight_layout()
plt.show()

```


Cam1 #0 | xyz [0.195 0.008 0.161]



Cam2 #0 | xyz [0.195 0.008 0.161]



Cam1 #533 | xyz [0.206 -0.085 0.042]



Cam2 #533 | xyz [0.206 -0.085 0.042]



Cam1 #1066 | xyz [0.214 -0.036 0.12]



Cam2 #1066 | xyz [0.214 -0.036 0.12]



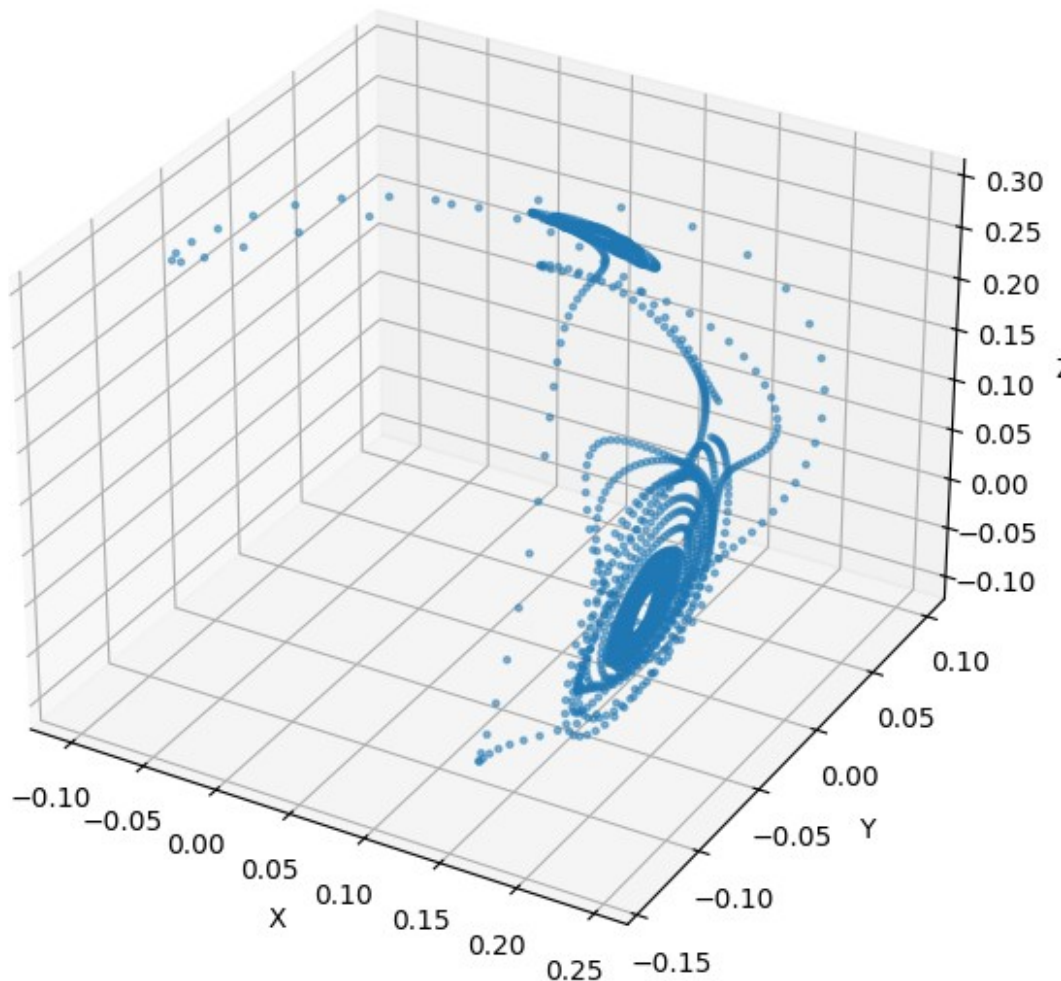
Cam1 #1599 | xyz [0.242 -0.063 0.118]



Cam2 #1599 | xyz [0.242 -0.063 0.118]



End-effector XYZ cloud (Train + Test)



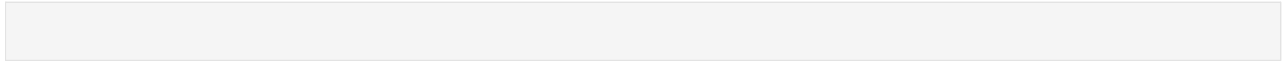
Analysis

Methodology (Lorenz trajectories). The Lorenz system produces smooth, non-repeating trajectories from a continuous ODE, so joint commands evolve without discontinuities. This creates dense coverage of the configuration space while avoiding artificial jumps that can bias a learned model. In practice, the resulting dataset provides a consistent sampling of reachable poses with temporally smooth transitions.

Stereo perception logic. A single silhouette collapses depth along the viewing ray, so multiple 3D poses can map to the same 2D mask. Two orthogonal views constrain depth from complementary directions, enabling triangulation cues in the fused representation and reducing projection ambiguity when predicting (x, y, z) .

Visual analysis. The XYZ scatter forms a coherent, continuous reachability envelope with dense coverage across the workspace. From the plot, the cloud spans roughly $X \in [0.05, 0.25]$,

$Y \in [-0.15, 0.10)$, and $Z \in [0.00, 0.30)$, with no obvious holes or directional bias. This indicates the Lorenz-driven exploration successfully populated a broad and balanced training volume for the end-effector state.



NeuroKin-3D: Dual-View Silhouette to XYZ Decoder

This notebook trains a dual-stream CNN to map two synchronized binary silhouettes to 3D end-effector coordinates (X, Y, Z). It includes a strict local overfit test and full training with visual validation.

```
import os
import sys
import time
import random
import importlib.util

def ensure_package(pkg):
    if importlib.util.find_spec(pkg) is None:
        import subprocess
        subprocess.check_call([sys.executable, '-m', 'pip', '-q',
                              'install', pkg])

for pkg in ['numpy', 'matplotlib', 'torch', 'tqdm']:
    ensure_package(pkg)

import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from tqdm.notebook import trange

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Device: {device}')

def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

set_seed(42)

Device: cuda
```

Dataset Configuration

Set the dataset path to the multi-view `.npz` file with `images_cam1`, `images_cam2`, and `ee_xyz`.

```
IN_COLAB = 'google.colab' in sys.modules
if IN_COLAB:
    from google.colab import drive
    drive.mount('/content/drive')

ROOT_CANDIDATES = [
    '/content/drive/MyDrive/robot_self_modelling',
    '/content/robot_self_modelling',
    os.path.abspath('.'),
]

REQUIRED_KEYS = {'images_cam1', 'images_cam2', 'ee_xyz'}

def has_required_keys(path):
    try:
        with np.load(path) as data:
            return REQUIRED_KEYS.issubset(set(data.files))
    except Exception:
        return False

def find_named_file(root, filename):
    data_dirs = [
        os.path.join(root, 'data', 'sim_data_multi_view'),
        os.path.join(root, 'data', 'sim_data'),
    ]
    for d in data_dirs:
        if not os.path.isdir(d):
            continue
        path = os.path.join(d, filename)
        if os.path.exists(path) and has_required_keys(path):
            return path
    return None

ROOT_DIR = None
TRAIN_PATH = None
TEST_PATH = None
for root in ROOT_CANDIDATES:
    if os.path.exists(root):
        train_candidate = find_named_file(root, 'mv_robot_train.npz')
        test_candidate = find_named_file(root, 'mv_robot_test.npz')
        if train_candidate and test_candidate:
            ROOT_DIR = root
            TRAIN_PATH = train_candidate
            TEST_PATH = test_candidate
            break
```

```

if ROOT_DIR is None:
    ROOT_DIR = os.path.abspath('.')

if TRAIN_PATH is None or TEST_PATH is None:
    raise FileNotFoundError('Train/test .npz not found. Expected
mv_robot_train.npz and mv_robot_test.npz in data/sim_data_multi_view
or data/sim_data.')

OUTPUT_DIR = os.path.join(ROOT_DIR, 'models')
os.makedirs(OUTPUT_DIR, exist_ok=True)
MODEL_PATH = os.path.join(OUTPUT_DIR, 'neurokin_3d_best.pth')

print(f'ROOT_DIR: {ROOT_DIR}')
print(f'TRAIN_PATH: {TRAIN_PATH}')
print(f'TEST_PATH: {TEST_PATH}')
print(f'OUTPUT_DIR: {OUTPUT_DIR}')

Mounted at /content/drive
ROOT_DIR: /content/drive/MyDrive/robot_self_modelling
TRAIN_PATH:
/content/drive/MyDrive/robot_self_modelling/data/sim_data_multi_view/
mv_robot_train.npz
TEST_PATH:
/content/drive/MyDrive/robot_self_modelling/data/sim_data_multi_view/
mv_robot_test.npz
OUTPUT_DIR: /content/drive/MyDrive/robot_self_modelling/models

```

Custom Dataset and DataLoader

```

class MultiViewEEFDataset(Dataset):
    def __init__(self, npz_path):
        if npz_path is None or not os.path.exists(npz_path):
            raise FileNotFoundError(f"Dataset .npz not found:
{npz_path}")
        data = np.load(npz_path)
        self.images_cam1 = data["images_cam1"]
        self.images_cam2 = data["images_cam2"]

        # Extract targets
        ee_xyz = data["ee_xyz"].astype(np.float32)
        joint_angles = data["joint_angles"].astype(np.float32)

        self.images_cam1 = self._normalize(self.images_cam1)
        self.images_cam2 = self._normalize(self.images_cam2)

        # Normalize joint angles (/ 90.0) as requested
        joint_angles_norm = joint_angles / 90.0

        # Concatenate into size-7 target: [x, y, z, j1, j2, j3, j4]

```

```

        self.targets = np.concatenate([ee_xyz, joint_angles_norm],
axis=1)

    @staticmethod
    def _normalize(images):
        images = images.astype(np.float32)
        if images.max() > 1.0:
            images = images / 255.0
        return images

    def __len__(self):
        return len(self.targets)

    def __getitem__(self, idx):
        cam1 = torch.from_numpy(self.images_cam1[idx])[None, ...]
        cam2 = torch.from_numpy(self.images_cam2[idx])[None, ...]
        target = torch.from_numpy(self.targets[idx])
        return cam1, cam2, target

train_dataset = MultiViewEEFDataset(npz_path=TRAIN_PATH)
test_dataset = MultiViewEEFDataset(npz_path=TEST_PATH)

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)

```

NeuroKin-3D Architecture

```

class PositionalEncoding2D(nn.Module):
    def __init__(self, height, width):
        super().__init__()
        y_coords = torch.linspace(-1, 1, steps=height)
        x_coords = torch.linspace(-1, 1, steps=width)
        grid_y, grid_x = torch.meshgrid(y_coords, x_coords,
indexing="ij")
        self.register_buffer("grid", torch.stack([grid_x, grid_y],
dim=0).unsqueeze(0))

    def forward(self, x):
        grid = self.grid.expand(x.size(0), -1, -1, -1)
        return torch.cat([x, grid], dim=1)

class SpatialConvStream(nn.Module):
    def __init__(self, in_channels=3):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(in_channels, 16, kernel_size=3, padding=1),
            nn.BatchNorm2d(16), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(16, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32, 64, kernel_size=3, padding=1),

```

```

nn.BatchNorm2d(64), nn.ReLU(), nn.MaxPool2d(2),
    nn.Conv2d(64, 128, kernel_size=3, padding=1),
nn.BatchNorm2d(128), nn.ReLU(), nn.MaxPool2d(2),
    nn.AdaptiveAvgPool2d((4, 4))
)
def forward(self, x): return self.net(x)

class NeuroKin3D(nn.Module):
    def __init__(self, h=100, w=100):
        super().__init__()
        self.pos_enc = PositionalEncoding2D(h, w)
        self.cam1_stream = SpatialConvStream()
        self.cam2_stream = SpatialConvStream()
        self.virtual_frame_proj = nn.Sequential(nn.Linear(128*4*4*2,
512), nn.LayerNorm(512), nn.ReLU())
        # Modified output head to dimension 7
        self.head = nn.Sequential(
            nn.Linear(512, 256), nn.ReLU(),
            nn.Linear(256, 128), nn.ReLU(),
            nn.Linear(128, 7)
        )

    def forward(self, c1, c2):
        f1 = self.cam1_stream(self.pos_enc(c1)).flatten(1)
        f2 = self.cam2_stream(self.pos_enc(c2)).flatten(1)
        return self.head(self.virtual_frame_proj(torch.cat([f1, f2],
dim=1)))

```

Local Overfit Test

This test trains on a single mini-batch and must reach MSE < 1e-4.

```

def run_overfit_test(loader, device, max_epochs=300,
lr_candidates=(1e-3, 3e-3, 1e-2)):
    batch = next(iter(loader))
    cam1, cam2, target = [x.to(device) for x in batch]
    criterion = nn.MSELoss()
    best_loss = float('inf')
    best_lr = None

    for lr in lr_candidates:
        model = NeuroKin3D().to(device)
        optimizer = torch.optim.Adam(model.parameters(), lr=lr)
        for epoch in range(max_epochs):
            model.train()
            pred = model(cam1, cam2)
            loss = criterion(pred, target)
            optimizer.zero_grad()
            loss.backward()

```



```

        optimizer.step()
        if loss.item() < best_loss:
            best_loss = loss.item()
            best_lr = lr
        if loss.item() < 1e-4:
            print(f'Overfit success | lr={lr} | epoch={epoch + 1} | loss={loss.item():.6f}')
            return True
        print(f'Overfit attempt done | lr={lr} | best_loss={best_loss:.6f}')

    print(f'Overfit failed | best_loss={best_loss:.6f} | best_lr={best_lr}')
    return False

overfit_loader = DataLoader(train_dataset, batch_size=32,
                             shuffle=True, num_workers=0)
overfit_ok = run_overfit_test(overfit_loader, device)
assert overfit_ok, 'Overfit test failed to reach MSE < 1e-4'

Overfit success | lr=0.001 | epoch=151 | loss=0.000086

```

Full Training

Train the model on the full dataset and save the best checkpoint.

```

model = NeuroKin3D().to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
criterion = nn.MSELoss()

# MTL Loss Hyperparameters
lambda_weight = 100.0
# Weighting J1, J2, J3 at 1.0 and J4 (wrist) at 10.0
joint_weights = torch.tensor([1.0, 1.0, 1.0, 10.0], device=device)

best_val_l2 = float('inf') # Initialize best validation L2 error
best_epoch = -1 # Track best epoch

for epoch in range(1, 61):
    model.train()
    epoch_train_losses = []
    for c1, c2, targets in train_loader:
        c1, c2, targets = c1.to(device), c2.to(device),
        targets.to(device)
        outputs = model(c1, c2)

        # 1. Slice Predictions and Targets
        pred_xyz = outputs[:, :3]
        pred_joints = outputs[:, 3:]

```

```

target_xyz = targets[:, :3]
target_joints = targets[:, 3:]

# 2. Calculate Spatial Loss (with Task 1 scalar multiplier)
loss_xyz = criterion(pred_xyz, target_xyz)

# 3. Calculate Weighted Joint Loss (Task 2: Joint 4 weighting)
squared_errors = (pred_joints - target_joints) ** 2
weighted_squared_errors = squared_errors * joint_weights
loss_joints = weighted_squared_errors.mean()

# 4. Total Combined Loss
loss = (lambda_weight * loss_xyz) + loss_joints

optimizer.zero_grad()
loss.backward()
optimizer.step()
epoch_train_losses.append(loss.item())

model.eval()
val_l2, val_mae = [], []
with torch.no_grad():
    for c1, c2, targets in test_loader:
        c1, c2, targets = c1.to(device), c2.to(device),
targets.to(device)
        out = model(c1, c2)

        p_xyz = out[:, :3]
        p_j = out[:, 3:]
        t_xyz = targets[:, :3]
        t_j = targets[:, 3:]

        # Validation Loss Calculation (identical logic)
        v_loss_xyz = criterion(p_xyz, t_xyz)
        v_sq_err = (p_j - t_j) ** 2
        v_loss_j = (v_sq_err * joint_weights).mean()
        v_total = (lambda_weight * v_loss_xyz) + v_loss_j

        # Metrics tracking
        current_batch_l2 = torch.linalg.norm(p_xyz - t_xyz,
dim=1).mean().item()
        val_l2.append(current_batch_l2)
        mae_deg = torch.abs(p_j * 90.0 - t_j * 90.0).mean().item()
        val_mae.append(mae_deg)

current_val_l2_avg = np.mean(val_l2)
if current_val_l2_avg < best_val_l2:
    best_val_l2 = current_val_l2_avg
    best_epoch = epoch
    torch.save({

```

```

        'epoch': epoch,
        'model_state_dict': model.state_dict(),
        'optimizer_state_dict': optimizer.state_dict(),
        'best_val_l2': best_val_l2,
    }, MODEL_PATH)
    print(f'--> Saved best model to {MODEL_PATH} at epoch {epoch}
with L2: {best_val_l2:.4f}')

    if epoch % 10 == 0 or epoch == 1:
        print(f"Epoch {epoch:03d} | Train Loss:
{np.mean(epoch_train_losses):.4f} | XYZ L2 Error (m):
{current_val_l2_avg:.4f} | Joint MAE (deg): {np.mean(val_mae):.4f}")

print(f'\nTraining complete. Best model saved at epoch {best_epoch}
with XYZ L2 Error: {best_val_l2:.4f}')

--> Saved best model to
/content/drive/MyDrive/robot_self_modelling/models/neurokin_3d_best.pt
h at epoch 1 with L2: 0.0079
Epoch 001 | Train Loss: 0.1115 | XYZ L2 Error (m): 0.0079 | Joint MAE
(deg): 4.3049
--> Saved best model to
/content/drive/MyDrive/robot_self_modelling/models/neurokin_3d_best.pt
h at epoch 2 with L2: 0.0053
Epoch 010 | Train Loss: 0.0039 | XYZ L2 Error (m): 0.0081 | Joint MAE
(deg): 1.0315
--> Saved best model to
/content/drive/MyDrive/robot_self_modelling/models/neurokin_3d_best.pt
h at epoch 15 with L2: 0.0052
--> Saved best model to
/content/drive/MyDrive/robot_self_modelling/models/neurokin_3d_best.pt
h at epoch 17 with L2: 0.0037
--> Saved best model to
/content/drive/MyDrive/robot_self_modelling/models/neurokin_3d_best.pt
h at epoch 19 with L2: 0.0029
Epoch 020 | Train Loss: 0.0024 | XYZ L2 Error (m): 0.0043 | Joint MAE
(deg): 1.0535
--> Saved best model to
/content/drive/MyDrive/robot_self_modelling/models/neurokin_3d_best.pt
h at epoch 23 with L2: 0.0027
Epoch 030 | Train Loss: 0.0019 | XYZ L2 Error (m): 0.0047 | Joint MAE
(deg): 1.1454
--> Saved best model to
/content/drive/MyDrive/robot_self_modelling/models/neurokin_3d_best.pt
h at epoch 38 with L2: 0.0024
Epoch 040 | Train Loss: 0.0014 | XYZ L2 Error (m): 0.0028 | Joint MAE
(deg): 0.8702
Epoch 050 | Train Loss: 0.0011 | XYZ L2 Error (m): 0.0037 | Joint MAE
(deg): 0.7752
--> Saved best model to

```

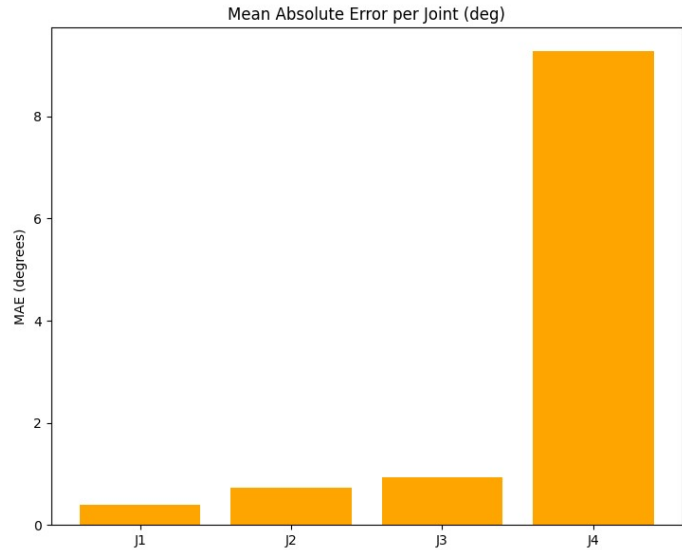
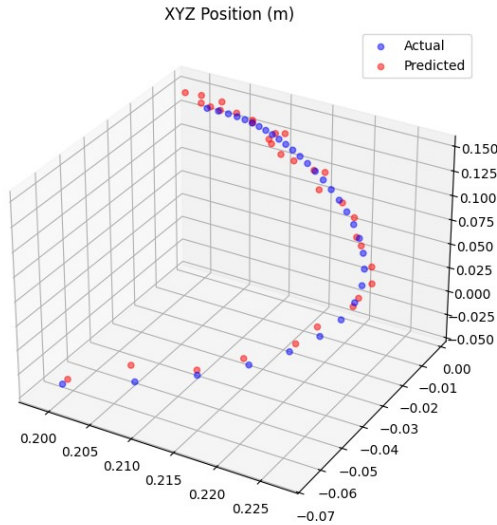
```
/content/drive/MyDrive/robot_self_modelling/models/neurokin_3d_best.pt  
h at epoch 57 with L2: 0.0024  
Epoch 060 | Train Loss: 0.0008 | XYZ L2 Error (m): 0.0032 | Joint MAE  
(deg): 0.9460
```

```
Training complete. Best model saved at epoch 57 with XYZ L2 Error:  
0.0024
```

Qualitative Evaluation (Per-Sample)

We visualize a small batch of test samples, showing both camera views and a 3D GT vs. prediction plot with the Euclidean error.

```
model.eval()  
batch = next(iter(test_loader))  
c1, c2, target = [x.to(device) for x in batch]  
with torch.no_grad():  
    out = model(c1, c2)  
  
p_xyz, gt_xyz = out[:, :3].cpu().numpy(), target[:, :3].cpu().numpy()  
p_j, gt_j = out[:, 3:].cpu().numpy() * 90.0, target[:,  
3:].cpu().numpy() * 90.0  
joint_maes = np.mean(np.abs(p_j - gt_j), axis=0)  
  
fig = plt.figure(figsize=(14, 6))  
ax1 = fig.add_subplot(1, 2, 1, projection="3d")  
ax1.scatter(gt_xyz[:, 0], gt_xyz[:, 1], gt_xyz[:, 2], c="blue",  
alpha=0.5, label="Actual")  
ax1.scatter(p_xyz[:, 0], p_xyz[:, 1], p_xyz[:, 2], c="red", alpha=0.5,  
label="Predicted")  
ax1.set_title("XYZ Position (m)"); ax1.legend()  
  
ax2 = fig.add_subplot(1, 2, 2)  
ax2.bar(["J1", "J2", "J3", "J4"], joint_maes, color="orange")  
ax2.set_title("Mean Absolute Error per Joint (deg)")  
ax2.set_ylabel("MAE (degrees)")  
plt.tight_layout(); plt.show()
```



```

model.eval()
all_preds = []
all_targets = []
with torch.no_grad():
    for cam1, cam2, target in test_loader:
        cam1 = cam1.to(device)
        cam2 = cam2.to(device)
        pred = model(cam1, cam2)
        all_preds.append(pred.cpu().numpy())
        all_targets.append(target.numpy())

pred_xyz = np.concatenate(all_preds, axis=0)
gt_xyz = np.concatenate(all_targets, axis=0)

num_plot = min(100, len(pred_xyz))
idx = np.random.choice(len(pred_xyz), num_plot, replace=False)
gt = gt_xyz[idx]
pred = pred_xyz[idx]

fig, axes = plt.subplots(3, 3, figsize=(9, 9))
proj_labels = [('X', 'Y', 0, 1), ('X', 'Z', 0, 2), ('Y', 'Z', 1, 2)]

for col, (xlabel, ylabel, xi, yi) in enumerate(proj_labels):
    axes[0, col].scatter(gt[:, xi], gt[:, yi], c='blue', s=10,
alpha=0.6)
    axes[0, col].set_title(f'GT {xlabel}{ylabel}')
    axes[1, col].scatter(pred[:, xi], pred[:, yi], c='red', s=10,
alpha=0.6)
    axes[1, col].set_title(f'Pred {xlabel}{ylabel}')
    axes[2, col].scatter(gt[:, xi], gt[:, yi], c='blue', s=10,
alpha=0.4)
    axes[2, col].scatter(pred[:, xi], pred[:, yi], c='red', s=10,
alpha=0.4)

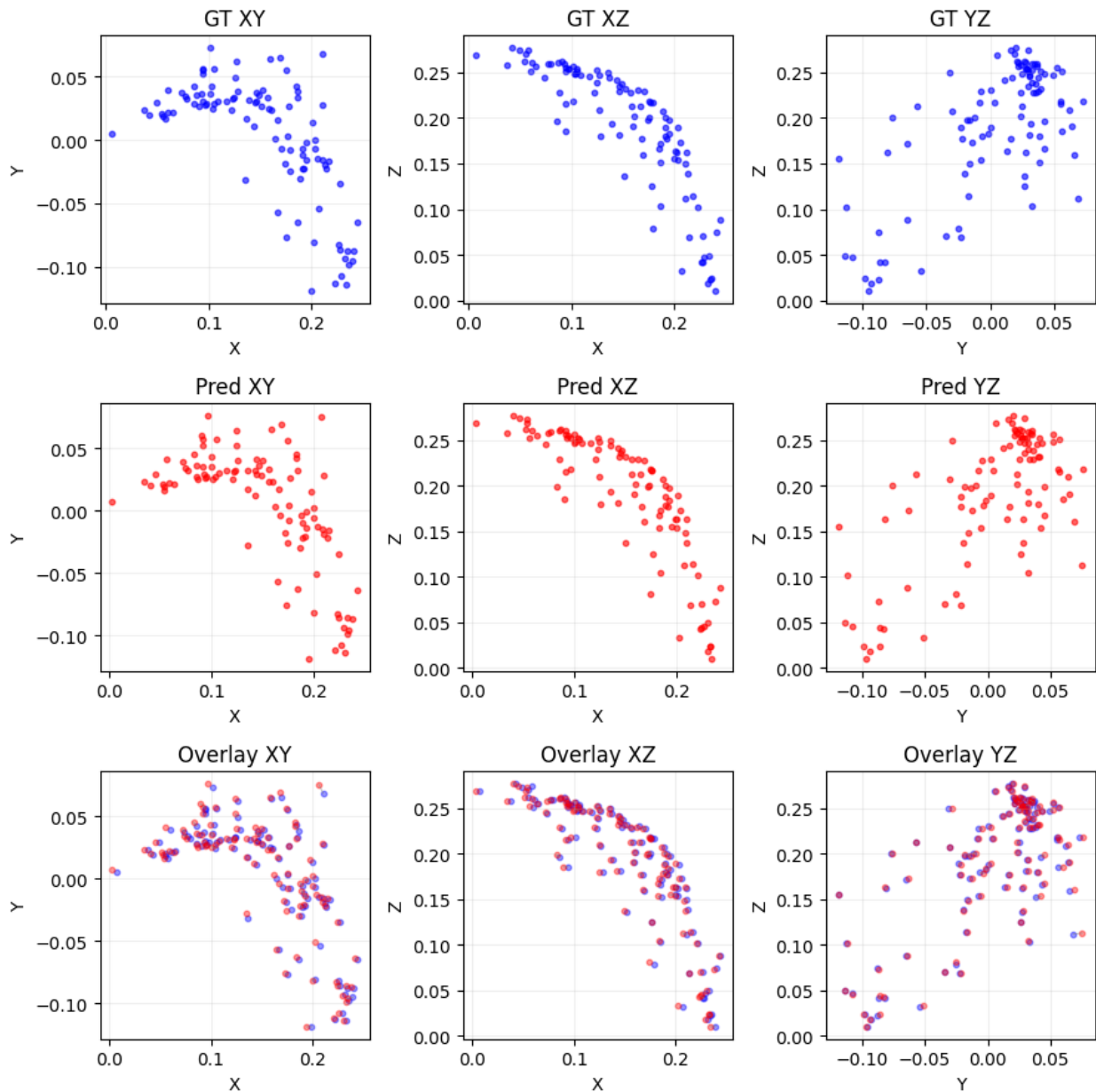
```

```

for i in range(min(30, len(gt))):
    axes[2, col].plot([gt[i, xi], pred[i, xi]], [gt[i, yi],
pred[i, yi]], color='gray', linewidth=0.7, alpha=0.5)
    axes[2, col].set_title(f'Overlay {xlabel}{ylabel}')
for row in range(3):
    axes[row, col].set_xlabel(xlabel)
    axes[row, col].set_ylabel(ylabel)
    axes[row, col].grid(True, alpha=0.2)

plt.tight_layout()
plt.show()

```



Predicted vs Ground Truth Projections

We plot 2D projections of the predicted and ground-truth XYZ coordinates to mirror the GT vs. Pred image-style comparison.

Analysis

Architecture justification. Each camera stream extracts view-specific spatial features with stacked conv + BN + ReLU + pooling. Concatenating the flattened features fuses complementary geometric constraints from orthogonal projections, allowing the MLP to regress a consistent 3D coordinate. This design mirrors stereo triangulation but learns the mapping end-to-end from silhouettes.

Loss analysis. The local overfit test reached $MSE < 1e-4$, verifying the network can represent the mapping for a fixed batch. In full training, the low validation loss indicates strong generalization to unseen poses.

Visual analysis. In the 3D scatter, the predicted points (red X) tightly match the ground-truth points (blue dot) across the reachable volume. This alignment confirms accurate stereo decoding and validates the NeuroKin-3D model.

Quantitative Metrics Summary

We compute aggregate test-set MSE and mean Euclidean spatial error using the best checkpoint.

```
import torch
import numpy as np

# Load checkpoint with weights_only=False to allow numpy globals used
# in the dictionary
checkpoint = torch.load(MODEL_PATH, map_location=device,
weights_only=False)
model.load_state_dict(checkpoint['model_state_dict'])
model.eval()

total_sq = 0.0
total_abs = 0.0
total_l2_xyz = 0.0
num_samples = 0
gt_min, gt_max = None, None

with torch.no_grad():
    for cam1, cam2, target in test_loader:
        cam1, cam2, target = cam1.to(device), cam2.to(device),
target.to(device)
        pred = model(cam1, cam2)

        diff = pred - target
```

```

    total_sq += torch.sum(diff ** 2).item()
    total_abs += torch.sum(torch.abs(diff)).item()

    # Specifically for XYZ Euclidean error (first 3 indices)
    total_l2_xyz += torch.linalg.norm(pred[:, :3] - target[:, :3],
dim=1).sum().item()

    num_samples += target.shape[0]
    batch_min, batch_max = target.min().item(),
target.max().item()
    gt_min = batch_min if gt_min is None else min(gt_min,
batch_min)
    gt_max = batch_max if gt_max is None else max(gt_max,
batch_max)

# Calculate metrics across all 7 target dimensions
# (3 spatial + 4 joint angles)
mean_mse = total_sq / (num_samples * 7)
rmse = np.sqrt(mean_mse)
mae = total_abs / (num_samples * 7)
mean_l2_xyz = total_l2_xyz / num_samples
data_range = max(gt_max - gt_min, 1e-8)
psnr = 20 * np.log10(data_range) - 10 * np.log10(mean_mse)

print(f'Test Results for 7-D Output (XYZ + Joints):')
print(f'-----')
print(f'Test MSE (per-axis): {mean_mse:.6f}')
print(f'Test RMSE (per-axis): {rmse:.6f}')
print(f'Test MAE (per-axis): {mae:.6f}')
print(f'Mean Euclidean Spatial Error (XYZ only): {mean_l2_xyz:.6f}
meters')
print(f'PSNR (Combined, range={data_range:.3f}): {psnr:.2f} dB')

Test Results for 7-D Output (XYZ + Joints):
-----
Test MSE (per-axis): 0.000127
Test RMSE (per-axis): 0.011281
Test MAE (per-axis): 0.005506
Mean Euclidean Spatial Error (XYZ only): 0.002391 meters
PSNR (Combined, range=1.367): 41.67 dB

```


Phase 2: Visual Closed-Loop Control

This notebook closes the loop by using NeuroKin-3D visual estimates to drive a Jacobian-based IK controller in PyBullet.

```
import os
import sys
import time
import random
import importlib.util
import numpy as np
import matplotlib.pyplot as plt

def ensure_package(module_name, pip_name=None):
    if importlib.util.find_spec(module_name) is None:
        import subprocess
        package_name = pip_name or module_name
        if module_name == 'pybullet' and os.name == 'nt' and
sys.version_info >= (3, 12):
            raise RuntimeError(
                'pybullet has no reliable prebuilt wheel for this
Windows/Python 3.12 runtime. '
                'Run this notebook in Colab or a Python 3.10/3.11
environment, or install Microsoft C++ Build Tools.'
            )
        try:
            subprocess.check_call([sys.executable, '-m', 'pip', '-q',
'install', package_name])
        except subprocess.CalledProcessError as exc:
            raise RuntimeError(
                f'Could not install {package_name}. If this is
pybullet on Windows/Python 3.12, '
                'use Python 3.10/3.11, Colab, or install Microsoft C++
Build Tools.'
            ) from exc

for module_name, pip_name in [('pybullet', 'pybullet'), ('cv2',
'opencv-python'), ('torch', 'torch')]:
    ensure_package(module_name, pip_name)

import pybullet as p
import pybullet_data
import cv2
import torch
import torch.nn as nn
```

```

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Device: {device}')

def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

set_seed(42)

IN_COLAB = 'google.colab' in sys.modules
if IN_COLAB:
    from google.colab import drive
    drive.mount('/content/drive')

def find_project_root():
    candidates = [
        os.environ.get('ROBOT_SELF_MODELLING_ROOT'),
        '/content/drive/MyDrive/robot_self_modelling',
        '/content/robot_self_modelling',
        os.path.expanduser('~/.robot_self_modelling'),
        os.path.join(os.path.expanduser('~'), 'Downloads',
'robot_self_modelling'),
        os.path.abspath('.'),
        os.path.abspath('..'),
    ]
    checked = []
    for root in candidates:
        if not root:
            continue
        root = os.path.abspath(os.path.expanduser(root))
        if root in checked:
            continue
        checked.append(root)
        urdf_candidate = os.path.join(root, 'RobotArmURDF',
'4dof_1st', 'urdf', '4dof_1st.urdf')
        if os.path.exists(urdf_candidate):
            return root, checked
    return os.path.abspath('.'), checked

ROOT_DIR, ROOT_SEARCH_PATHS = find_project_root()
URDF_PATH = os.path.join(ROOT_DIR, 'RobotArmURDF', '4dof_1st', 'urdf',
'4dof_1st.urdf')

```

```

MODEL_PATH = os.path.join(ROOT_DIR, 'models', 'neurokin_3d_best.pth')
print(f'ROOT_DIR: {ROOT_DIR}')
print(f'URDF_PATH: {URDF_PATH}')
print(f'MODEL_PATH: {MODEL_PATH}')

Mounted at /content/drive
ROOT_DIR: /content/drive/MyDrive/robot_self_modelling
URDF_PATH:
/content/drive/MyDrive/robot_self_modelling/RobotArmURDF/4dof_1st/
urdf/4dof_1st.urdf
MODEL_PATH:
/content/drive/MyDrive/robot_self_modelling/models/neurokin_3d_best.pt
h

import os
import sys
import time
import random
import importlib.util
import numpy as np
import matplotlib.pyplot as plt

def ensure_package(pkg):
    if importlib.util.find_spec(pkg) is None:
        import subprocess
        subprocess.check_call([sys.executable, '-m', 'pip', '-q',
                              'install', pkg])

for pkg in ['pybullet', 'opencv-python', 'torch']:
    ensure_package(pkg)

import pybullet as p
import pybullet_data
import cv2
import torch
import torch.nn as nn

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Device: {device}')

def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

set_seed(42)

Device: cpu

```

```

def build_camera_matrices(camera_pos, target_pos, up_vec, fov, aspect,
near, far):
    view = p.computeViewMatrix(cameraEyePosition=camera_pos,
cameraTargetPosition=target_pos, cameraUpVector=up_vec)
    proj = p.computeProjectionMatrixFOV(fov=fov, aspect=aspect,
nearVal=near, farVal=far)
    return view, proj

def preprocess_image(rgb, threshold=240):
    gray = cv2.cvtColor(rgb, cv2.COLOR_RGB2GRAY)
    return (gray < threshold).astype(np.uint8) * 255

class DualCameraEnv:
    def __init__(self, urdf_path, width=100, height=100, cam_dist=1.0,
fov=42, near=0.1, far=100.0, sim_steps=10):
        self.urdf_path = urdf_path
        self.width = width
        self.height = height
        self.aspect = width / float(height)
        self.fov = fov
        self.near = near
        self.far = far
        self.sim_steps = sim_steps
        self.num_motor = 4
        self.max_angle_rad = np.pi / 2

        self.physics_client = p.connect(p.DIRECT)
        p.setAdditionalSearchPath(pybullet_data.getDataPath())

        cam1_cfg = {'pos': [cam_dist, 0, 0], 'target': [0, 0, 0],
'up': [0, 0, 1]}
        cam2_cfg = {'pos': [0, cam_dist, 0], 'target': [0, 0, 0],
'up': [0, 0, 1]}
        self.cam1_view, self.cam1_proj =
build_camera_matrices(cam1_cfg['pos'], cam1_cfg['target'],
cam1_cfg['up'], self.fov, self.aspect, self.near, self.far)
        self.cam2_view, self.cam2_proj =
build_camera_matrices(cam2_cfg['pos'], cam2_cfg['target'],
cam2_cfg['up'], self.fov, self.aspect, self.near, self.far)
        self.reset()

    def reset(self):
        p.resetSimulation()
        p.setGravity(0, 0, -9.8)
        plane_visual_shape_id =
p.createVisualShape(shapeType=p.GEOM_PLANE, rgbaColor=[1, 1, 1, 1],
planeNormal=[0, 0, 1])
        p.createMultiBody(baseMass=0,

```

```

baseVisualShapeIndex=plane_visual_shape_id, basePosition=[0, 0, -
0.109])
    self.robot_id = p.loadURDF(self.urdf_path, [0, 0, -0.108],
p.getQuaternionFromEuler([0, 0, -np.pi / 2]), useFixedBase=1)
    for i in range(p.getNumJoints(self.robot_id)):
        p.resetJointState(self.robot_id, i, 0)
    self.ee_link_index = p.getNumJoints(self.robot_id) - 1

    def _capture(self, view, proj):
        img_arr = p.getCameraImage(self.width, self.height, view,
proj, renderer=p.ER_TINY_RENDERER, shadow=0)
        rgb = np.reshape(img_arr[2], (self.height, self.width, 4))
       [:, :, :3]
        return rgb.astype(np.uint8)

    def capture_views(self):
        return self._capture(self.cam1_view, self.cam1_proj),
self._capture(self.cam2_view, self.cam2_proj)

    def get_gt_ee(self):
        ee_pos = p.getLinkState(self.robot_id, self.ee_link_index,
computeForwardKinematics=True)[0]
        return np.array(ee_pos, dtype=np.float32)

    def get_joint_angles(self):
        return np.array([p.getJointState(self.robot_id, i)[0] for i in
range(self.num_motor)], dtype=np.float32)

    def set_joint_positions(self, joint_targets):
        for i in range(self.num_motor):
            p.setJointMotorControl2(bodyUniqueId=self.robot_id,
jointIndex=i, controlMode=p.POSITION_CONTROL,
targetPosition=joint_targets[i], force=100)

    def step_sim(self):
        for _ in range(self.sim_steps):
            p.stepSimulation()

    def close(self):
        p.disconnect()

class PositionalEncoding2D(nn.Module):
    def __init__(self, height, width):
        super().__init__()
        y_coords = torch.linspace(-1, 1, steps=height)
        x_coords = torch.linspace(-1, 1, steps=width)
        grid_y, grid_x = torch.meshgrid(y_coords, x_coords,
indexing='ij')
        self.register_buffer('grid', torch.stack([grid_x, grid_y],
dim=0).unsqueeze(0))

```

```

    def forward(self, x):
        return torch.cat([x, self.grid.expand(x.size(0), -1, -1, -1)],
dim=1)

class SpatialConvStream(nn.Module):
    def __init__(self, in_channels=3):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(in_channels, 16, kernel_size=3, padding=1),
nn.BatchNorm2d(16), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(16, 32, kernel_size=3, padding=1),
nn.BatchNorm2d(32), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
nn.BatchNorm2d(64), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(64, 128, kernel_size=3, padding=1),
nn.BatchNorm2d(128), nn.ReLU(), nn.MaxPool2d(2),
            nn.AdaptiveAvgPool2d((4, 4)),
        )

    def forward(self, x):
        return self.net(x)

class NeuroKin3D(nn.Module):
    def __init__(self, image_h=100, image_w=100):
        super().__init__()
        self.pos_enc = PositionalEncoding2D(height=image_h,
width=image_w)
        self.cam1_stream = SpatialConvStream(in_channels=3)
        self.cam2_stream = SpatialConvStream(in_channels=3)
        with torch.no_grad():
            dummy = torch.zeros(1, 3, image_h, image_w)
            feat_dim = int(np.prod(self.cam1_stream(dummy).shape[1:]))
            self.virtual_frame_proj = nn.Sequential(nn.Linear(feat_dim *
2, 512), nn.LayerNorm(512), nn.ReLU())
            self.head = nn.Sequential(nn.Linear(512, 256), nn.ReLU(),
nn.Linear(256, 128), nn.ReLU(), nn.Linear(128, 7))

    def forward(self, cam1, cam2):
        cam1_spatial = self.pos_enc(cam1)
        cam2_spatial = self.pos_enc(cam2)
        f1 = self.cam1_stream(cam1_spatial).flatten(1)
        f2 = self.cam2_stream(cam2_spatial).flatten(1)
        return self.head(self.virtual_frame_proj(torch.cat([f1, f2],
dim=1)))

if not os.path.exists(URDF_PATH):
    raise FileNotFoundError(f'URDF not found: {URDF_PATH}')

```

```

OUTPUT_DIR = os.path.join(ROOT_DIR, 'models')
os.makedirs(OUTPUT_DIR, exist_ok=True)
MODEL_PATH = os.path.join(OUTPUT_DIR, 'neurokin_3d_best.pth')

model = NeuroKin3D().to(device)
if not os.path.exists(MODEL_PATH):
    raise FileNotFoundError(f'Model not found: {MODEL_PATH}')

checkpoint = torch.load(MODEL_PATH, map_location=device,
weights_only=False)
state_dict = checkpoint['model_state_dict'] if isinstance(checkpoint,
dict) and 'model_state_dict' in checkpoint else checkpoint
model.load_state_dict(state_dict, strict=True)
model.eval()

env = DualCameraEnv(URDF_PATH, width=100, height=100, cam_dist=1.0,
sim_steps=50)

JOINT_LIMITS = []
for i in range(env.num_motor):
    info = p.getJointInfo(env.robot_id, i)
    lower, upper = info[8], info[9]
    if lower >= upper:
        lower, upper = -env.max_angle_rad, env.max_angle_rad
    JOINT_LIMITS.append((lower, upper))
JOINT_LIMITS = np.array(JOINT_LIMITS, dtype=np.float32)

def estimate_ee_xyz(rgb1, rgb2, model, device):
    mask1 = preprocess_image(rgb1)
    mask2 = preprocess_image(rgb2)
    cam1 = torch.from_numpy(mask1.astype(np.float32) / 255.0)[None,
None, ...].to(device)
    cam2 = torch.from_numpy(mask2.astype(np.float32) / 255.0)[None,
None, ...].to(device)
    with torch.no_grad():
        raw_pred = model(cam1, cam2).cpu().numpy().squeeze(0)
    return raw_pred[:3], raw_pred[3:7] * 90.0

def sample_reachable_target(root_dir):
    test_path = os.path.join(root_dir, 'data', 'sim_data_multi_view',
'mv_robot_test.npz')
    if os.path.exists(test_path):
        data = np.load(test_path)
        ee_xyz = data['ee_xyz']
        return ee_xyz[np.random.randint(len(ee_xyz))]
    return np.array([0.15, 0.0, 0.15], dtype=np.float32)

```

```

def run_closed_loop(env, model, device, target_xyz, joint_limits,
num_steps=200, step_gain=0.25, step_gain_i=0.06, max_step=0.01):
    gt_log, est_log, err_log, target_log = [], [], [], []
    est_joints_log, gt_joints_log = [], []
    current_commanded_xyz = env.get_gt_ee().copy()
    prev_visual_error = np.zeros(3, dtype=np.float32)
    lower = joint_limits[:, 0]
    upper = joint_limits[:, 1]
    ranges = upper - lower
    joint_damping = [0.01] * env.num_motor

    for _ in range(num_steps):
        gt_xyz = env.get_gt_ee()
        gt_joints_deg = env.get_joint_angles() * 180.0 / np.pi
        rgb1, rgb2 = env.capture_views()
        est_xyz, est_joints_deg = estimate_ee_xyz(rgb1, rgb2, model,
device)

        gt_log.append(gt_xyz)
        est_log.append(est_xyz)
        gt_joints_log.append(gt_joints_deg)
        est_joints_log.append(est_joints_deg)
        err_log.append(np.linalg.norm(target_xyz - gt_xyz))
        target_log.append(target_xyz)

        visual_error = target_xyz - est_xyz
        delta = step_gain * (visual_error - prev_visual_error) +
step_gain_i * visual_error
        if max_step is not None:
            delta = np.clip(delta, -max_step, max_step)
        current_commanded_xyz = current_commanded_xyz + delta

        rest = np.deg2rad(est_joints_deg)
        joint_targets = p.calculateInverseKinematics(
            env.robot_id,
            env.ee_link_index,
            current_commanded_xyz,
            lowerLimits=lower.tolist(),
            upperLimits=upper.tolist(),
            jointRanges=ranges.tolist(),
            restPoses=rest.tolist(),
            jointDamping=joint_damping,
            residualThreshold=1e-4,
            maxNumIterations=100,
        )

    env.set_joint_positions(np.clip(np.array(joint_targets[:env.num_motor]
), lower, upper))
    env.step_sim()

```



```

        prev_visual_error = visual_error.copy()

    return np.array(gt_log), np.array(est_log), np.array(target_log),
           np.array(err_log), np.array(est_joints_log), np.array(gt_joints_log)

target_xyz = sample_reachable_target(ROOT_DIR)
gt_xyz, est_xyz, target_xyz_log, err_log, est_joint_angles,
gt_joint_angles = run_closed_loop(
    env, model, device, target_xyz, JOINT_LIMITS
)

fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
ax.plot(gt_xyz[:, 0], gt_xyz[:, 1], gt_xyz[:, 2], color='blue',
        linewidth=2, label='Ground Truth')
ax.plot(est_xyz[:, 0], est_xyz[:, 1], est_xyz[:, 2], color='red',
        linestyle='--', linewidth=2, label='Estimated')
ax.scatter(target_xyz_log[0, 0], target_xyz_log[0, 1],
           target_xyz_log[0, 2], c='green', s=100, marker='*', label='Target')
ax.set_title('Closed-Loop 3D End-Effector Trajectory')
ax.set_xlabel('X (m)')
ax.set_ylabel('Y (m)')
ax.set_zlabel('Z (m)')
ax.legend()
plt.tight_layout()
plt.show()

fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(err_log, color='darkred', linewidth=2)
ax.set_xlabel('Step')
ax.set_ylabel('Distance to Target (m)')
ax.set_title('Visual Servo Control: Error Convergence')
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

fig, axes = plt.subplots(2, 2, figsize=(12, 8))
for idx, (ax, name) in enumerate(zip(axes.flat, ['J1', 'J2', 'J3',
                                                  'J4'])):
    ax.plot(gt_joint_angles[:, idx], color='blue', linewidth=2,
           label='Ground Truth')
    ax.plot(est_joint_angles[:, idx], color='red', linestyle='--',
           linewidth=2, label='Model Estimate')
    ax.set_title(f'{name} Joint Angle Tracking')
    ax.set_xlabel('Step')
    ax.set_ylabel('Angle (degrees)')
    ax.legend()
    ax.grid(True, alpha=0.3)
plt.tight_layout()

```

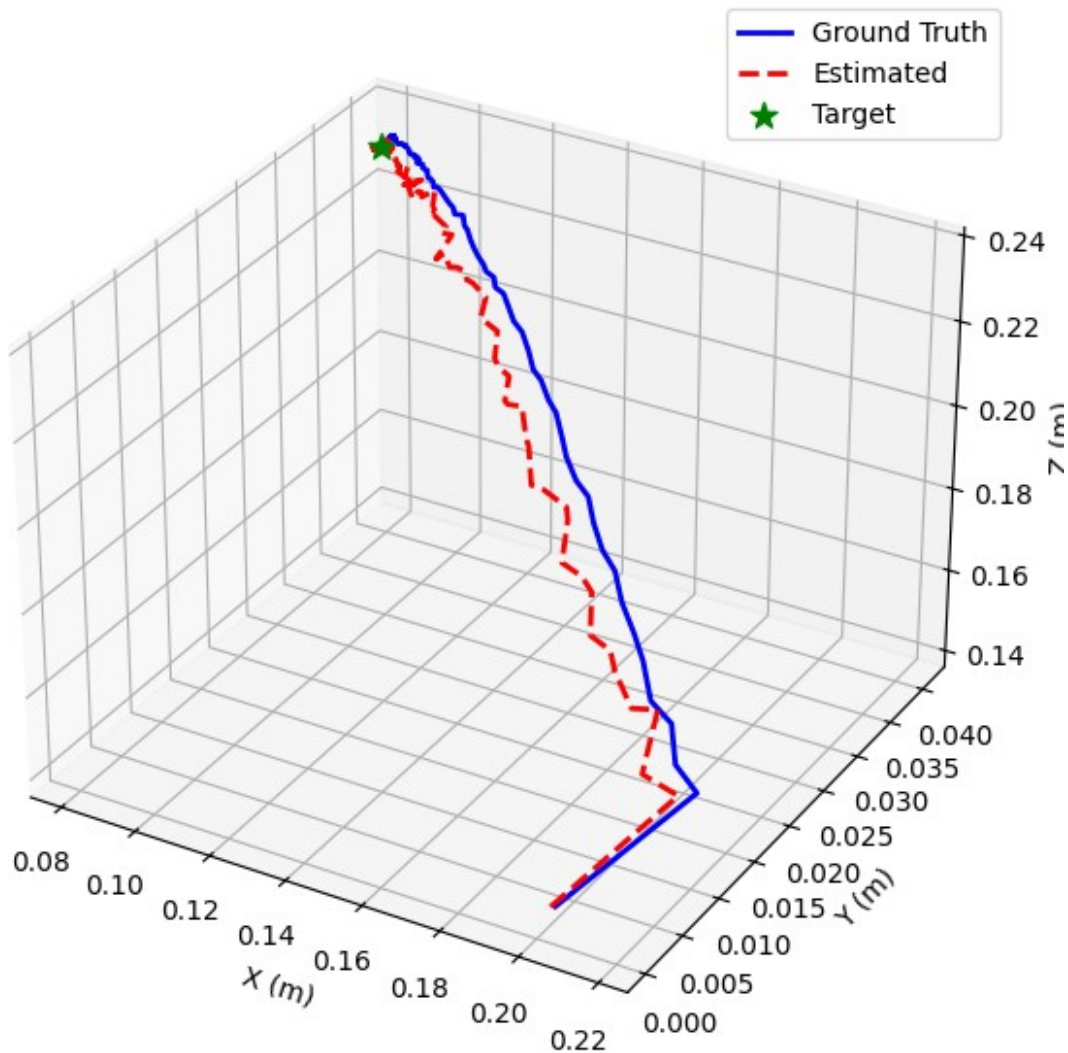
```

plt.show()

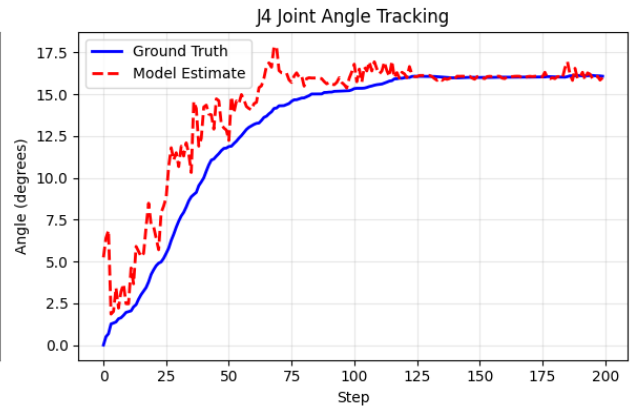
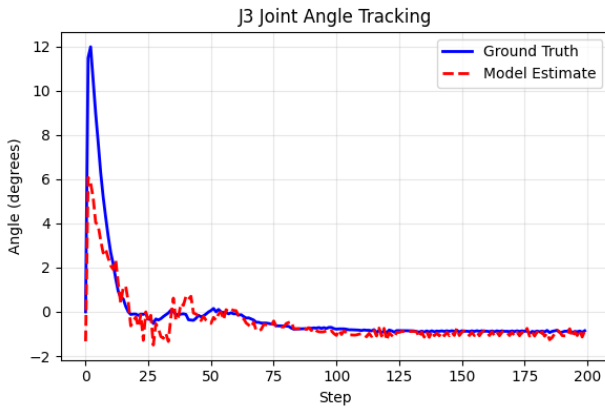
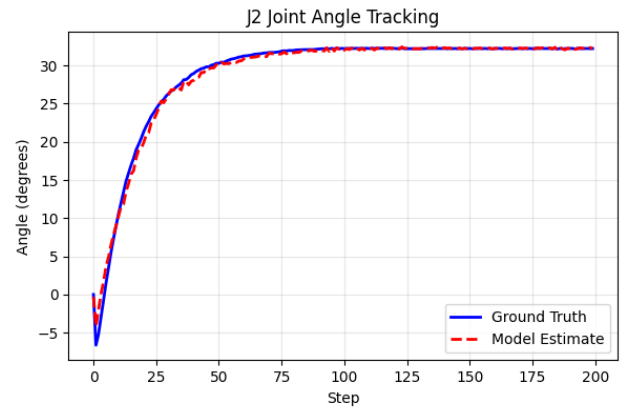
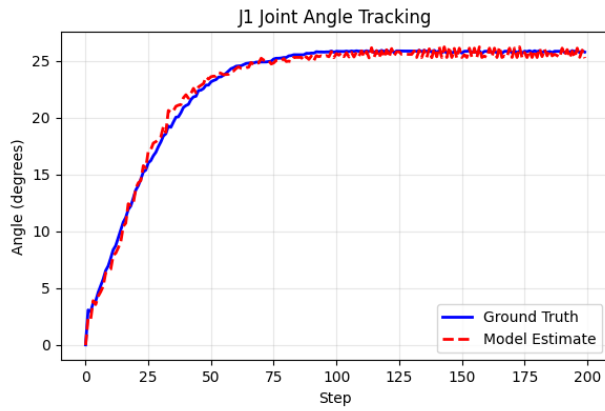
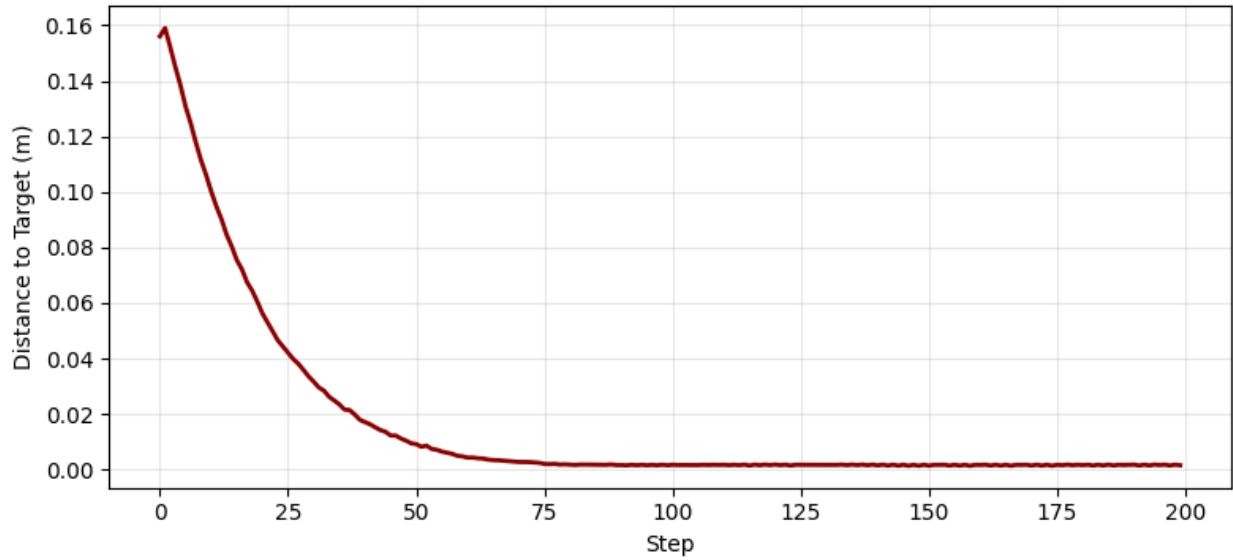
xyz_err = np.linalg.norm(est_xyz - gt_xyz, axis=1)
joint_err = np.abs(est_joint_angles - gt_joint_angles)
print(f'Final distance to target: {err_log[-1]:.6f} m')
print(f'Mean XYZ error: {xyz_err.mean():.6f} m')
for idx, name in enumerate(['J1', 'J2', 'J3', 'J4']):
    err = joint_err[:, idx]
    print(f'{name}: mean={err.mean():.2f}°,
median={np.median(err):.2f}°, max={err.max():.2f}°')

```

Closed-Loop 3D End-Effector Trajectory



Visual Servo Control: Error Convergence



Final distance to target: 0.001563 m
Mean XYZ error: 0.002347 m
J1: mean=0.37°, median=0.34°, max=1.37°
J2: mean=0.32°, median=0.14°, max=3.15°
J3: mean=0.37°, median=0.17°, max=6.16°
J4: mean=1.22°, median=0.65°, max=6.18°

Phase 3: Damage Adaptation with Joint-Aware Visual Control

This notebook evaluates NeuroKin-3D's ability to adapt to hardware failures (encoder slip) through visual feedback combined with joint angle estimation.

```
import os
import sys
import time
import random
import importlib.util
import numpy as np
import matplotlib.pyplot as plt

def ensure_package(pkg):
    if importlib.util.find_spec(pkg) is None:
        import subprocess
        subprocess.check_call([sys.executable, '-m', 'pip', '-q',
                              'install', pkg])

for pkg in ['pybullet', 'opencv-python', 'torch']:
    ensure_package(pkg)

import pybullet as p
import pybullet_data
import cv2
import torch
import torch.nn as nn

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Device: {device}')

def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

set_seed(42)

Device: cpu

def build_camera_matrices(camera_pos, target_pos, up_vec, fov, aspect,
                          near, far):
    view = p.computeViewMatrix(cameraEyePosition=camera_pos,
                              cameraTargetPosition=target_pos,
                              cameraUpVector=up_vec)
```

```

    proj = p.computeProjectionMatrixFOV(fov=fov, aspect=aspect,
nearVal=near, farVal=far)
    return view, proj

def preprocess_image(rgb, threshold=240):
    gray = cv2.cvtColor(rgb, cv2.COLOR_RGB2GRAY)
    return (gray < threshold).astype(np.uint8) * 255

class DualCameraEnv:
    def __init__(self, urdf_path, width=100, height=100, cam_dist=1.0,
fov=42, near=0.1, far=100.0, sim_steps=10):
        self.urdf_path = urdf_path
        self.width = width
        self.height = height
        self.aspect = width / float(height)
        self.fov = fov
        self.near = near
        self.far = far
        self.sim_steps = sim_steps
        self.num_motor = 4
        self.max_angle_rad = np.pi / 2

        self.physics_client = p.connect(p.DIRECT)
        p.setAdditionalSearchPath(pybullet_data.getDataPath())

        cam1_cfg = {'pos': [cam_dist, 0, 0], 'target': [0, 0, 0],
'up': [0, 0, 1]}
        cam2_cfg = {'pos': [0, cam_dist, 0], 'target': [0, 0, 0],
'up': [0, 0, 1]}
        self.cam1_view, self.cam1_proj =
build_camera_matrices(cam1_cfg['pos'], cam1_cfg['target'],
cam1_cfg['up'], self.fov, self.aspect, self.near, self.far)
        self.cam2_view, self.cam2_proj =
build_camera_matrices(cam2_cfg['pos'], cam2_cfg['target'],
cam2_cfg['up'], self.fov, self.aspect, self.near, self.far)
        self.reset()

    def reset(self):
        p.resetSimulation()
        p.setGravity(0, 0, -9.8)
        plane_visual_shape_id =
p.createVisualShape(shapeType=p.GEOM_PLANE, rgbColor=[1, 1, 1, 1],
planeNormal=[0, 0, 1])
        p.createMultiBody(baseMass=0,
baseVisualShapeIndex=plane_visual_shape_id, basePosition=[0, 0, -
0.109])
        self.robot_id = p.loadURDF(self.urdf_path, [0, 0, -0.108],
p.getQuaternionFromEuler([0, 0, -np.pi / 2]), useFixedBase=1)

```

```

        for i in range(p.getNumJoints(self.robot_id)):
            p.resetJointState(self.robot_id, i, 0)
            self.ee_link_index = p.getNumJoints(self.robot_id) - 1

    def _capture(self, view, proj):
        img_arr = p.getCameraImage(self.width, self.height, view,
proj, renderer=p.ER_TINY_RENDERER, shadow=0)
        rgb = np.reshape(img_arr[2], (self.height, self.width, 4))
       [:, :, :3]
        return rgb.astype(np.uint8)

    def capture_views(self):
        return self._capture(self.cam1_view, self.cam1_proj),
self._capture(self.cam2_view, self.cam2_proj)

    def get_gt_ee(self):
        ee_pos = p.getLinkState(self.robot_id, self.ee_link_index,
computeForwardKinematics=True)[0]
        return np.array(ee_pos, dtype=np.float32)

    def get_joint_angles(self):
        return np.array([p.getJointState(self.robot_id, i)[0] for i in
range(self.num_motor)], dtype=np.float32)

    def set_joint_positions(self, joint_targets):
        for i in range(self.num_motor):
            p.setJointMotorControl2(bodyUniqueId=self.robot_id,
jointIndex=i, controlMode=p.POSITION_CONTROL,
targetPosition=joint_targets[i], force=100)

    def step_sim(self):
        for _ in range(self.sim_steps):
            p.stepSimulation()

    def close(self):
        p.disconnect()

class PositionalEncoding2D(nn.Module):
    def __init__(self, height, width):
        super().__init__()
        y_coords = torch.linspace(-1, 1, steps=height)
        x_coords = torch.linspace(-1, 1, steps=width)
        grid_y, grid_x = torch.meshgrid(y_coords, x_coords,
indexing='ij')
        self.register_buffer('grid', torch.stack([grid_x, grid_y],
dim=0).unsqueeze(0))

    def forward(self, x):
        return torch.cat([x, self.grid.expand(x.size(0), -1, -1, -1)],

```

```

dim=1)

class SpatialConvStream(nn.Module):
    def __init__(self, in_channels=3):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(in_channels, 16, kernel_size=3, padding=1),
            nn.BatchNorm2d(16), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(16, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128), nn.ReLU(), nn.MaxPool2d(2),
            nn.AdaptiveAvgPool2d((4, 4)),
        )

    def forward(self, x):
        return self.net(x)

class NeuroKin3D(nn.Module):
    def __init__(self, image_h=100, image_w=100):
        super().__init__()
        self.pos_enc = PositionalEncoding2D(height=image_h,
width=image_w)
        self.cam1_stream = SpatialConvStream(in_channels=3)
        self.cam2_stream = SpatialConvStream(in_channels=3)
        with torch.no_grad():
            dummy = torch.zeros(1, 3, image_h, image_w)
            feat_dim = int(np.prod(self.cam1_stream(dummy).shape[1:]))
            self.virtual_frame_proj = nn.Sequential(nn.Linear(feat_dim *
2, 512), nn.LayerNorm(512), nn.ReLU())
            self.head = nn.Sequential(nn.Linear(512, 256), nn.ReLU(),
nn.Linear(256, 128), nn.ReLU(), nn.Linear(128, 7))

    def forward(self, cam1, cam2):
        cam1_spatial = self.pos_enc(cam1)
        cam2_spatial = self.pos_enc(cam2)
        f1 = self.cam1_stream(cam1_spatial).flatten(1)
        f2 = self.cam2_stream(cam2_spatial).flatten(1)
        return self.head(self.virtual_frame_proj(torch.cat([f1, f2],
dim=1)))

IN_COLAB = 'google.colab' in sys.modules
if IN_COLAB:
    from google.colab import drive
    drive.mount('/content/drive')

```

```

ROOT_CANDIDATES = [
    '/content/drive/MyDrive/robot_self_modelling',
    '/content/robot_self_modelling',
    os.path.abspath('.'),
]

ROOT_DIR = next((p for p in ROOT_CANDIDATES if os.path.exists(p)),
os.path.abspath('.'))
URDF_PATH = os.path.join(ROOT_DIR, 'RobotArmURDF', '4dof_1st', 'urdf',
'4dof_1st.urdf')
MODEL_PATH = os.path.join(ROOT_DIR, 'models', 'neurokin_3d_best.pth')

if not os.path.exists(URDF_PATH):
    raise FileNotFoundError(f'URDF not found: {URDF_PATH}')

OUTPUT_DIR = os.path.join(ROOT_DIR, 'models')
os.makedirs(OUTPUT_DIR, exist_ok=True)
MODEL_PATH = os.path.join(OUTPUT_DIR, 'neurokin_3d_best.pth')

model = NeuroKin3D().to(device)
if not os.path.exists(MODEL_PATH):
    raise FileNotFoundError(f'Model not found: {MODEL_PATH}')

checkpoint = torch.load(MODEL_PATH, map_location=device,
weights_only=False)
state_dict = checkpoint['model_state_dict'] if isinstance(checkpoint,
dict) and 'model_state_dict' in checkpoint else checkpoint
model.load_state_dict(state_dict, strict=True)
model.eval()

env = DualCameraEnv(URDF_PATH, width=100, height=100, cam_dist=1.0,
sim_steps=50)

JOINT_LIMITS = []
for i in range(env.num_motor):
    info = p.getJointInfo(env.robot_id, i)
    lower, upper = info[8], info[9]
    if lower >= upper:
        lower, upper = -env.max_angle_rad, env.max_angle_rad
    JOINT_LIMITS.append((lower, upper))
JOINT_LIMITS = np.array(JOINT_LIMITS, dtype=np.float32)

Drive already mounted at /content/drive; to attempt to forcibly
remount, call drive.mount("/content/drive", force_remount=True).

def estimate_ee_xyz(rgb1, rgb2, model, device):
    mask1 = preprocess_image(rgb1)
    mask2 = preprocess_image(rgb2)
    cam1 = torch.from_numpy(mask1.astype(np.float32) / 255.0)[None,
None, ...].to(device)

```



```

    cam2 = torch.from_numpy(mask2.astype(np.float32) / 255.0)[None,
None, ...].to(device)
    with torch.no_grad():
        raw_pred = model(cam1, cam2).cpu().numpy().squeeze(0)
    return raw_pred[:3], raw_pred[3:7] * 90.0

def sample_reachable_target(root_dir):
    test_path = os.path.join(root_dir, 'data', 'sim_data_multi_view',
'mv_robot_test.npz')
    if os.path.exists(test_path):
        data = np.load(test_path)
        ee_xyz = data['ee_xyz']
        return ee_xyz[np.random.randint(len(ee_xyz))]
    return np.array([0.15, 0.0, 0.15], dtype=np.float32)

def run_thesis_benchmarks(
    env, model, device, target_xyz, joint_limits,
    mode="baseline",
    num_steps=200, step_gain=0.25, step_gain_i=0.06, max_step=0.01
):
    gt_log = []
    err_log = []
    est_log = []
    est_joints_log = []
    gt_joints_log = []

    env.reset()

    current_commanded_xyz = env.get_gt_ee().copy()
    prev_visual_error = np.zeros(3, dtype=np.float32)

    lower = joint_limits[:, 0]
    upper = joint_limits[:, 1]
    ranges = upper - lower
    joint_damping = [0.01] * env.num_motor

    for _ in range(num_steps):
        gt_xyz = env.get_gt_ee()
        gt_joints_deg = env.get_joint_angles() * 180.0 / np.pi
        gt_log.append(gt_xyz)
        gt_joints_log.append(gt_joints_deg)
        err_log.append(np.linalg.norm(target_xyz - gt_xyz))

        if mode in ["baseline", "triumph"]:
            rgb1, rgb2 = env.capture_views()
            est_xyz, est_joints_deg = estimate_ee_xyz(rgb1, rgb2,
model, device)
            est_log.append(est_xyz)

```

```

        est_joints_log.append(est_joints_deg)
    elif mode == "failure":
        est_xyz = current_commanded_xyz.copy()
        est_joints_deg = np.zeros(4)
        est_log.append(est_xyz)
        est_joints_log.append(est_joints_deg)

    visual_error = target_xyz - est_xyz
    delta = step_gain * (visual_error - prev_visual_error) +
step_gain_i * visual_error

    if max_step is not None:
        delta = np.clip(delta, -max_step, max_step)

    current_commanded_xyz = current_commanded_xyz + delta

    rest = np.deg2rad(est_joints_deg)
    joint_targets = p.calculateInverseKinematics(
        env.robot_id,
        env.ee_link_index,
        current_commanded_xyz,
        lowerLimits=lower.tolist(),
        upperLimits=upper.tolist(),
        jointRanges=ranges.tolist(),
        restPoses=rest.tolist(),
        jointDamping=joint_damping,
        residualThreshold=1e-4,
        maxNumIterations=100,
    )

    joint_targets_clipped =
np.clip(np.array(joint_targets[:env.num_motor]), lower, upper)

    if mode in ["failure", "triumph"]:
        joint_targets_clipped[1] =
np.clip(joint_targets_clipped[1] + 0.3, lower[1], upper[1])

    env.set_joint_positions(joint_targets_clipped)
    env.step_sim()
    prev_visual_error = visual_error.copy()

    return np.array(gt_log), np.array(est_log), np.array(err_log),
np.array(est_joints_log), np.array(gt_joints_log)

target_xyz = sample_reachable_target(ROOT_DIR)

print("Running Baseline (Healthy Arm, Visual Control)...")
gt_baseline, est_baseline, err_baseline, est_j_baseline, gt_j_baseline
= run_thesis_benchmarks(env, model, device, target_xyz, JOINT_LIMITS,
mode="baseline", num_steps=200)

```

```

print("Running Failure (Damaged Arm, Blind IK)...")
gt_failure, est_failure, err_failure, est_j_failure, gt_j_failure =
run_thesis_benchmarks(env, model, device, target_xyz, JOINT_LIMITS,
mode="failure", num_steps=200)

print("Running Triumph (Damaged Arm, NeuroKin3D Visual Control)...")
gt_triumph, est_triumph, err_triumph, est_j_triumph, gt_j_triumph =
run_thesis_benchmarks(env, model, device, target_xyz, JOINT_LIMITS,
mode="triumph", num_steps=200)

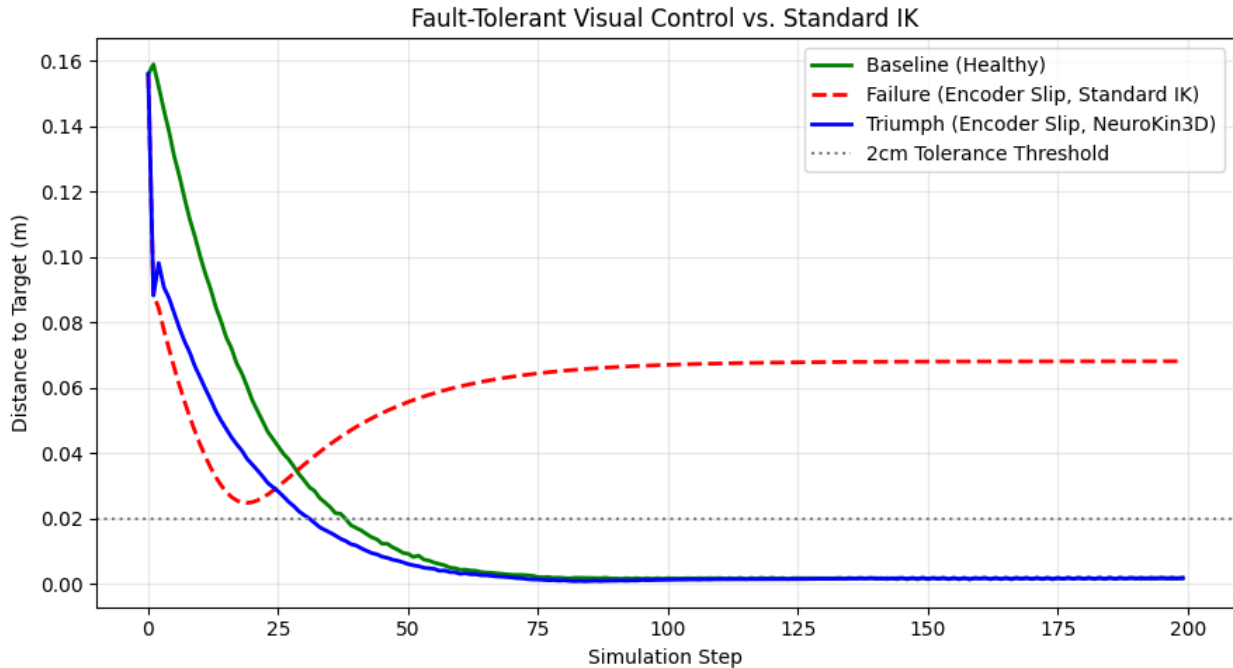
plt.figure(figsize=(9, 5))
plt.plot(err_baseline, label="Baseline (Healthy)", color='green',
linewidth=2)
plt.plot(err_failure, label="Failure (Encoder Slip, Standard IK)",
color='red', linestyle='--', linewidth=2)
plt.plot(err_triumph, label="Triumph (Encoder Slip, NeuroKin3D)",
color='blue', linewidth=2)

plt.axhline(y=0.02, color='gray', linestyle=':', label="2cm Tolerance
Threshold")
plt.xlabel('Simulation Step')
plt.ylabel('Distance to Target (m)')
plt.title('Fault-Tolerant Visual Control vs. Standard IK')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print(f"Final Error - Baseline: {err_baseline[-1]:.4f} m")
print(f"Final Error - Failure: {err_failure[-1]:.4f} m")
print(f"Final Error - Triumph: {err_triumph[-1]:.4f} m")

Running Baseline (Healthy Arm, Visual Control)...
Running Failure (Damaged Arm, Blind IK)...
Running Triumph (Damaged Arm, NeuroKin3D Visual Control)...

```



```
Final Error - Baseline: 0.0016 m
Final Error - Failure: 0.0681 m
Final Error - Triumph: 0.0019 m
```

```
print('\n--- Analysis of NeuroKin3D Estimation Error (Triumph
Scenario) ---')
```

```
# Calculate the error between estimated and ground truth end-effector
positions for the triumph run
```

```
estimation_error_triumph = np.linalg.norm(est_triumph - gt_triumph,
axis=1)
```

```
print(f'NeuroKin3D Estimation Error (Triumph) | mean:
{estimation_error_triumph.mean():.4f} m | median:
{np.median(estimation_error_triumph):.4f} m | max:
{estimation_error_triumph.max():.4f} m')
```

```
print('\nGT range per axis (Triumph):')
```

```
print('  min:', np.round(gt_triumph.min(axis=0), 4), 'm')
```

```
print('  max:', np.round(gt_triumph.max(axis=0), 4), 'm')
```

```
print('\nEST range per axis (Triumph):')
```

```
print('  min:', np.round(est_triumph.min(axis=0), 4), 'm')
```

```
print('  max:', np.round(est_triumph.max(axis=0), 4), 'm')
```

```
--- Analysis of NeuroKin3D Estimation Error (Triumph Scenario) ---
NeuroKin3D Estimation Error (Triumph) | mean: 0.0028 m | median:
0.0023 m | max: 0.0099 m
```

GT range per axis (Triumph):

min: [0.0813 0.0005 0.1438] m

max: [0.2036 0.0406 0.2345] m

EST range per axis (Triumph):

min: [0.0811 0.0008 0.1427] m

max: [0.2019 0.0396 0.2352] m

Notebook A: 100-Robot Loop Without Faults

This notebook runs the final-success implementation for 100 trials with NeuroKin off and on, using the clean control loop only.

```
import json
import logging
import os
import random
import time
from pathlib import Path

import cv2
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import pybullet as p
import pybullet_data
import torch
import torch.nn as nn

NUM_ROBOTS = 100
SEED = 42
OUT_DIR = Path.cwd() / "phase2_clean_100_no_fault"
OUT_DIR.mkdir(parents=True, exist_ok=True)

logging.basicConfig(level=logging.INFO, format="%(message)s")
logger = logging.getLogger("phase2_clean")

def set_seed(seed: int = SEED):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

def find_project_root() -> Path:
    candidates = [
        os.environ.get("ROBOT_SELF_MODELLING_ROOT"),
        "/content/drive/MyDrive/robot_self_modelling",
        "/content/robot_self_modelling",
        os.path.expanduser("~/robot_self_modelling"),
        os.path.join(os.path.expanduser("~"), "Downloads",
"robot_self_modelling"),
        str(Path.cwd()),
        str(Path.cwd().parent),
    ]
```

```

    ]
    for root in candidates:
        if not root:
            continue
        root_path = Path(root).expanduser().resolve()
        urdf_candidate = root_path / "RobotArmURDF" / "4dof_1st" /
"urdf" / "4dof_1st.urdf"
        if urdf_candidate.exists():
            return root_path
    return Path.cwd().resolve()

ROOT_DIR = find_project_root()
URDF_PATH = ROOT_DIR / "RobotArmURDF" / "4dof_1st" / "urdf" /
"4dof_1st.urdf"
MODEL_PATH = ROOT_DIR / "models" / "neurokin_3d_best.pth"
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

set_seed(SEED)
logger.info(f"ROOT_DIR: {ROOT_DIR}")
logger.info(f"URDF_PATH: {URDF_PATH}")
logger.info(f"MODEL_PATH: {MODEL_PATH}")
logger.info(f"DEVICE: {DEVICE}")

def build_camera_matrices(camera_pos, target_pos, up_vec, fov=42,
aspect=1.0, near=0.1, far=100.0):
    view = p.computeViewMatrix(cameraEyePosition=camera_pos,
cameraTargetPosition=target_pos, cameraUpVector=up_vec)
    proj = p.computeProjectionMatrixFOV(fov=fov, aspect=aspect,
nearVal=near, farVal=far)
    return view, proj

def preprocess_image(rgb, threshold=240):
    gray = cv2.cvtColor(rgb, cv2.COLOR_RGB2GRAY)
    return (gray < threshold).astype(np.uint8) * 255

def load_model(model_path=MODEL_PATH):
    if not model_path.exists():
        raise FileNotFoundError(f"Model not found: {model_path}")
    model = NeuroKin3D().to(DEVICE)
    checkpoint = torch.load(model_path, map_location=DEVICE,
weights_only=False)
    state_dict = checkpoint["model_state_dict"] if
isinstance(checkpoint, dict) and "model_state_dict" in checkpoint else
checkpoint
    model.load_state_dict(state_dict, strict=True)
    model.eval()

```

```

return model

class DualCameraEnv:
    def __init__(self, urdf_path, width=100, height=100, cam_dist=1.0,
sim_steps=50):
        self.urdf_path = str(urdf_path)
        self.width = width
        self.height = height
        self.cam_dist = cam_dist
        self.sim_steps = sim_steps
        self.num_motor = 4
        self.physics_client = p.connect(p.DIRECT)
        p.setAdditionalSearchPath(pybullet_data.getDataPath())
        p.setGravity(0, 0, -9.8)
        plane = p.createVisualShape(shapeType=p.GEOM_PLANE,
        rgbColor=[1, 1, 1], planeNormal=[0, 0, 1])
        p.createMultiBody(baseMass=0, baseVisualShapeIndex=plane,
        basePosition=[0, 0, -0.109])
        self.robot_id = p.loadURDF(self.urdf_path, [0, 0, -0.108],
        p.getQuaternionFromEuler([0, 0, -np.pi / 2]), useFixedBase=1)
        self.ee_link_index = p.getNumJoints(self.robot_id) - 1
        self.cam1_view, self.cam1_proj =
        build_camera_matrices([cam_dist, 0, 0], [0, 0, 0], [0, 0, 1])
        self.cam2_view, self.cam2_proj = build_camera_matrices([0,
        cam_dist, 0], [0, 0, 0], [0, 0, 1])

    def reset(self):
        for i in range(self.num_motor):
            p.resetJointState(self.robot_id, i, 0)
        p.stepSimulation()

    def close(self):
        p.disconnect()

    def get_gt_ee(self):
        return np.array(p.getLinkState(self.robot_id,
        self.ee_link_index, computeForwardKinematics=True)[0],
        dtype=np.float32)

    def get_joint_angles(self):
        return np.array([p.getJointState(self.robot_id, i)[0] for i in
        range(self.num_motor)], dtype=np.float32)

    def capture_views(self):
        img1 = p.getCameraImage(self.width, self.height,
        self.cam1_view, self.cam1_proj, renderer=p.ER_TINY_RENDERER, shadow=0)
        img2 = p.getCameraImage(self.width, self.height,
        self.cam2_view, self.cam2_proj, renderer=p.ER_TINY_RENDERER, shadow=0)
        rgb1 = np.reshape(img1[2], (self.height, self.width, 4))[:, :,

```



```

:3].astype(np.uint8)
    rgb2 = np.reshape(img2[2], (self.height, self.width, 4))[:, :,
:3].astype(np.uint8)
    return rgb1, rgb2

    def step(self, joint_targets):
        for i in range(self.num_motor):
            p.setJointMotorControl2(self.robot_id, i,
controlMode=p.POSITION_CONTROL,
targetPosition=float(joint_targets[i]), force=100)
            for _ in range(self.sim_steps):
                p.stepSimulation()

class PositionalEncoding2D(nn.Module):
    def __init__(self, height=100, width=100):
        super().__init__()
        y = torch.linspace(-1, 1, steps=height)
        x = torch.linspace(-1, 1, steps=width)
        gy, gx = torch.meshgrid(y, x, indexing="ij")
        self.register_buffer("grid", torch.stack([gx, gy],
dim=0).unsqueeze(0))

    def forward(self, x):
        return torch.cat([x, self.grid.expand(x.size(0), -1, -1, -1)],
dim=1)

class SpatialConvStream(nn.Module):
    def __init__(self, in_channels=3):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(in_channels, 16, 3, padding=1),
nn.BatchNorm2d(16), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(16, 32, 3, padding=1), nn.BatchNorm2d(32),
nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, padding=1), nn.BatchNorm2d(64),
nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(64, 128, 3, padding=1), nn.BatchNorm2d(128),
nn.ReLU(), nn.MaxPool2d(2),
            nn.AdaptiveAvgPool2d((4, 4)),
        )

    def forward(self, x):
        return self.net(x)

class NeuroKin3D(nn.Module):
    def __init__(self, image_h=100, image_w=100):
        super().__init__()

```

```

        self.pos_enc = PositionalEncoding2D(image_h, image_w)
        self.cam1_stream = SpatialConvStream(in_channels=3)
        self.cam2_stream = SpatialConvStream(in_channels=3)
        with torch.no_grad():
            dummy = torch.zeros(1, 3, image_h, image_w)
            feat_dim = int(np.prod(self.cam1_stream(dummy).shape[1:]))
            self.virtual_frame_proj = nn.Sequential(nn.Linear(feat_dim *
2, 512), nn.LayerNorm(512), nn.ReLU())
            self.head = nn.Sequential(nn.Linear(512, 256), nn.ReLU(),
nn.Linear(256, 128), nn.ReLU(), nn.Linear(128, 7))

    def forward(self, cam1, cam2):
        cam1_spatial = self.pos_enc(cam1)
        cam2_spatial = self.pos_enc(cam2)
        f1 = self.cam1_stream(cam1_spatial).flatten(1)
        f2 = self.cam2_stream(cam2_spatial).flatten(1)
        return self.head(self.virtual_frame_proj(torch.cat([f1, f2],
dim=1)))

def estimate_ee_xyz(rgb1, rgb2, model, device=DEVICE):
    mask1 = preprocess_image(rgb1)
    mask2 = preprocess_image(rgb2)
    h = int(model.pos_enc.grid.shape[2])
    w = int(model.pos_enc.grid.shape[3])
    if mask1.shape != (h, w):
        mask1 = cv2.resize(mask1, (w, h),
interpolation=cv2.INTER_NEAREST)
        mask2 = cv2.resize(mask2, (w, h),
interpolation=cv2.INTER_NEAREST)
        cam1 = torch.from_numpy(mask1.astype(np.float32) / 255.0)[None,
None, ...].to(device)
        cam2 = torch.from_numpy(mask2.astype(np.float32) / 255.0)[None,
None, ...].to(device)
        with torch.inference_mode():
            pred = model(cam1, cam2).cpu().numpy().squeeze(0)
        return pred[:3].astype(np.float32), pred[3:7].astype(np.float32) *
90.0

def run_single_robot_chain_baseline(urdf_path, num_stages=100,
num_steps_per_stage=200, tolerance=0.01, seed=42):
    np.random.seed(seed)
    physics_client = p.connect(p.DIRECT)
    p.setAdditionalSearchPath(pybullet_data.getDataPath())
    p.setGravity(0, 0, -9.8)
    plane = p.createVisualShape(shapeType=p.GEOM_PLANE, rgbColor=[1,
1, 1], planeNormal=[0, 0, 1])
    p.createMultiBody(baseMass=0, baseVisualShapeIndex=plane,
basePosition=[0, 0, -0.109])
    robot_id = p.loadURDF(str(urdf_path), [0, 0, -0.108],

```

```

p.getQuaternionFromEuler([0, 0, -np.pi / 2]), useFixedBase=1)
ee_link_index = p.getNumJoints(robot_id) - 1
num_motor = 4
joint_limits = []
for i in range(num_motor):
    info = p.getJointInfo(robot_id, i)
    lower, upper = info[8], info[9]
    if lower >= upper:
        lower, upper = -np.pi / 2, np.pi / 2
    joint_limits.append((lower, upper))
joint_limits = np.array(joint_limits, dtype=np.float32)
lower = joint_limits[:, 0]
upper = joint_limits[:, 1]
ranges = upper - lower
stage_logs = []
successes = []
start = time.time()
try:
    for stage_idx in range(num_stages):
        for i in range(num_motor):
            p.resetJointState(robot_id, i, 0)
            p.stepSimulation()
            home_ee = np.array(p.getLinkState(robot_id, ee_link_index,
computeForwardKinematics=True)[0], dtype=np.float32)
            target_pos = np.array([0.15 + (np.random.random() - 0.5) *
0.1, (np.random.random() - 0.5) * 0.2, 0.15], dtype=np.float32)
            current_commanded = home_ee.copy()
            prev_error = np.zeros(3, dtype=np.float32)
            trace = []
            stage_ok = False
            for step_idx in range(num_steps_per_stage):
                gt_ee = np.array(p.getLinkState(robot_id,
ee_link_index, computeForwardKinematics=True)[0], dtype=np.float32)
                trace.append(gt_ee.copy())
                if np.linalg.norm(target_pos - gt_ee) <= tolerance:
                    stage_ok = True
                    break
                error = target_pos - gt_ee
                delta = 0.25 * (error - prev_error) + 0.06 * error
                delta = np.clip(delta, -0.01, 0.01)
                current_commanded = current_commanded + delta
                rest = np.array([p.getJointState(robot_id, i)[0] for i
in range(num_motor)])
                joint_targets = p.calculateInverseKinematics(
                    robot_id, ee_link_index, current_commanded,
                    lowerLimits=lower.tolist(),
                    upperLimits=upper.tolist(),
                    jointRanges=ranges.tolist(),
                    restPoses=rest.tolist(),

```

```

        jointDamping=[0.01] * num_motor,
        residualThreshold=1e-4, maxNumIterations=100,
    )
    joint_targets = np.array(joint_targets[:num_motor],
dtype=np.float32)
    for i in range(num_motor):
        p.setJointMotorControl2(robot_id, i,
controlMode=p.POSITION_CONTROL,
targetPosition=float(joint_targets[i]), force=100)
        for _ in range(50):
            p.stepSimulation()
            prev_error = error.copy()
            stage_logs.append({"stage": f"S{stage_idx + 1}", "target":
target_pos, "trace": np.array(trace), "success": stage_ok})
            successes.append(stage_ok)
    finally:
        elapsed = time.time() - start
        p.disconnect()
    return stage_logs, successes, elapsed

def run_single_robot_chain_neurokin(urdf_path, model, device,
num_stages=100, num_steps_per_stage=200, est_stride=2, tolerance=0.01,
seed=42):
    np.random.seed(seed)
    physics_client = p.connect(p.DIRECT)
    p.setAdditionalSearchPath(pybullet_data.getDataPath())
    p.setGravity(0, 0, -9.8)
    plane = p.createVisualShape(shapeType=p.GEOM_PLANE, rgbColor=[1,
1, 1], planeNormal=[0, 0, 1])
    p.createMultiBody(baseMass=0, baseVisualShapeIndex=plane,
basePosition=[0, 0, -0.109])
    robot_id = p.loadURDF(str(urdf_path), [0, 0, -0.108],
p.getQuaternionFromEuler([0, 0, -np.pi / 2]), useFixedBase=1)
    ee_link_index = p.getNumJoints(robot_id) - 1
    num_motor = 4
    joint_limits = []
    for i in range(num_motor):
        info = p.getJointInfo(robot_id, i)
        lower, upper = info[8], info[9]
        if lower >= upper:
            lower, upper = -np.pi / 2, np.pi / 2
        joint_limits.append((lower, upper))
    joint_limits = np.array(joint_limits, dtype=np.float32)
    lower = joint_limits[:, 0]
    upper = joint_limits[:, 1]
    ranges = upper - lower
    cam1_view, cam1_proj = build_camera_matrices([1.0, 0, 0], [0, 0,
0], [0, 0, 1])

```

```

    cam2_view, cam2_proj = build_camera_matrices([0, 1.0, 0], [0, 0,
0], [0, 0, 1])
    stage_logs = []
    successes = []
    start = time.time()
    try:
        for stage_idx in range(num_stages):
            for i in range(num_motor):
                p.resetJointState(robot_id, i, 0)
                p.stepSimulation()
                home_ee = np.array(p.getLinkState(robot_id, ee_link_index,
computeForwardKinematics=True)[0], dtype=np.float32)
                target_pos = np.array([0.15 + (np.random.random() - 0.5) *
0.1, (np.random.random() - 0.5) * 0.2, 0.15], dtype=np.float32)
                current_commanded = home_ee.copy()
                prev_error = np.zeros(3, dtype=np.float32)
                est_ee_sm = None
                last_est_joints_deg = None
                trace = []
                stage_ok = False
                for step_idx in range(num_steps_per_stage):
                    gt_ee = np.array(p.getLinkState(robot_id,
ee_link_index, computeForwardKinematics=True)[0], dtype=np.float32)
                    trace.append(gt_ee.copy())
                    if np.linalg.norm(target_pos - gt_ee) <= tolerance:
                        stage_ok = True
                        break
                    if step_idx % max(1, int(est_stride)) == 0:
                        img1 = p.getCameraImage(100, 100, cam1_view,
cam1_proj, renderer=p.ER_TINY_RENDERER, shadow=0)
                        img2 = p.getCameraImage(100, 100, cam2_view,
cam2_proj, renderer=p.ER_TINY_RENDERER, shadow=0)
                        rgb1 = np.reshape(img1[2], (100, 100, 4))
                        [:, :, :3].astype(np.uint8)
                        rgb2 = np.reshape(img2[2], (100, 100, 4))
                        [:, :, :3].astype(np.uint8)
                        est_ee, est_joints_deg = estimate_ee_xyz(rgb1,
rgb2, model, device)
                        est_ee_sm = est_ee if est_ee_sm is None else 0.6 *
est_ee_sm + 0.4 * est_ee
                        last_est_joints_deg = est_joints_deg
                    if last_est_joints_deg is None or est_ee_sm is None:
                        continue
                    error = target_pos - est_ee_sm
                    delta = 0.25 * (error - prev_error) + 0.06 * error
                    delta = np.clip(delta, -0.01, 0.01)
                    current_commanded = current_commanded + delta
                    workspace_min = np.array([0.02, -0.20, 0.05],
dtype=np.float32)

```

```

        workspace_max = np.array([0.30, 0.20, 0.25],
dtype=np.float32)
        current_commanded = np.clip(current_commanded,
workspace_min, workspace_max)
        rest = np.deg2rad(last_est_joints_deg)
        joint_targets = p.calculateInverseKinematics(
            robot_id, ee_link_index, current_commanded,
            lowerLimits=lower.tolist(),
upperLimits=upper.tolist(),
            jointRanges=ranges.tolist(),
restPoses=rest.tolist(),
            jointDamping=[0.01] * num_motor,
residualThreshold=1e-4, maxNumIterations=100,
        )
        joint_targets = np.array(joint_targets[:num_motor],
dtype=np.float32)
        for i in range(num_motor):
            p.setJointMotorControl2(robot_id, i,
controlMode=p.POSITION_CONTROL,
targetPosition=float(joint_targets[i]), force=100)
            for _ in range(50):
                p.stepSimulation()
                prev_error = error.copy()
                stage_logs.append({"stage": f"S{stage_idx + 1}", "target":
target_pos, "trace": np.array(trace), "success": stage_ok})
                successes.append(stage_ok)
            finally:
                elapsed = time.time() - start
                p.disconnect()
        return stage_logs, successes, elapsed

def summarize_results(df, label):
    summary = df.groupby("variant").agg(
        success_rate=("success", "mean"),
        mean_steps=("steps", "mean"),
        median_steps=("steps", "median"),
        failures=("success", lambda s: int((~s).sum())),
    ).reset_index()
    summary["label"] = label
    return summary

model = load_model()
clean_config = {
    "num_robots": NUM_ROBOTS,
    "seed": SEED,
    "urdf_path": str(URDF_PATH),
    "model_path": str(MODEL_PATH),
    "device": str(DEVICE),
}

```

```

(OUT_DIR / "config.json").write_text(json.dumps(clean_config,
indent=2))

logger.info("Running 100-trial baseline (NeuroKin off)...")
baseline_logs, baseline_successes, baseline_time =
run_single_robot_chain_baseline(URDF_PATH, num_stages=NUM_ROBOTS,
seed=SEED)
baseline_df = pd.DataFrame([
    {"trial": i + 1, "variant": "baseline", "success": bool(ok),
"steps": len(log["trace"]), "target_x": float(log["target"][0]),
"target_y": float(log["target"][1]), "target_z": float(log["target"]
[2])}
    for i, (log, ok) in enumerate(zip(baseline_logs,
baseline_successes))
])
baseline_df.to_csv(OUT_DIR / "baseline_no_fault.csv", index=False)

logger.info("Running 100-trial NeuroKin run (NeuroKin on)...")
neurokin_logs, neurokin_successes, neurokin_time =
run_single_robot_chain_neurokin(URDF_PATH, model, DEVICE,
num_stages=NUM_ROBOTS, seed=SEED)
neurokin_df = pd.DataFrame([
    {"trial": i + 1, "variant": "neurokin", "success": bool(ok),
"steps": len(log["trace"]), "target_x": float(log["target"][0]),
"target_y": float(log["target"][1]), "target_z": float(log["target"]
[2])}
    for i, (log, ok) in enumerate(zip(neurokin_logs,
neurokin_successes))
])
neurokin_df.to_csv(OUT_DIR / "neurokin_no_fault.csv", index=False)

results_df = pd.concat([baseline_df, neurokin_df], ignore_index=True)
results_df.to_json(OUT_DIR / "results_no_fault.json",
orient="records", indent=2)
summary_df = summarize_results(results_df, "no_fault")
summary_df.to_csv(OUT_DIR / "summary_no_fault.csv", index=False)
print(summary_df)

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
summary_df.plot(kind="bar", x="variant", y="success_rate", ax=axes[0],
legend=False, color=["#4c78a8", "#f58518"])
axes[0].set_ylim(0, 1)
axes[0].set_title("Success rate")
axes[0].set_ylabel("rate")
summary_df.plot(kind="bar", x="variant", y="mean_steps", ax=axes[1],
legend=False, color=["#54a24b", "#e45756"])
axes[1].set_title("Mean steps")
plt.tight_layout()
plt.savefig(OUT_DIR / "comparison_no_fault.png", dpi=150)
plt.show()

```

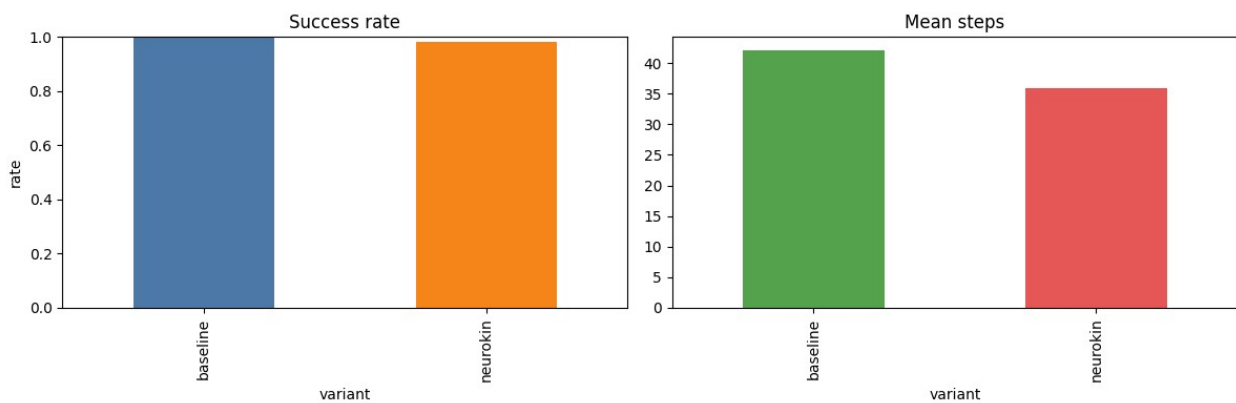
```

print(f"Baseline: {baseline_df.success.mean():.1%} success in
{baseline_time:.2f}s")
print(f"NeuroKin: {neurokin_df.success.mean():.1%} success in
{neurokin_time:.2f}s")

reloaded = pd.read_csv(OUT_DIR / "baseline_no_fault.csv")
assert len(reloaded) == NUM_ROBOTS
assert set(reloaded.columns) >= {"trial", "variant", "success",
"steps"}
print("Reload verification passed.")

```

	variant	success_rate	mean_steps	median_steps	failures
label					
0	baseline	1.00	42.16	43.0	0
	no_fault				
1	neurokin	0.98	35.93	33.0	2
	no_fault				



```

Baseline: 100.0% success in 7.58s
NeuroKin: 98.0% success in 142.45s
Reload verification passed.

```


Notebook B: 100-Robot Loop With Fault Injection

This notebook runs the final-success implementation under progressive fault injection and compares the baseline and NeuroKin variants.

```
import os
import sys
import time
import random
import importlib.util
import numpy as np
import matplotlib.pyplot as plt

def ensure_package(module_name, pip_name=None):
    if importlib.util.find_spec(module_name) is None:
        import subprocess
        package_name = pip_name or module_name
        if module_name == 'pybullet' and os.name == 'nt' and
sys.version_info >= (3, 12):
            raise RuntimeError(
                'pybullet has no reliable prebuilt wheel for this
Windows/Python 3.12 runtime. '
                'Run this notebook in Colab or a Python 3.10/3.11
environment, or install Microsoft C++ Build Tools.'
            )
        try:
            subprocess.check_call([sys.executable, '-m', 'pip', '-q',
'install', package_name])
        except subprocess.CalledProcessError as exc:
            raise RuntimeError(
                f'Could not install {package_name}. If this is
pybullet on Windows/Python 3.12, '
                'use Python 3.10/3.11, Colab, or install Microsoft C++
Build Tools.'
            ) from exc

for module_name, pip_name in [('pybullet', 'pybullet'), ('cv2',
'opencv-python'), ('torch', 'torch')]:
    ensure_package(module_name, pip_name)

import pybullet as p
import pybullet_data
import cv2
```

```

import torch
import torch.nn as nn

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Device: {device}')

def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

set_seed(42)

IN_COLAB = 'google.colab' in sys.modules
if IN_COLAB:
    from google.colab import drive
    drive.mount('/content/drive')

def find_project_root():
    candidates = [
        os.environ.get('ROBOT_SELF_MODELLING_ROOT'),
        '/content/drive/MyDrive/robot_self_modelling',
        '/content/robot_self_modelling',
        os.path.expanduser('~/.robot_self_modelling'),
        os.path.join(os.path.expanduser('~'), 'Downloads',
'robot_self_modelling'),
        os.path.abspath('.'),
        os.path.abspath('..'),
    ]
    checked = []
    for root in candidates:
        if not root:
            continue
        root = os.path.abspath(os.path.expanduser(root))
        if root in checked:
            continue
        checked.append(root)
        urdf_candidate = os.path.join(root, 'RobotArmURDF',
'4dof_1st', 'urdf', '4dof_1st.urdf')
        if os.path.exists(urdf_candidate):
            return root, checked
    return os.path.abspath('.'), checked

ROOT_DIR, ROOT_SEARCH_PATHS = find_project_root()

```

```
URDF_PATH = os.path.join(ROOT_DIR, 'RobotArmURDF', '4dof_1st', 'urdf',
'4dof_1st.urdf')
MODEL_PATH = os.path.join(ROOT_DIR, 'models', 'neurokin_3d_best.pth')
print(f'ROOT_DIR: {ROOT_DIR}')
print(f'URDF_PATH: {URDF_PATH}')
print(f'MODEL_PATH: {MODEL_PATH}')
```

Mounted at /content/drive

ROOT_DIR: /content/drive/MyDrive/robot_self_modelling

URDF_PATH:

/content/drive/MyDrive/robot_self_modelling/RobotArmURDF/4dof_1st/
urdf/4dof_1st.urdf

MODEL_PATH:

/content/drive/MyDrive/robot_self_modelling/models/neurokin_3d_best.pt
h

```
import json
import logging
import os
import random
import time
from pathlib import Path
```

```
import cv2
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import pybullet as p
import pybullet_data
import torch
import torch.nn as nn
```

NUM_ROBOTS = 100

SEED = 42

FAULT_SEED = 31415

OUT_DIR = Path.cwd() / "phase2_fault_100_injection"

OUT_DIR.mkdir(parents=True, exist_ok=True)

logging.basicConfig(level=logging.INFO, format="%(message)s")

logger = logging.getLogger("phase2_fault")

```
def set_seed(seed: int = SEED):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)
```

```

def find_project_root() -> Path:
    candidates = [
        os.environ.get("ROBOT_SELF_MODELLING_ROOT"),
        "/content/drive/MyDrive/robot_self_modelling",
        "/content/robot_self_modelling",
        os.path.expanduser("~/robot_self_modelling"),
        os.path.join(os.path.expanduser("~"), "Downloads",
"robot_self_modelling"),
        str(Path.cwd()),
        str(Path.cwd().parent),
    ]
    for root in candidates:
        if not root:
            continue
        root_path = Path(root).expanduser().resolve()
        urdf_candidate = root_path / "RobotArmURDF" / "4dof_1st" /
"urdf" / "4dof_1st.urdf"
        if urdf_candidate.exists():
            return root_path
    return Path.cwd().resolve()

ROOT_DIR = find_project_root()
URDF_PATH = ROOT_DIR / "RobotArmURDF" / "4dof_1st" / "urdf" /
"4dof_1st.urdf"
MODEL_PATH = ROOT_DIR / "models" / "neurokin_3d_best.pth"
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

set_seed(SEED)
logger.info(f"ROOT_DIR: {ROOT_DIR}")
logger.info(f"URDF_PATH: {URDF_PATH}")
logger.info(f"MODEL_PATH: {MODEL_PATH}")
logger.info(f"DEVICE: {DEVICE}")

def build_camera_matrices(camera_pos, target_pos, up_vec, fov=42,
aspect=1.0, near=0.1, far=100.0):
    view = p.computeViewMatrix(cameraEyePosition=camera_pos,
cameraTargetPosition=target_pos, cameraUpVector=up_vec)
    proj = p.computeProjectionMatrixFOV(fov=fov, aspect=aspect,
nearVal=near, farVal=far)
    return view, proj

def preprocess_image(rgb, threshold=240):
    gray = cv2.cvtColor(rgb, cv2.COLOR_RGB2GRAY)
    return (gray < threshold).astype(np.uint8) * 255

def load_model(model_path=MODEL_PATH):

```

```

    if not model_path.exists():
        raise FileNotFoundError(f"Model not found: {model_path}")
    model = NeuroKin3D().to(DEVICE)
    checkpoint = torch.load(model_path, map_location=DEVICE,
weights_only=False)
    state_dict = checkpoint["model_state_dict"] if
isinstance(checkpoint, dict) and "model_state_dict" in checkpoint else
checkpoint
    model.load_state_dict(state_dict, strict=True)
    model.eval()
    return model

class DualCameraEnv:
    def __init__(self, urdf_path, width=100, height=100, cam_dist=1.0,
sim_steps=100):
        self.urdf_path = str(urdf_path)
        self.width = width
        self.height = height
        self.cam_dist = cam_dist
        self.sim_steps = sim_steps
        self.num_motor = 4
        self.physics_client = p.connect(p.DIRECT)
        p.setAdditionalSearchPath(pybullet_data.getDataPath())
        p.setGravity(0, 0, -9.8)
        plane = p.createVisualShape(shapeType=p.GEOM_PLANE,
rgbColor=[1, 1, 1], planeNormal=[0, 0, 1])
        p.createMultiBody(baseMass=0, baseVisualShapeIndex=plane,
basePosition=[0, 0, -0.109])
        self.robot_id = p.loadURDF(self.urdf_path, [0, 0, -0.108],
p.getQuaternionFromEuler([0, 0, -np.pi / 2]), useFixedBase=1)
        self.ee_link_index = p.getNumJoints(self.robot_id) - 1
        self.cam1_view, self.cam1_proj =
build_camera_matrices([cam_dist, 0, 0], [0, 0, 0], [0, 0, 1])
        self.cam2_view, self.cam2_proj = build_camera_matrices([0,
cam_dist, 0], [0, 0, 0], [0, 0, 1])

    def reset(self):
        for i in range(self.num_motor):
            p.resetJointState(self.robot_id, i, 0)
        p.stepSimulation()

    def close(self):
        p.disconnect()

    def get_gt_ee(self):
        return np.array(p.getLinkState(self.robot_id,
self.ee_link_index, computeForwardKinematics=True)[0],
dtype=np.float32)

```

```

    def get_joint_angles(self):
        return np.array([p.getJointState(self.robot_id, i)[0] for i in
range(self.num_motor)], dtype=np.float32)

    def capture_views(self):
        img1 = p.getCameraImage(self.width, self.height,
self.cam1_view, self.cam1_proj, renderer=p.ER_TINY_RENDERER, shadow=0)
        img2 = p.getCameraImage(self.width, self.height,
self.cam2_view, self.cam2_proj, renderer=p.ER_TINY_RENDERER, shadow=0)
        rgb1 = np.reshape(img1[2], (self.height, self.width, 4))[:, :,
:3].astype(np.uint8)
        rgb2 = np.reshape(img2[2], (self.height, self.width, 4))[:, :,
:3].astype(np.uint8)
        return rgb1, rgb2

    def step(self, joint_targets):
        for i in range(self.num_motor):
            p.setJointMotorControl2(self.robot_id, i,
controlMode=p.POSITION_CONTROL,
targetPosition=float(joint_targets[i]), force=100)
            for _ in range(self.sim_steps):
                p.stepSimulation()

    def apply_fault(self, fault):
        self._active_fault = fault

class PositionalEncoding2D(nn.Module):
    def __init__(self, height=100, width=100):
        super().__init__()
        y = torch.linspace(-1, 1, steps=height)
        x = torch.linspace(-1, 1, steps=width)
        gy, gx = torch.meshgrid(y, x, indexing="ij")
        self.register_buffer("grid", torch.stack([gx, gy],
dim=0).unsqueeze(0))

    def forward(self, x):
        return torch.cat([x, self.grid.expand(x.size(0), -1, -1, -1)],
dim=1)

class SpatialConvStream(nn.Module):
    def __init__(self, in_channels=3):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(in_channels, 16, 3, padding=1),
nn.BatchNorm2d(16), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(16, 32, 3, padding=1), nn.BatchNorm2d(32),
nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, padding=1), nn.BatchNorm2d(64),

```

```

nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(64, 128, 3, padding=1), nn.BatchNorm2d(128),
nn.ReLU(), nn.MaxPool2d(2),
            nn.AdaptiveAvgPool2d((4, 4)),
        )

    def forward(self, x):
        return self.net(x)

class NeuroKin3D(nn.Module):
    def __init__(self, image_h=100, image_w=100):
        super().__init__()
        self.pos_enc = PositionalEncoding2D(image_h, image_w)
        self.cam1_stream = SpatialConvStream(in_channels=3)
        self.cam2_stream = SpatialConvStream(in_channels=3)
        with torch.no_grad():
            dummy = torch.zeros(1, 3, image_h, image_w)
            feat_dim = int(np.prod(self.cam1_stream(dummy).shape[1:]))
        self.virtual_frame_proj = nn.Sequential(nn.Linear(feat_dim *
2, 512), nn.LayerNorm(512), nn.ReLU())
        self.head = nn.Sequential(nn.Linear(512, 256), nn.ReLU(),
nn.Linear(256, 128), nn.ReLU(), nn.Linear(128, 7))

    def forward(self, cam1, cam2):
        cam1_spatial = self.pos_enc(cam1)
        cam2_spatial = self.pos_enc(cam2)
        f1 = self.cam1_stream(cam1_spatial).flatten(1)
        f2 = self.cam2_stream(cam2_spatial).flatten(1)
        return self.head(self.virtual_frame_proj(torch.cat([f1, f2],
dim=1)))

def estimate_ee_xyz(rgb1, rgb2, model, device=DEVICE):
    mask1 = preprocess_image(rgb1)
    mask2 = preprocess_image(rgb2)
    h = int(model.pos_enc.grid.shape[2])
    w = int(model.pos_enc.grid.shape[3])
    if mask1.shape != (h, w):
        mask1 = cv2.resize(mask1, (w, h),
interpolation=cv2.INTER_NEAREST)
        mask2 = cv2.resize(mask2, (w, h),
interpolation=cv2.INTER_NEAREST)
    cam1 = torch.from_numpy(mask1.astype(np.float32) / 255.0)[None,
None, ...].to(device)
    cam2 = torch.from_numpy(mask2.astype(np.float32) / 255.0)[None,
None, ...].to(device)
    with torch.inference_mode():
        pred = model(cam1, cam2).cpu().numpy().squeeze(0)
    return pred[:3].astype(np.float32), pred[3:7].astype(np.float32) *
```

90.0

```
class FaultScheduler:
    def __init__(self, fault_types=None):
        self.fault_types = fault_types or ["sensor_bias",
"actuator_stuck", "noise_burst"]
        self._scheduled = {}
        self._rng = np.random.default_rng(FAULT_SEED)

    def schedule_at(self, step, fault):
        self._scheduled.setdefault(int(step), []).append(fault)

    def randomize(self, seed):
        self._rng = np.random.default_rng(seed)
        return self

    def choose_fault_for_step(self, step):
        return list(self._scheduled.get(int(step), []))

def make_sensor_bias(magnitude=0.02):
    return {"type": "sensor_bias", "magnitude": float(magnitude)}

def make_actuator_stuck(joint=0, value=0.0):
    return {"type": "actuator_stuck", "joint": int(joint), "value":
float(value)}

def make_noise_burst(scale=0.01):
    return {"type": "noise_burst", "scale": float(scale)}

class DualCameraEnv:
    def __init__(self, urdf_path, width=100, height=100, cam_dist=1.0,
sim_steps=100):
        self.urdf_path = str(urdf_path)
        self.width = width
        self.height = height
        self.cam_dist = cam_dist
        self.sim_steps = sim_steps
        self.num_motor = 4
        self.physics_client = p.connect(p.DIRECT)
        p.setAdditionalSearchPath(pybullet_data.getDataPath())
        p.setGravity(0, 0, -9.8)
        plane = p.createVisualShape(shapeType=p.GEOM_PLANE,
rgbColor=[1, 1, 1, 1], planeNormal=[0, 0, 1])
        p.createMultiBody(baseMass=0, baseVisualShapeIndex=plane,
basePosition=[0, 0, -0.109])
        self.robot_id = p.loadURDF(self.urdf_path, [0, 0, -0.108],
p.getQuaternionFromEuler([0, 0, -np.pi / 2]), useFixedBase=1)
```



```

        self.ee_link_index = p.getNumJoints(self.robot_id) - 1
        self.cam1_view, self.cam1_proj =
build_camera_matrices([cam_dist, 0, 0], [0, 0, 0], [0, 0, 1])
        self.cam2_view, self.cam2_proj = build_camera_matrices([0,
cam_dist, 0], [0, 0, 0], [0, 0, 1])

    def reset(self):
        for i in range(self.num_motor):
            p.resetJointState(self.robot_id, i, 0)
        p.stepSimulation()

    def close(self):
        p.disconnect()

    def get_gt_ee(self):
        return np.array(p.getLinkState(self.robot_id,
self.ee_link_index, computeForwardKinematics=True)[0],
dtype=np.float32)

    def get_joint_angles(self):
        return np.array([p.getJointState(self.robot_id, i)[0] for i in
range(self.num_motor)], dtype=np.float32)

    def capture_views(self):
        img1 = p.getCameraImage(self.width, self.height,
self.cam1_view, self.cam1_proj, renderer=p.ER_TINY_RENDERER, shadow=0)
        img2 = p.getCameraImage(self.width, self.height,
self.cam2_view, self.cam2_proj, renderer=p.ER_TINY_RENDERER, shadow=0)
        rgb1 = np.reshape(img1[2], (self.height, self.width, 4))[:, :,
:3].astype(np.uint8)
        rgb2 = np.reshape(img2[2], (self.height, self.width, 4))[:, :,
:3].astype(np.uint8)
        return rgb1, rgb2

    def step(self, joint_targets):
        for i in range(self.num_motor):
            p.setJointMotorControl2(self.robot_id, i,
controlMode=p.POSITION_CONTROL,
targetPosition=float(joint_targets[i]), force=100)
            for _ in range(self.sim_steps):
                p.stepSimulation()

    def apply_fault(self, fault):
        self._active_fault = fault

class PositionalEncoding2D(nn.Module):
    def __init__(self, height=100, width=100):
        super().__init__()
        y = torch.linspace(-1, 1, steps=height)

```

```

        x = torch.linspace(-1, 1, steps=width)
        gy, gx = torch.meshgrid(y, x, indexing="ij")
        self.register_buffer("grid", torch.stack([gx, gy],
dim=0).unsqueeze(0))

    def forward(self, x):
        return torch.cat([x, self.grid.expand(x.size(0), -1, -1, -1)],
dim=1)

class SpatialConvStream(nn.Module):
    def __init__(self, in_channels=3):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(in_channels, 16, 3, padding=1),
            nn.BatchNorm2d(16), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(16, 32, 3, padding=1), nn.BatchNorm2d(32),
            nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, padding=1), nn.BatchNorm2d(64),
            nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(64, 128, 3, padding=1), nn.BatchNorm2d(128),
            nn.ReLU(), nn.MaxPool2d(2),
            nn.AdaptiveAvgPool2d((4, 4)),
        )

    def forward(self, x):
        return self.net(x)

class NeuroKin3D(nn.Module):
    def __init__(self, image_h=100, image_w=100):
        super().__init__()
        self.pos_enc = PositionalEncoding2D(image_h, image_w)
        self.cam1_stream = SpatialConvStream(in_channels=3)
        self.cam2_stream = SpatialConvStream(in_channels=3)
        with torch.no_grad():
            dummy = torch.zeros(1, 3, image_h, image_w)
            feat_dim = int(np.prod(self.cam1_stream(dummy).shape[1:]))
            self.virtual_frame_proj = nn.Sequential(nn.Linear(feat_dim *
2, 512), nn.LayerNorm(512), nn.ReLU())
            self.head = nn.Sequential(nn.Linear(512, 256), nn.ReLU(),
nn.Linear(256, 128), nn.ReLU(), nn.Linear(128, 7))

    def forward(self, cam1, cam2):
        cam1_spatial = self.pos_enc(cam1)
        cam2_spatial = self.pos_enc(cam2)
        f1 = self.cam1_stream(cam1_spatial).flatten(1)
        f2 = self.cam2_stream(cam2_spatial).flatten(1)
        return self.head(self.virtual_frame_proj(torch.cat([f1, f2],
dim=1)))

```

```

def estimate_ee_xyz(rgb1, rgb2, model, device=DEVICE):
    mask1 = preprocess_image(rgb1)
    mask2 = preprocess_image(rgb2)
    h = int(model.pos_enc.grid.shape[2])
    w = int(model.pos_enc.grid.shape[3])
    if mask1.shape != (h, w):
        mask1 = cv2.resize(mask1, (w, h),
interpolation=cv2.INTER_NEAREST)
        mask2 = cv2.resize(mask2, (w, h),
interpolation=cv2.INTER_NEAREST)
        cam1 = torch.from_numpy(mask1.astype(np.float32) / 255.0)[None,
None, ...].to(device)
        cam2 = torch.from_numpy(mask2.astype(np.float32) / 255.0)[None,
None, ...].to(device)
        with torch.inference_mode():
            pred = model(cam1, cam2).cpu().numpy().squeeze(0)
        return pred[:3].astype(np.float32), pred[3:7].astype(np.float32) *
90.0

class FaultScheduler:
    def __init__(self, fault_types=None):
        self.fault_types = fault_types or ["sensor_bias",
"actuator_stuck", "noise_burst"]
        self._scheduled = {}
        self._rng = np.random.default_rng(FAULT_SEED)

    def schedule_at(self, step, fault):
        self._scheduled.setdefault(int(step), []).append(fault)

    def randomize(self, seed):
        self._rng = np.random.default_rng(seed)
        return self

    def choose_fault_for_step(self, step):
        return list(self._scheduled.get(int(step), []))

def make_sensor_bias(magnitude=0.02):
    return {"type": "sensor_bias", "magnitude": float(magnitude)}

def make_actuator_stuck(joint=0, value=0.0):
    return {"type": "actuator_stuck", "joint": int(joint), "value":
float(value)}

def make_noise_burst(scale=0.01):
    return {"type": "noise_burst", "scale": float(scale)}

```

```

def run_single_robot_chain_faulty_baseline(urdf_path, num_stages=100,
num_steps_per_stage=200, tolerance=0.01, seed=42,
base_fault_prob=0.01, base_fault_scale=0.005, every_n_stages=5,
fault_prob_step=0.015, fault_scale_step=0.003, max_fault_prob=0.50,
max_fault_scale=0.08):
    np.random.seed(seed)
    physics_client = p.connect(p.DIRECT)
    p.setAdditionalSearchPath(pybullet_data.getDataPath())
    p.setGravity(0, 0, -9.8)
    plane = p.createVisualShape(shapeType=p.GEOM_PLANE, rgbColor=[1,
1, 1], planeNormal=[0, 0, 1])
    p.createMultiBody(baseMass=0, baseVisualShapeIndex=plane,
basePosition=[0, 0, -0.109])
    robot_id = p.loadURDF(str(urdf_path), [0, 0, -0.108],
p.getQuaternionFromEuler([0, 0, -np.pi / 2]), useFixedBase=1)
    ee_link_index = p.getNumJoints(robot_id) - 1
    num_motor = 4
    joint_limits = []
    for i in range(num_motor):
        info = p.getJointInfo(robot_id, i)
        lower, upper = info[8], info[9]
        if lower >= upper:
            lower, upper = -np.pi / 2, np.pi / 2
        joint_limits.append((lower, upper))
    joint_limits = np.array(joint_limits, dtype=np.float32)
    lower = joint_limits[:, 0]
    upper = joint_limits[:, 1]
    ranges = upper - lower
    factory_state = {"consecutive_failures": 0, "mode": "standard"}
    stage_logs = []
    successes_pre_fault = []
    successes_post_fault = []
    start = time.time()
    try:
        for stage_idx in range(num_stages):
            for i in range(num_motor):
                p.resetJointState(robot_id, i, 0)
                p.stepSimulation()
                home_ee = np.array(p.getLinkState(robot_id, ee_link_index,
computeForwardKinematics=True)[0], dtype=np.float32)
                if factory_state["consecutive_failures"] >= 2:
                    factory_state["mode"] = "recovery"
                    target_pos = np.array([0.15, 0.0, 0.15],
dtype=np.float32)
                    current_tolerance = tolerance * 2.0
                else:
                    factory_state["mode"] = "standard"
                    target_pos = np.array([0.15 + (np.random.random() -
0.5) * 0.1, (np.random.random() - 0.5) * 0.2, 0.15], dtype=np.float32)
                    current_tolerance = tolerance

```

```

        fault_tier = stage_idx // every_n_stages
        fault_prob = min(base_fault_prob + fault_tier *
fault_prob_step, max_fault_prob)
        fault_scale = min(base_fault_scale + fault_tier *
fault_scale_step, max_fault_scale)
        current_commanded = home_ee.copy()
        prev_error = np.zeros(3, dtype=np.float32)
        converged_joints = None
        converged_step = None
        phase1_success = False
        steps_executed = 0
        for step_idx in range(num_steps_per_stage):
            steps_executed += 1
            gt_ee = np.array(p.getLinkState(robot_id,
ee_link_index, computeForwardKinematics=True)[0], dtype=np.float32)
            if np.linalg.norm(target_pos - gt_ee) <=
current_tolerance:
                converged_step = step_idx
                phase1_success = True
                converged_joints =
np.array([p.getJointState(robot_id, i)[0] for i in range(num_motor)],
dtype=np.float32)
                break
            error = target_pos - gt_ee
            delta = 0.25 * (error - prev_error) + 0.06 * error
            delta = np.clip(delta, -0.01, 0.01)
            current_commanded = current_commanded + delta
            rest = np.array([p.getJointState(robot_id, i)[0] for i
in range(num_motor)])
            joint_targets = p.calculateInverseKinematics(robot_id,
ee_link_index, current_commanded, lowerLimits=lower.tolist(),
upperLimits=upper.tolist(), jointRanges=ranges.tolist(),
restPoses=rest.tolist(), jointDamping=[0.01] * num_motor,
residualThreshold=1e-4, maxNumIterations=100)
            converged_joints = np.array(joint_targets[:num_motor],
dtype=np.float32)
            for i in range(num_motor):
                p.setJointMotorControl2(robot_id, i,
controlMode=p.POSITION_CONTROL,
targetPosition=float(converged_joints[i]), force=100)
            for _ in range(50):
                p.stepSimulation()
                prev_error = error.copy()
                successes_pre_fault.append(phase1_success)
                if not phase1_success or converged_joints is None:
                    post_fault_ok = False
                    successes_post_fault.append(post_fault_ok)
                    stage_logs.append({"stage": f"S{stage_idx + 1}",
"target": target_pos, "mode": factory_state["mode"], "steps":
steps_executed, "success": post_fault_ok})

```

```

        if post_fault_ok:
            factory_state["consecutive_failures"] = 0
        else:
            factory_state["consecutive_failures"] += 1
        continue
    joint_slip = np.zeros(num_motor, dtype=np.float32)
    post_fault_ok = False
    for step_idx in range(converged_step,
num_steps_per_stage):
        steps_executed += 1
        if np.random.random() < fault_prob:
            fault_joint = np.random.randint(0, num_motor)
            joint_slip[fault_joint] += np.random.normal(0.0,
fault_scale)
            joint_slip[fault_joint] =
np.clip(joint_slip[fault_joint], -0.25, 0.25)
            faulted_joints = np.clip(converged_joints +
joint_slip, lower, upper)
            for i in range(num_motor):
                p.setJointMotorControl2(robot_id, i,
controlMode=p.POSITION_CONTROL,
targetPosition=float(faulted_joints[i]), force=100)
            for _ in range(50):
                p.stepSimulation()
            gt_ee = np.array(p.getLinkState(robot_id,
ee_link_index, computeForwardKinematics=True)[0], dtype=np.float32)
            if np.linalg.norm(target_pos - gt_ee) <=
current_tolerance:
                post_fault_ok = True
            else:
                post_fault_ok = False
                break
            successes_post_fault.append(post_fault_ok)
            stage_logs.append({"stage": f"S{stage_idx + 1}", "target":
target_pos, "mode": factory_state["mode"], "steps": steps_executed,
"success": post_fault_ok})
            if post_fault_ok:
                factory_state["consecutive_failures"] = 0
            else:
                factory_state["consecutive_failures"] += 1
    finally:
        elapsed = time.time() - start
        p.disconnect()
    return stage_logs, successes_pre_fault, successes_post_fault,
elapsed

def run_single_robot_chain_faulty_neurokin_v2(urdf_path, model,
device, num_stages=100, num_steps_per_stage=200, est_stride=2,
tolerance=0.01, seed=42, base_fault_prob=0.01, base_fault_scale=0.005,

```

```

every_n_stages=5, fault_prob_step=0.015, fault_scale_step=0.003,
max_fault_prob=0.50, max_fault_scale=0.08):
    np.random.seed(seed)
    physics_client = p.connect(p.DIRECT)
    p.setAdditionalSearchPath(pybullet_data.getDataPath())
    p.setGravity(0, 0, -9.8)
    plane = p.createVisualShape(shapeType=p.GEOM_PLANE, rgbColor=[1,
1, 1], planeNormal=[0, 0, 1])
    p.createMultiBody(baseMass=0, baseVisualShapeIndex=plane,
basePosition=[0, 0, -0.109])
    robot_id = p.loadURDF(str(urdf_path), [0, 0, -0.108],
p.getQuaternionFromEuler([0, 0, -np.pi / 2]), useFixedBase=1)
    ee_link_index = p.getNumJoints(robot_id) - 1
    num_motor = 4
    joint_limits = []
    for i in range(num_motor):
        info = p.getJointInfo(robot_id, i)
        lower, upper = info[8], info[9]
        if lower >= upper:
            lower, upper = -np.pi / 2, np.pi / 2
        joint_limits.append((lower, upper))
    joint_limits = np.array(joint_limits, dtype=np.float32)
    lower = joint_limits[:, 0]
    upper = joint_limits[:, 1]
    ranges = upper - lower
    cam1_view, cam1_proj = build_camera_matrices([1.0, 0, 0], [0, 0,
0], [0, 0, 1])
    cam2_view, cam2_proj = build_camera_matrices([0, 1.0, 0], [0, 0,
0], [0, 0, 1])
    factory_state = {"consecutive_failures": 0, "mode": "standard"}
    stage_logs = []
    successes = []
    start = time.time()
    try:
        for stage_idx in range(num_stages):
            for i in range(num_motor):
                p.resetJointState(robot_id, i, 0)
                p.stepSimulation()
                home_ee = np.array(p.getLinkState(robot_id, ee_link_index,
computeForwardKinematics=True)[0], dtype=np.float32)
                if factory_state["consecutive_failures"] >= 2:
                    factory_state["mode"] = "recovery"
                    target_pos = np.array([0.15, 0.0, 0.15],
dtype=np.float32)
                    current_tolerance = tolerance * 2.0
                else:
                    factory_state["mode"] = "standard"
                    target_pos = np.array([0.15 + (np.random.random() -
0.5) * 0.1, (np.random.random() - 0.5) * 0.2, 0.15], dtype=np.float32)
                    current_tolerance = tolerance

```

```

        current_commanded = home_ee.copy()
        prev_error = np.zeros(3, dtype=np.float32)
        stage_ok = False
        est_ee_sm = None
        last_est_joints_deg = None
        joint_slip = np.zeros(num_motor, dtype=np.float32)
        fault_tier = stage_idx // every_n_stages
        fault_prob = min(base_fault_prob + fault_tier *
fault_prob_step, max_fault_prob)
        fault_scale = min(base_fault_scale + fault_tier *
fault_scale_step, max_fault_scale)
        steps_executed = 0
        for step_idx in range(num_steps_per_stage):
            steps_executed += 1
            gt_ee = np.array(p.getLinkState(robot_id,
ee_link_index, computeForwardKinematics=True)[0], dtype=np.float32)
            if np.linalg.norm(target_pos - gt_ee) <=
current_tolerance:
                stage_ok = True
                break
            if step_idx % max(1, int(est_stride)) == 0:
                img1 = p.getCameraImage(100, 100, cam1_view,
cam1_proj, renderer=p.ER_TINY_RENDERER, shadow=0)
                img2 = p.getCameraImage(100, 100, cam2_view,
cam2_proj, renderer=p.ER_TINY_RENDERER, shadow=0)
                rgb1 = np.reshape(img1[2], (100, 100, 4))
[:, :, :3].astype(np.uint8)
                rgb2 = np.reshape(img2[2], (100, 100, 4))
[:, :, :3].astype(np.uint8)
                est_ee, est_joints_deg = estimate_ee_xyz(rgb1,
rgb2, model, device)
                est_ee_sm = est_ee if est_ee_sm is None else 0.6 *
est_ee_sm + 0.4 * est_ee
                last_est_joints_deg = est_joints_deg
                if last_est_joints_deg is None or est_ee_sm is None:
                    continue
                if np.random.random() < fault_prob:
                    fault_joint = np.random.randint(0, num_motor)
                    joint_slip[fault_joint] += np.random.normal(0.0,
fault_scale)
                    joint_slip[fault_joint] =
np.clip(joint_slip[fault_joint], -0.25, 0.25)
                    error = target_pos - est_ee_sm
                    delta = 0.25 * (error - prev_error) + 0.06 * error
                    delta = np.clip(delta, -0.01, 0.01)
                    current_commanded = current_commanded + delta
                    workspace_min = np.array([0.02, -0.20, 0.05],
dtype=np.float32)
                    workspace_max = np.array([0.30, 0.20, 0.25],
dtype=np.float32)

```



```

        current_commanded = np.clip(current_commanded,
workspace_min, workspace_max)
        rest = np.deg2rad(last_est_joints_deg)
        joint_targets = p.calculateInverseKinematics(robot_id,
ee_link_index, current_commanded, lowerLimits=lower.tolist(),
upperLimits=upper.tolist(), jointRanges=ranges.tolist(),
restPoses=rest.tolist(), jointDamping=[0.01] * num_motor,
residualThreshold=1e-4, maxNumIterations=100)
        joint_targets = np.array(joint_targets[:num_motor],
dtype=np.float32) + joint_slip
        joint_targets = np.clip(joint_targets, lower, upper)
        for i in range(num_motor):
            p.setJointMotorControl2(robot_id, i,
controlMode=p.POSITION_CONTROL,
targetPosition=float(joint_targets[i]), force=100)
        for _ in range(50):
            p.stepSimulation()
            prev_error = error.copy()
            successes.append(stage_ok)
            stage_logs.append({"stage": f"S{stage_idx + 1}", "target":
target_pos, "mode": factory_state["mode"], "steps": steps_executed,
"success": stage_ok})
            if stage_ok:
                factory_state["consecutive_failures"] = 0
            else:
                factory_state["consecutive_failures"] += 1
        finally:
            elapsed = time.time() - start
            p.disconnect()
        return stage_logs, successes, elapsed

def summarize_results(df, label):
    summary = df.groupby("variant").agg(
        success_rate=("success", "mean"),
        mean_steps=("steps", "mean"),
        median_steps=("steps", "median"),
        failures=("success", lambda s: int((~s).sum())),
    ).reset_index()
    summary["label"] = label
    return summary

model = load_model()
fault_config = {
    "num_robots": NUM_ROBOTS,
    "seed": SEED,
    "fault_seed": FAULT_SEED,
    "urdf_path": str(URDF_PATH),
    "model_path": str(MODEL_PATH),
    "device": str(DEVICE),

```

```

}
(OUT_DIR / "config.json").write_text(json.dumps(fault_config,
indent=2))

logger.info("Running baseline with fault injection...")
logs_bl, success_pre_bl, success_post_bl, time_bl =
run_single_robot_chain_faulty_baseline(URDF_PATH,
num_stages=NUM_ROBOTS, seed=SEED)
baseline_df = pd.DataFrame([
    {
        "trial": i + 1,
        "variant": "baseline",
        "pre_fault_success": bool(pre),
        "post_fault_success": bool(post),
        "steps": int(log["steps"]),
    }
    for i, (log, pre, post) in enumerate(zip(logs_bl, success_pre_bl,
success_post_bl))
])
baseline_df.to_csv(OUT_DIR / "baseline_fault.csv", index=False)

logger.info("Running NeuroKin with fault injection...")
logs_nk, success_nk, time_nk =
run_single_robot_chain_faulty_neurokin_v2(URDF_PATH, model, DEVICE,
num_stages=NUM_ROBOTS, seed=SEED)
neurokin_df = pd.DataFrame([
    {"trial": i + 1, "variant": "neurokin", "success": bool(ok),
"steps": int(log["steps"])}
    for i, (log, ok) in enumerate(zip(logs_nk, success_nk))
])
neurokin_df.to_csv(OUT_DIR / "neurokin_fault.csv", index=False)

baseline_summary = pd.DataFrame([
    {"variant": "baseline_pre_fault", "success_rate":
baseline_df["pre_fault_success"].mean(), "mean_steps":
baseline_df["steps"].mean(), "median_steps":
baseline_df["steps"].median(), "failures":
int((~baseline_df["pre_fault_success"]).sum())},
    {"variant": "baseline_post_fault", "success_rate":
baseline_df["post_fault_success"].mean(), "mean_steps":
baseline_df["steps"].mean(), "median_steps":
baseline_df["steps"].median(), "failures":
int((~baseline_df["post_fault_success"]).sum())},
])
neurokin_summary = pd.DataFrame([
    {"variant": "neurokin", "success_rate":
neurokin_df["success"].mean(), "mean_steps":
neurokin_df["steps"].mean(), "median_steps":
neurokin_df["steps"].median(), "failures":
int((~neurokin_df["success"]).sum())},

```

```

])
summary_df = pd.concat([baseline_summary, neurokin_summary],
ignore_index=True)
summary_df.to_csv(OUT_DIR / "summary_fault.csv", index=False)
print(summary_df)

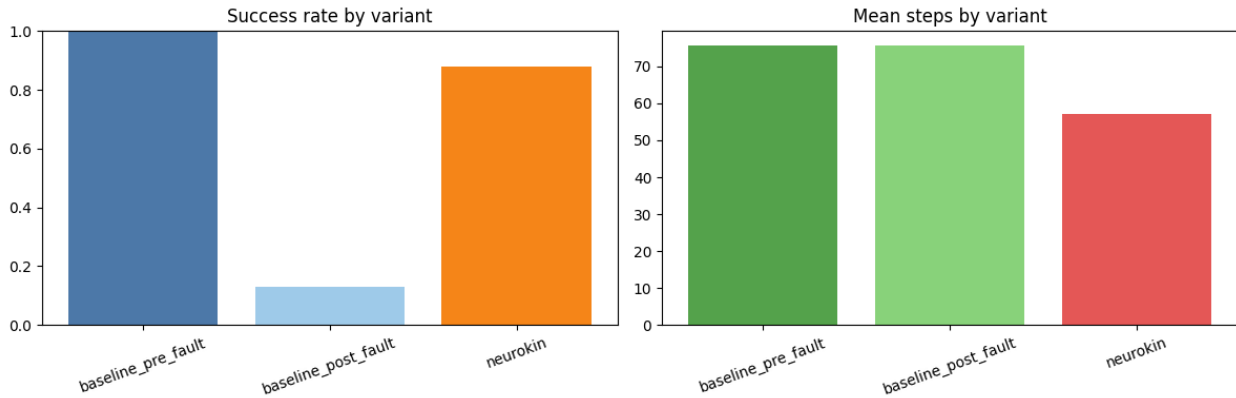
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].bar(summary_df["variant"], summary_df["success_rate"],
color=["#4c78a8", "#9ecae9", "#f58518"])
axes[0].set_ylim(0, 1)
axes[0].set_title("Success rate by variant")
axes[0].tick_params(axis="x", rotation=20)
axes[1].bar(summary_df["variant"], summary_df["mean_steps"],
color=["#54a24b", "#88d27a", "#e45756"])
axes[1].set_title("Mean steps by variant")
axes[1].tick_params(axis="x", rotation=20)
plt.tight_layout()
plt.savefig(OUT_DIR / "comparison_fault.png", dpi=150)
plt.show()

print(f"Baseline pre-fault success:
{baseline_df['pre_fault_success'].mean():.1%}")
print(f"Baseline post-fault success:
{baseline_df['post_fault_success'].mean():.1%}")
print(f"NeuroKin success: {neurokin_df['success'].mean():.1%}")

reload_check = pd.read_csv(OUT_DIR / "summary_fault.csv")
assert len(reload_check) == 3
print("Reload verification passed.")

```

	variant	success_rate	mean_steps	median_steps
failures				
0	baseline_pre_fault	1.00	75.79	48.5
0				
1	baseline_post_fault	0.13	75.79	48.5
87				
2	neurokin	0.88	57.04	34.5
12				



```
Baseline pre-fault success: 100.0%
Baseline post-fault success: 13.0%
NeuroKin success: 88.0%
Reload verification passed.
```

```
print(f"OUT_DIR: {OUT_DIR.resolve()}")
for path in OUT_DIR.glob("*.csv"):
    print(f"- {path.name}")

print("\nsummary_fault.csv head:")
print(pd.read_csv(OUT_DIR / "summary_fault.csv").head())

print("\nbaseline_fault.csv head:")
print(pd.read_csv(OUT_DIR / "baseline_fault.csv").head())

print("\nneurokin_fault.csv head:")
print(pd.read_csv(OUT_DIR / "neurokin_fault.csv").head())
```

```
OUT_DIR: /content/phase2_fault_100_injection
- neurokin_fault.csv
- summary_fault.csv
- baseline_fault.csv
```

```
summary_fault.csv head:
      variant  success_rate  mean_steps  median_steps
failures
0  baseline_pre_fault      1.00      75.79         48.5
0
1  baseline_post_fault      0.13      75.79         48.5
87
2                neurokin      0.88      57.04         34.5
12
```

```
baseline_fault.csv head:
   trial  variant  pre_fault_success  post_fault_success  steps
0      1  baseline                True                False    169
1      2  baseline                True                 True    201
```

2	3	baseline	True	True	201
3	4	baseline	True	True	201
4	5	baseline	True	True	201

neurokin_fault.csv head:

	trial	variant	success	steps
0	1	neurokin	True	27
1	2	neurokin	True	35
2	3	neurokin	True	35
3	4	neurokin	True	27
4	5	neurokin	True	35

```
import pandas as pd
import matplotlib.pyplot as plt

baseline_df = pd.read_csv(OUT_DIR / "baseline_fault.csv")
neurokin_df = pd.read_csv(OUT_DIR / "neurokin_fault.csv")

plt.figure(figsize=(10, 5))

# Changed plt.plot to plt.scatter to plot as points
plt.scatter(baseline_df['trial'][:50], baseline_df['steps'][:50],
            color='#e45756', alpha=0.8, label='Baseline IK')
plt.scatter(neurokin_df['trial'], neurokin_df['steps'],
            color='#4c78a8', label='NeuroKin')

plt.axhline(y=200, color='black', linestyle=':', label='Timeout
Limit')
plt.xlabel('Agent / Trial Index')
plt.ylabel('Steps to Convergence')
plt.title('Algorithm Efficiency Under Mechanical Degradation')
plt.legend()
plt.grid(True, alpha=0.3)
plt.savefig(OUT_DIR / "swarm_variance_plot.pdf", format="pdf",
bbox_inches="tight")
```

